

Synthetic Process คืออะไร?

สร้าง process ใหม่ที่ "กลั่นหาค่ากลาง" จาก 2 assets ที่เดิน random walk

ตัวอย่างหลัก: Bybit BTC Perp ↔ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

ราคา BTC ขึ้นลงไม่มีทิศทาง (random walk) แต่ถ้าเอาราคา BTC จากสองตลาดมาหักล้างกัน ด้วยอัตราส่วนที่เหมาะสม จะได้ process ใหม่ที่ **วนเวียนอยู่รอบค่ากลาง** — นั่นคือสิ่งที่เราจะ trade

$$\varepsilon = \log(P_A) - \beta \cdot \log(P_B)$$

1.1 ปัญหาของการ Trade Single Asset

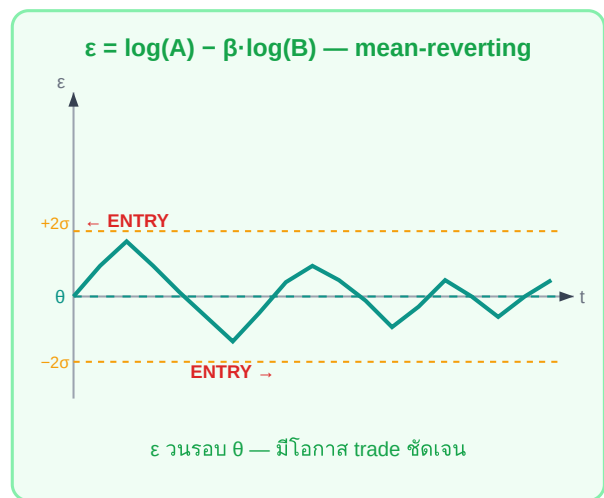
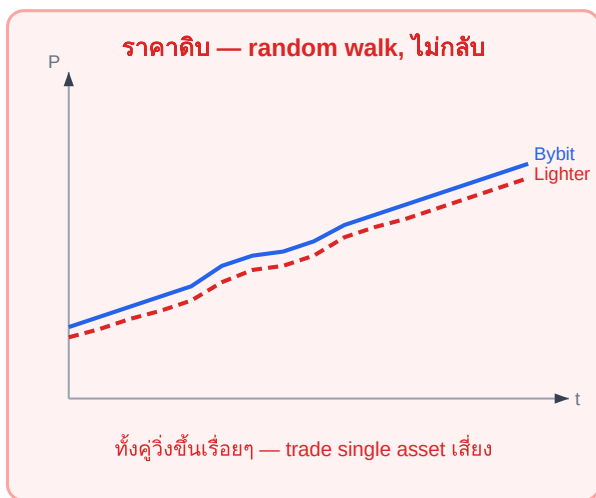
ลองเทียบสองกลยุทธ์:

กลยุทธ์ A — Short BTC เมื่อราคาสูง (อันตราย)

Short BTC ที่ \$65,000 เพราะ "แพงเกินไป" — แต่ราคาวิ่งไป \$90,000
ขาดทุน \$25,000 ต่อ BTC และ **ไม่มีเหตุผลว่าราคาจะ "กลับมา"** ที่ \$65,000
Random walk ไม่มีแรงดึงกลับ — ราคาอาจไปเรื่อยๆ

กลยุทธ์ B — Trade Spread ระหว่างสองตลาด (ปลอดภัยกว่า)

Bybit BTC perp ราคา \$65,050, Lighter BTC perp ราคา \$64,980
Spread = \$70 (สูงกว่าปกติที่ ~\$20)
Trade: Short Bybit, Long Lighter — รอให้ spread กลับมา \$20
แรงดึงกลับมีอยู่จริง: ทั้งสองตลาดต้อง track ราคา BTC เดียวกัน



ซ้าย: ราคาดิบทั้งสองตลาดวิ่งขึ้นพร้อมกัน ยากจะ time | ขวา: ε = ความต่าง กลับหาค่ากลางช้าๆ

1.2 สร้าง Synthetic Process: สูตรและส่วนประกอบ

สูตรหลัก

$$\epsilon = F(A) - \beta \cdot F(B)$$

$F(\cdot)$ = transformation (log price, basis, IV)

β = อัตราส่วนที่ทำให้ ϵ กลับหาค่ากลาง

ϵ = process ใหม่ที่เราจะ trade ("spread" หรือ "epsilon")

1.3 β คือ Entry Ratio — ไม่ใช่แค่ตัวเลขในสูตร

นี่คือจุดที่หลายคนเข้าใจผิดที่สุด: β บอกว่าต้อง trade ในอัตราส่วนเท่าไร เพื่อให้ ϵ ไม่ drift

ตัวอย่างเพื่อเห็นภาพ

สมมติ: A ขึ้น +3% ทุกครั้งที่ B ขึ้น +5% (A sensitive น้อยกว่า B)

ถ้า trade 1:1 $\rightarrow \epsilon$ เปลี่ยน = +3% - +5% = -2% ทุกครั้ง $\rightarrow \epsilon$ drift ลง (ไม่ดี)

ถ้า trade 1:0.6 $\rightarrow \epsilon$ เปลี่ยน = +3% - 0.6×5% = 3% - 3% = 0% ทุกครั้ง $\rightarrow \epsilon$ flat ✓

ดังนั้น $\beta = 3\%/5\% = 0.6$ คือ "อัตราส่วนที่ถูกต้อง"

ทำไม $\beta = 0.6$ ถึงทำให้ ϵ flat?

เมื่อตลาดขึ้น: A +3%, B +5%

A ขึ้น +3%

B ขึ้น +5%

ผลลัพธ์ ϵ ขึ้นอยู่กับ β ที่เลือก

$\beta = 1$ (ผิด — trade 1:1)

Long A: +3%

Short B: -1×5% = -5%

ϵ เปลี่ยน = 3% - 5% = -2% ↓ drift!

-2% drift

$\beta = 0.6$ (ถูก — trade 1:0.6)

Long A: +3%

Short 0.6×B: -0.6×5% = -3%

ϵ เปลี่ยน = 3% - 3% = 0% ✓ flat!

0% flat

สูตรหา β จากอัตราผลตอบแทน

$$\beta = \text{sensitivity(A)} / \text{sensitivity(B)} = 3\%/5\% = 0.6$$

ในทางปฏิบัติ หา β จาก OLS regression (บทที่ 4)

$\beta = 0.6$ หมายถึง: Long A 100,000 บาท $\rightarrow$ Short B เพียง 60,000 บาท (ไม่ใช่ 1:1)

ตาราง: β ต่างกัน $\rightarrow$ Position Size ต่างกัน

β	ถ้า Long A = \$100,000	ต้อง Short B เท่าไร	ความหมาย
$\beta = 1.00$	\$100,000	\$100,000	A และ B sensitive เท่ากันพอดี
$\beta = 0.60$	\$100,000	\$60,000	B volatile กว่า $\rightarrow$ short น้อยกว่า
$\beta = 1.50$	\$100,000	\$150,000	B stable กว่า $\rightarrow$ short มากกว่า
$\beta = 1.003$	\$100,000	\$100,300	Bybit $\leftrightarrow$ Lighter: ต่างนิดหน่อย แต่ไม่ใช่ 1:1

β — อ่านบน number line



$\beta = 1$ เป็นแค่กรณีพิเศษ ในทางปฏิบัติ β แทบไม่เคยเป็น 1 พอดี

1.4 ทำไม Bybit ↔ Lighter ถึงมี $\beta \neq 1$?

Running Example: Bybit BTCUSDT Perp ↔ Lighter BTCUSDT Perp

ราคาทั้งสองเกือบเท่ากัน (~\$65,000) แต่ $\beta \neq 1$ เพราะ:

- **Funding rate mechanism ต่างกัน:** Bybit ใช้ 8h, Lighter ใช้ 1h — ทำให้ราคา drift ต่างกันในช่วงสั้น
- **Liquidity depth ต่างกัน:** Bybit มี volume มากกว่า → ราคาตอบสนองต่อ order flow ต่างกัน
- **Execution latency ต่างกัน:** ราคาที่ fetch มาไม่ synchronous พอดี → ทำให้ sensitivity วัดได้ต่างกัน

สมมติ $\beta = 1.003$ จากการ calibrate → Long Bybit \$100,000 ต้อง Short Lighter \$100,300 → ต่างจาก 1:1 เพียง \$300 แต่สำคัญมากเมื่อ trade บ่อย

1.5 ϵ ที่ดี ต้องมีคุณสมบัติอะไร?

ไม่ใช่ทุก "spread" ที่จะ trade ได้ ϵ ที่ดีต้องผ่านเกณฑ์:

คุณสมบัติ	ความหมาย	วัดอย่างไร
Stationary	ϵ วนอยู่รอบค่ากลาง ไม่ drift ออกไปเรื่อยๆ	ADF test (บทที่ 5)
Mean-reverting	ยิ่งห่างจาก 0 มาก แรงดึงกลับยิ่งแรง	Half-life (บทที่ 3)
Half-life trade-able	กลับเร็วพอสำหรับ strategy ที่วางแผน	2h–2wk เหมาะสม
Spread profit > ค่าธรรมเนียม	σ_ϵ ใหญ่กว่า fee+slippage	Edge analysis (บทที่ 19)

1.6 รูปแบบ $F(\cdot)$ ที่ใช้บ่อย

$F(\cdot)$	สูตร ϵ	ใช้กับ
Log price	$\log(P_A) - \beta \cdot \log(P_B)$	Bybit $\leftrightarrow$ Lighter perp (เล่มนี้)
Absolute price	$P_A - \beta \cdot P_B$	ราคาระดับเดียวกัน, unit เดียว
Basis	$P_{\text{perp}} - P_{\text{spot}}$	Perpetual vs spot arbitrage
Implied Vol	$IV_A - \beta \cdot IV_B$	Options volatility surface arb
Calendar spread	$P_{\text{front}} - \beta \cdot P_{\text{back}}$	Futures term structure

ทำไมต้อง log price สำหรับ Bybit $\leftrightarrow$ Lighter?

ราคา BTC ระดับ \$65,000 $\rightarrow$ ถ้าใช้ $P_A - \beta \cdot P_B$ ตรงๆ spread จะเปลี่ยนตามระดับราคา (high price = large absolute spread)

$\log(P_A) - \beta \cdot \log(P_B)$ คือ log ratio $\rightarrow$ เป็น % deviation $\rightarrow$ ไม่ขึ้นกับระดับราคา $\rightarrow$ **stationary** กว่า

โจทย์ 1.1 — คำนวณ position size จาก β

Bybit BTC perp ขึ้น 2.1% ในช่วงที่แล้ว ขณะที่ Lighter BTC perp ขึ้น 2.3%
จาก calibration: $\beta = 0.913$

ถาม: ถ้า Long Bybit \$500,000 ต้อง Short Lighter เท่าไร?

► ดูคำตอบ

โจทย์ 1.2 — ϵ drift บอกอะไร?

trader ใช้ $\beta=1.0$ (แทนที่จะเป็น $\beta=0.913$) trade Bybit $\leftrightarrow$ Lighter มา 30 วัน
สังเกตว่า ϵ ค่อยๆ ลดลงทุกวัน แม้ไม่ได้เปิด position

ถาม: อธิบายว่าเกิดอะไรขึ้น และ drift นี้ส่งผลอย่างไรต่อ P&L ถ้า trader มักจะ short spread?

► ดูคำตอบ

โจทย์ 1.3 — เลือก $F(\cdot)$ ที่เหมาะสม

คุณต้องการ trade spread ระหว่าง BTC June futures (\$65,200) กับ BTC September futures (\$66,100)

ถาม: ควรใช้ $F(\cdot)$ แบบไหน และ spread นี้คือ "basis" ประเภทไหน?

► คำตอบ

STATISTICAL ARBITRAGE · บทที่ 2

Stochastic Calculus พื้นฐาน

Brownian Motion · $(dW)^2 = dt$ · Itô's Lemma

เครื่องมือที่จำเป็นสำหรับอ่าน SDE ของ spread

ทำไมต้องเรียนบทนี้

Spread ε เดิน SDE แบบ OU Process: $d\varepsilon = \kappa(\theta - \varepsilon)dt + \sigma dW$

เพื่ออ่านสมการนี้ได้ต้องรู้ว่า dW คืออะไร และทำไม $(dW)^2$ ถึงไม่ตัดทิ้ง

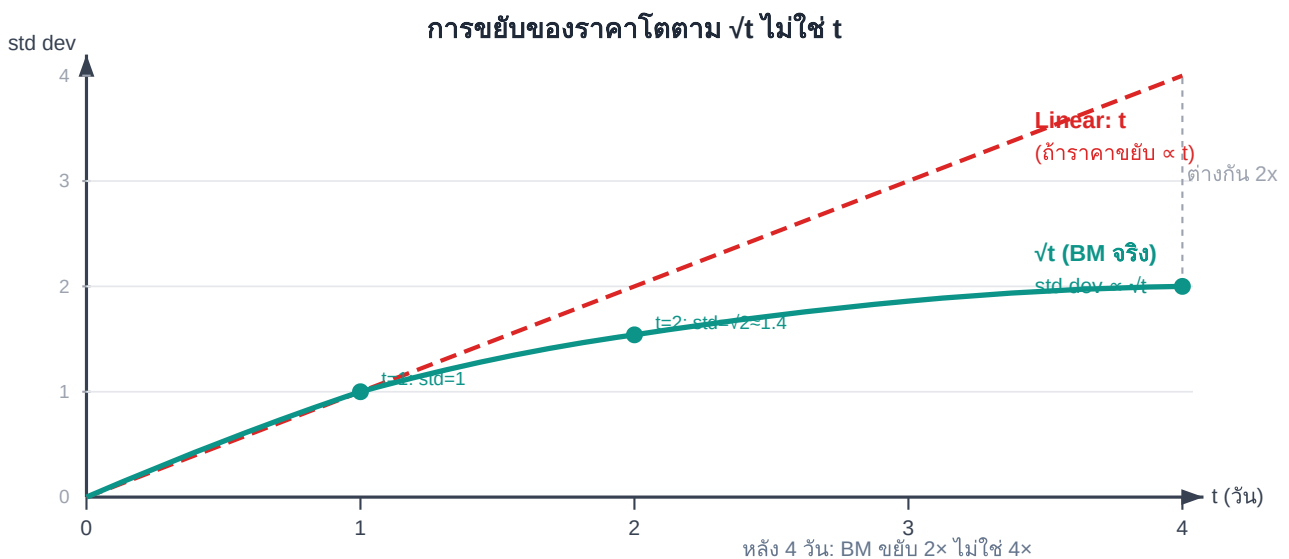
2.1 Brownian Motion — ราคาเดิน Random Walk อย่างไร

Brownian Motion (หรือ Wiener Process) $W(t)$ คือโมเดลที่ใช้อธิบายการเคลื่อนที่แบบสุ่มของราคา มีคุณสมบัติหลัก 3 ข้อ:

3 คุณสมบัติสำคัญของ $W(t)$

1. $W(0) = 0$: เริ่มต้นที่จุดเดิม
2. **Increments อิสระ**: การขยับในช่วง $[s, t]$ ไม่ขึ้นกับการขยับในช่วง $[r, s]$ — "อดีตไม่ส่งผลต่ออนาคต"
3. $W(t) - W(s) \sim N(0, t-s)$: ในช่วงเวลา $\Delta t = t-s$ ราคาขยับประมาณ $\pm\sqrt{\Delta t}$ (ไม่ใช่ $\pm\Delta t$)

ข้อ 3 สำคัญมาก: ราคาขยับตาม $\sqrt{t}$ ไม่ใช่ t



Brownian Motion โต "ช้ากว่าที่คิด" — ต้องรอ 4 วันจึงจะขยับ 2 เท่าของวันเดียว

2.2 dW คืออะไร และทำไม $(dW)^2 = dt$

เราใช้ dW_t แทน $W(t+dt) - W(t)$ — การขยับเล็กๆ ในช่วงเวลา dt

$dW_t \sim N(0, dt)$ $E[dW_t] = 0$ ← ค่าเฉลี่ยเป็นศูนย์ (ไม่รู้จะขึ้นหรือลง) $Var[dW_t] = dt$ ← variance = dt

ทำไม $(dW)^2 = dt$ — ไม่ตัดทิ้งแบบ Calculus ธรรมดา?

Calculus ธรรมดา: $(dx)^2 \rightarrow 0$

ถ้า x เดินแบบ smooth: $dx = v \cdot dt$

$(dx)^2 = v^2 \cdot (dt)^2 \rightarrow$ ตัดทิ้งได้ เพราะ dt^2 เล็กมาก

เช่น $dt = 0.001 \rightarrow dt^2 = 0.000001 \approx 0$

Stochastic: $(dW)^2 = dt$ (ตัดไม่ได้!)

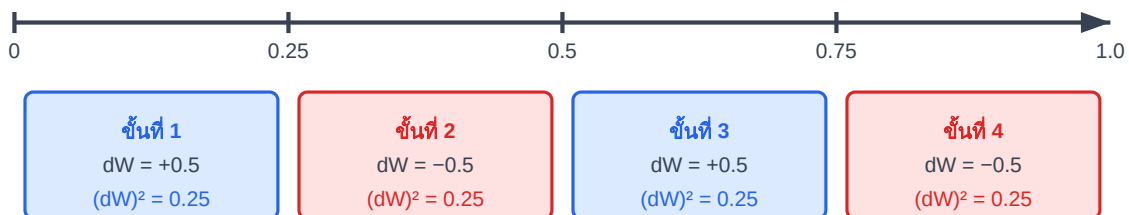
W เดินแบบ random: $dW \approx \pm\sqrt{dt}$

$(dW)^2 \approx (\pm\sqrt{dt})^2 = dt \rightarrow$ ไม่ใช่ dt^2 แต่เป็น dt

เช่น $dt = 0.001 \rightarrow (dW)^2 \approx 0.001 \neq 0$

ทำไม $(dW)^2 = dt$ — ดูจากตัวเลข

ช่วงเวลา $T=1$ แบ่งเป็น $n=4$ ขั้น ($dt = 0.25$ แต่ละขั้น)



$$\Sigma (dW)^2 = 0.25 + 0.25 + 0.25 + 0.25 = 1.0 = T$$

สรุป: ไม่ว่า dW จะเป็น + หรือ - เมื่อยกกำลังสอง จะได้ dt เสมอ

เพราะ $dW = \pm\sqrt{dt} \rightarrow (dW)^2 = (\sqrt{dt})^2 = dt$

$\Sigma (dW)^2 \approx n \times dt = n \times (T/n) = T \rightarrow$ สะสมเป็นค่าจริง ไม่หายไป

เปรียบ: ราคา BTC ขึ้นลงเดี๋ยวนี้ แต่ความเปลี่ยนแปลงสะสมทุกนาที $\rightarrow$ ตัดทิ้งไม่ได้

สัญชาตญาณ: dW ขยับสลับ +/- แต่ $(dW)^2$ เป็นบวกเสมอ $\rightarrow$ สะสมได้ $\rightarrow$ ไม่ตัดทิ้ง

2.3 Stochastic Differential Equation (SDE)

SDE คือสมการที่บอกว่า process X เปลี่ยนอย่างไรในแต่ละช่วง dt :

รูปแบบ SDE ทั่วไป

$$dX_t = \mu(X, t) dt + \sigma(X, t) dW_t$$

แยกส่วนของ SDE

dX_t การเปลี่ยน X ใน dt	=	$\mu(X, t) \cdot dt$ Drift — ทิศทาง ที่คาดได้ล่วงหน้า	+	$\sigma(X, t) \cdot dW_t$ Diffusion — ความสุม ที่คาดไม่ได้
-------------------------------------	---	---	---	--

ตัวอย่าง SDE ที่สำคัญ

GBM (Black-Scholes):

$$\mu = \mu_S, \sigma = \sigma_S$$

OU Process (Spread):

$$\mu = \kappa(\theta - \varepsilon), \sigma = \sigma$$

Random Walk:

$$\mu = 0, \sigma = \sigma$$

OU Process: drift = $\kappa(\theta - \varepsilon)$ → ดึงกลับเมื่อ ε ห่างจาก θ | Random Walk: drift = 0 → ไม่มีแรงดึง

2.4 Itô's Lemma — Chain Rule ของ Stochastic World

ถ้าเราต้องการหา SDE ของ $f(X)$ เมื่อรู้ SDE ของ X ใช้ Itô's Lemma:

ITÔ'S LEMMA

$$df = (\partial f / \partial t + \mu \cdot \partial f / \partial X + \frac{1}{2} \sigma^2 \cdot \partial^2 f / \partial X^2) dt + \sigma \cdot \partial f / \partial X \cdot dW$$

เทียบกับ Chain Rule ธรรมดา ทีละขั้น

#	Calculus ธรรมดา	Itô's Lemma (Stochastic)	เหตุผลที่ต่างกัน
1	$df = (\partial f / \partial t)dt + (\partial f / \partial x)dx$	เริ่มเหมือนกัน	—
2	แทน $dx = \mu dt + \sigma dW$	แทน $dX = \mu dt + \sigma dW$	—
3	คิด Taylor: $(dx)^2 \rightarrow 0$ ตัดทิ้ง	คิด Taylor: $(dX)^2 = \sigma^2 dt$ ไม่ตัด!	$(dW)^2 = dt$ ไม่ใช่ 0
4	$df = (\partial f / \partial t + \mu \partial f / \partial x)dt + \sigma \partial f / \partial x dW$	$df = (\partial f / \partial t + \mu \partial f / \partial X + \frac{1}{2} \sigma^2 \partial^2 f / \partial X^2)dt + \sigma \partial f / \partial X dW$	Term พิเศษ $\frac{1}{2} \sigma^2 (\partial^2 f / \partial X^2) dt$

Term พิเศษ $\frac{1}{2} \sigma^2 (\partial^2 f / \partial X^2) dt$ มาจากไหน?

Taylor expansion ของ $f(X+dX)$: $f(X+dX) = f(X) + f' \cdot dX + \frac{1}{2} f'' \cdot (dX)^2 + \dots$

ใน calculus ธรรมดา: $(dX)^2 = (\mu dt + \sigma dW)^2 \approx \sigma^2 (dW)^2 \rightarrow$ ถ้า $(dW)^2 = 0$ ก็ตัดทิ้ง

ใน stochastic: $(dW)^2 = dt \rightarrow \frac{1}{2} f'' \cdot \sigma^2 \cdot dt$ **ไม่หายไป** $\rightarrow$ กลายเป็น term ใหม่ในสมการ

ตัวอย่างสำคัญ: $\log(S)$ เมื่อ $S \sim \text{GBM}$

1

กำหนด: $dS = \mu S dt + \sigma S dW$, $f(S) = \log(S)$

$$\partial f / \partial S = 1/S, \partial^2 f / \partial S^2 = -1/S^2$$

2

แทนใน Itô's Lemma:

$$d(\log S) = [\mu S \cdot (1/S) + \frac{1}{2} (\sigma S)^2 \cdot (-1/S^2)] dt + \sigma S \cdot (1/S) dW$$

$$= [\mu - \frac{1}{2} \sigma^2] dt + \sigma dW$$

ความหมาย: $\log(S)$ มี drift = $\mu - \frac{1}{2}\sigma^2$ (ไม่ใช่ μ)

Term $-\frac{1}{2}\sigma^2$ คือ "Itô correction" — เกิดจาก Jensen's inequality บนฟังก์ชัน concave $\log$

Itô correction สำคัญสำหรับ Stat Arb อย่างไร?

ถ้าใช้ log price spread: $\varepsilon = \log(P_A) - \beta \cdot \log(P_B)$

drift ของ ε จาก Itô = $(\mu_A - \frac{1}{2}\sigma_A^2) - \beta(\mu_B - \frac{1}{2}\sigma_B^2)$

ถ้าไม่รวม Itô correction → ประเมิน drift ผิด → เลือก β ผิด → ε drift ไม่หาย

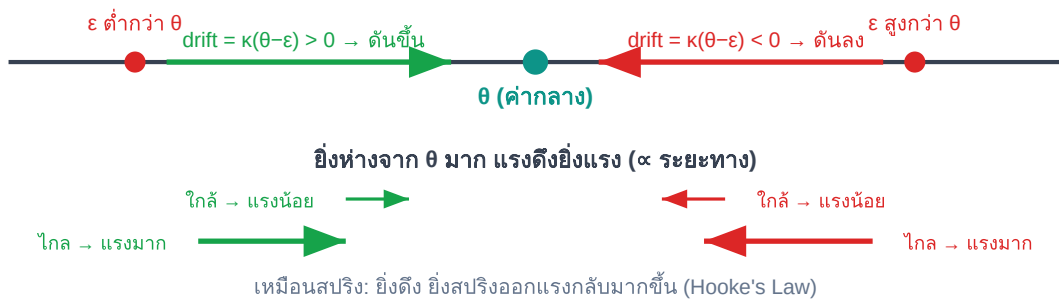
2.5 Preview: OU Process — SDE ที่เราต้องการ

สิ่งที่เราต้องการสำหรับ spread ε คือ SDE ที่มี drift "ดึงกลับ" เมื่อห่างจากค่ากลาง:

ORNSTEIN-UHLENBECK PROCESS

$$d\varepsilon = \kappa(\theta - \varepsilon) dt + \sigma dW$$

drift ของ OU Process: $\kappa(\theta - \varepsilon)$



OU drift ทำงานเหมือนสปริง — ยิ่งดึง spread ออกไปไกล แรงดึงกลับยิ่งมาก

โจทย์ 2.1 — $(dW)^2 = dt$

สมมติแบ่งช่วงเวลา $T=2$ ชั่วโมง เป็น $n=8$ ชั้น ($dt = 0.25$ ชม.) และ $\sigma = 0.1\%$ ต่อ $\sqrt{\text{ชม.}}$.

ถาม: (a) dW แต่ละชั้นมี std dev เท่าไร? (b) $\sum (dW)^2$ ทั้งหมดมีค่าเท่าไร (โดย E)?

► คำตอบ

โจทย์ 2.2 — Itô's Lemma

กำหนด $dX = 3 dt + 2 dW$ และ $f(X) = X^2$

ถาม: หา df โดยใช้ Itô's Lemma และบอกว่า Itô correction เท่าไร

► คำตอบ

โจทย์ 2.3 — อ่าน OU drift

Spread ε เดิน OU: $d\varepsilon = 2(0.001 - \varepsilon) dt + 0.003 dW$

ขณะนี้ $\varepsilon = 0.005$

ถาม: drift ตอนนี้อยู่ที่เท่าไร (ต่อวัน) และบอกทิศทางและขนาด

► **ดูคำตอบ**

Ornstein-Uhlenbeck Process

หัวใจของ Stat Arb — โมเดลที่อธิบายว่า spread "วนกลับ" อย่างไร
 κ (speed) · θ (mean) · σ (noise) · Half-life · Calibration

แก่นของบทนี้ — อ่านก่อน

OU Process คือ "สปริงที่มี noise" — ยิ่งดึง spread ออกจากค่ากลาง สปริงยิ่งดึงกลับแรง

Half-life บอกว่า spread ใช้เวลากลับครึ่งทางนานแค่ไหน → กำหนดว่าเราจะ trade แบบ intraday หรือ swing

$$d\varepsilon = \kappa(\theta - \varepsilon) dt + \sigma dW$$

3.1 Parameters ของ OU Process

Parameter	ชื่อ	ความหมาย trade	ค่าปกติ
$\kappa > 0$	Mean-reversion speed	ยิ่งมาก spread กลับเร็ว → ต้อง execute เร็ว	1–5 ต่อวัน
θ	Long-run mean	ค่ากลางที่ entry/exit อิงอยู่ — ถ้าเปลี่ยนบ่อยต้อง re-calibrate	≈ 0 หรือ avg funding diff
$\sigma > 0$	Diffusion (noise)	ยิ่งมาก spread ผันผวนมาก → threshold ต้อง กว้างกว่า fee	0.01–0.5% ต่อ $\sqrt{\text{วัน}}$

3.2 OU Solution — ε เดินไปไหนได้บ้าง

SDE นี้ solve ได้ closed-form ทำให้เราคำนวณ expected value ได้ทุกเวลา:

$\varepsilon_t = \theta + (\varepsilon_0 - \theta) \cdot e^{-\kappa t} + \sigma \cdot \int_0^t e^{-\kappa(t-s)} dW_s$ ↑ ↑ ↑ ค่ากลาง decay จากจุดเริ่ม noise สะสม

$$E[\varepsilon_t | \varepsilon_0] = \theta + (\varepsilon_0 - \theta) \cdot e^{-\kappa t}$$

เริ่มที่ $\varepsilon_0 = \theta + \text{gap}_0$ (ห่างจากค่ากลาง gap_0)

หลังเวลา t : ε คาดว่าจะอยู่ที่ $\theta + \text{gap}_0 \cdot e^{-\kappa t}$

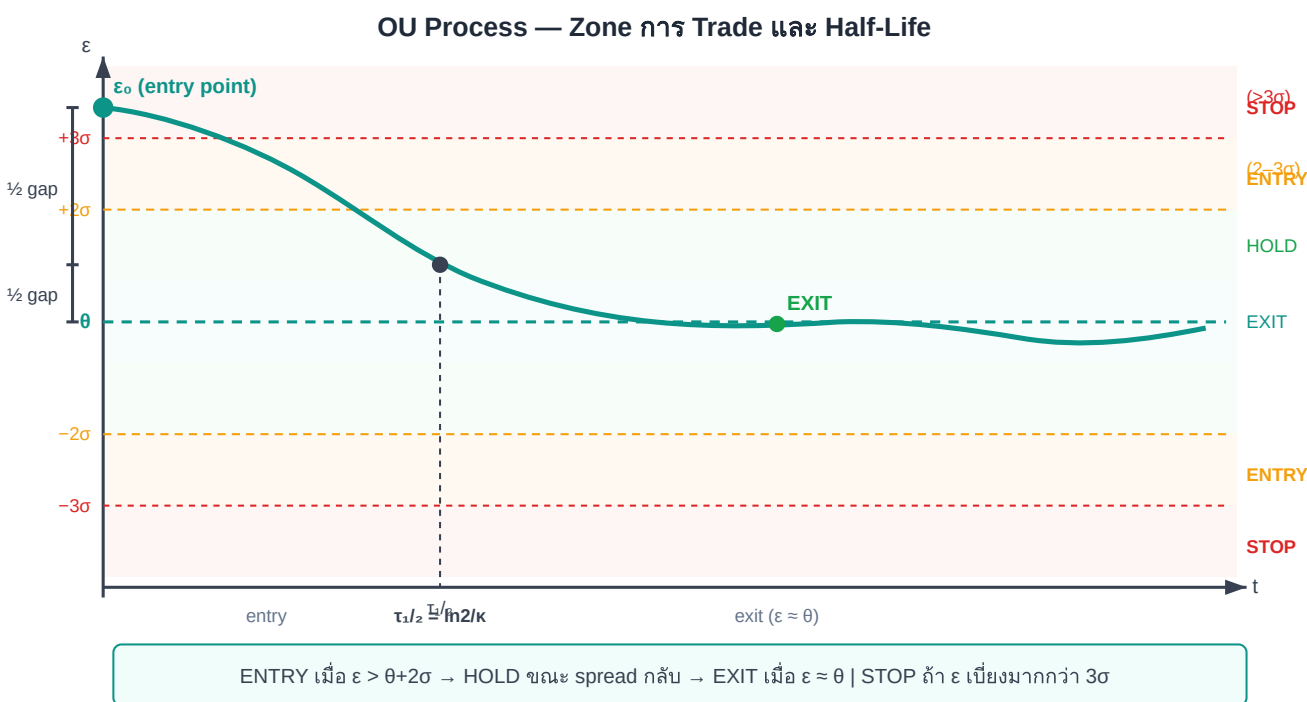
gap ลดลงด้วยอัตรา $e^{-\kappa t} \rightarrow$ ยิ่งนาน ยิ่งใกล้ θ

3.3 Half-Life — ตัวเลขที่สำคัญที่สุดสำหรับ Trader

HALF-LIFE

$$\tau_{1/2} = \ln(2) / \kappa \approx 0.693 / \kappa$$

เวลาที่ gap ระหว่าง ε และ θ ลดลงครึ่งหนึ่ง
ถ้า spread เปียง 1% จาก $\theta \rightarrow$ หลัง $\tau_{1/2}$ จะเหลือ 0.5%



Half-life = ช่วงเวลาที่ spread เดินทางครึ่งทางกลับ — ใช้วางแผน expected holding time

Half-life กับประเภท Strategy

κ (ต่อวัน)	Half-life	ประเภท Trade	ข้อกำหนด Execution
0.1	~7 วัน	Position trade / swing	ไม่เร่ง ต้องมี margin buffer
0.5	~33 ชม.	Swing ข้ามวัน	Monitor วันละครั้ง
1–2	8–17 ชม.	Intraday (เหมาะสม)	Check ทุก 1–2 ชม.
5	~3 ชม.	Fast intraday	Algo-assisted execution

10+	<2 ชม.	Near HFT	ต้องการ co-location, API
-----	--------	----------	--------------------------

3.4 Stationary Distribution — Band ของ Trade

เมื่อ $t \rightarrow \infty$ OU process เข้าสู่ stationary distribution ซึ่งเป็น Normal:

$$\varepsilon_{\infty} \sim N(\theta, \sigma^2/(2\kappa)) \quad \sigma_{\varepsilon} = \sigma/\sqrt{2\kappa} \leftarrow \text{stationary std dev (ใช้ตั้ง entry band)}$$

σ_{ε} ใช้ทำอะไร?

- Entry threshold: $\varepsilon > \theta + 2\sigma_{\varepsilon}$ หรือ $\varepsilon < \theta - 2\sigma_{\varepsilon}$ (เกิดราว 5% ของเวลา)
- Exit threshold: $\varepsilon \approx \theta \pm 0.5\sigma_{\varepsilon}$
- Stop-loss: $|\varepsilon - \theta| > 3\sigma_{\varepsilon}$ (spread "หลุด" ผิดปกติ)
- ตรวจสอบ: ε ควรอยู่ในช่วง $\theta \pm 3\sigma_{\varepsilon}$ กว่า 99.7% ของเวลา — ถ้าไม่ใช่ = signal ให้ re-calibrate

3.5 AR(1): OU Process ในโลกข้อมูล Discrete

ข้อมูลจริงเป็น discrete (รายนาที รายชั่วโมง) เมื่อแปลง OU เป็น AR(1):

OU → AR(1) (TIME STEP Δt)

$$\varepsilon_t = a + b \cdot \varepsilon_{t-1} + \eta_t$$

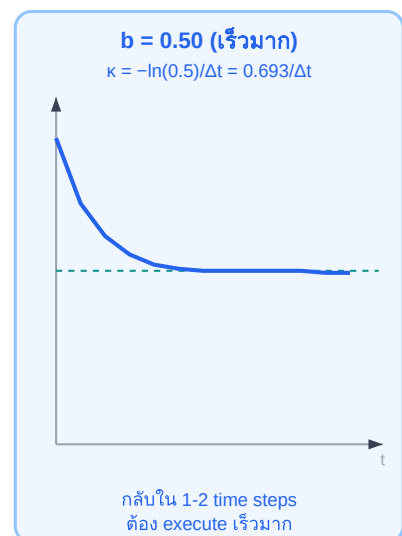
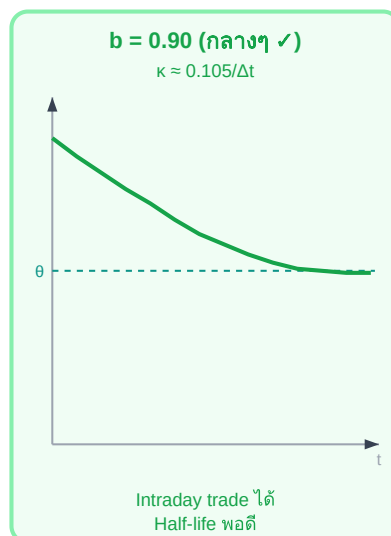
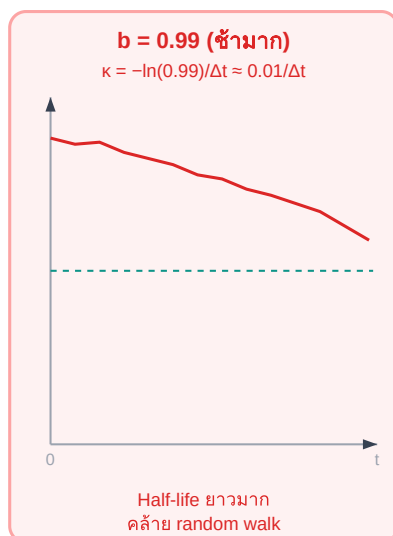
$b = e^{-\kappa \Delta t}$ ← mean-reversion coefficient

$a = \theta(1-b)$ ← intercept

$\eta_t \sim N(0, \sigma^2(1-b^2)/(2\kappa))$ ← noise

อ่าน b บน Scale — b บอกอะไร?

ค่า b ใน AR(1) บอกความเร็ว mean-reversion



b → 1.0 (ช้า, random walk)

b ≈ 0.9 (sweet spot)

b → 0 (เร็วมาก, white noise)

$b \approx 0.9-0.95$ ต่อ time step มักเป็น sweet spot สำหรับ intraday stat arb

สูตรแปลง AR(1) → OU PARAMETERS

$$\kappa = -\ln(b) / \Delta t \quad \theta = a / (1 - b)$$

ตัวอย่าง: $b = 0.90$, $\Delta t = 1$ ชม. = $1/24$ วัน

$\kappa = -\ln(0.90) / (1/24) \rightarrow -\ln(0.90) = 0.1054 \rightarrow 0.1054 \times 24 = 2.53$ ต่อวัน

Half-life = $\ln(2) / 2.53 = 0.274$ วัน = 6.6 ชม.

3.6 Calibration — หา κ , θ , σ จากข้อมูลจริง

เราเพิ่งเรียนรู้ว่า AR(1) ที่ fit บน spread ε_t ก็คือ OU process ในรูปแบบ discrete — ดังนั้นวิธีหา κ , θ , σ จากข้อมูลจริงคือ **fit AR(1) แล้วแปลงค่า** ตามขั้นตอนนี้:

1

สร้าง spread series: $\varepsilon_t = \log(P_{\text{Bybit}_t}) - \beta \cdot \log(P_{\text{Lighter}_t})$

(β ได้จาก OLS บน log returns — บทที่ 4)

2

Regress AR(1): $\varepsilon_t = a + b \cdot \varepsilon_{t-1} + \eta_t$

บันทึก: a , b , และ residuals η

3

คำนวณ OU parameters:

$\theta = a/(1-b)$, $\kappa = -\ln(b)/\Delta t$, $\sigma_{\text{stat}} \approx \text{std}(\varepsilon - \theta)$

4

คำนวณ trade parameters:

$\tau_{1/2} = \ln(2)/\kappa \rightarrow$ กำหนด holding time

Entry band = $\theta \pm 2\sigma_{\text{stat}} \rightarrow$ กำหนด entry threshold

Pseudo-code: OU Calibration

```
function calibrate_ou(eps_series, delta_t):
```

```
    # Step 1: AR(1) regression
```

```
    y = eps_series[1:] #  $\varepsilon_t$ 
```

```
    x = eps_series[:-1] #  $\varepsilon_{t-1}$ 
```

```
    b, a = ols_slope_intercept(x, y)
```

```
    residuals = y - (a + b * x)
```

```
    # Step 2: OU parameters
```

```
    theta = a / (1 - b)
```

```
    kappa = -log(b) / delta_t
```

```
    sigma_stat = std(eps_series - theta)
```

```
    half_life = log(2) / kappa
```

```
    # Step 3: Trade bands
```

```
    entry_upper = theta + 2 * sigma_stat
```

```
    entry_lower = theta - 2 * sigma_stat
```

```
    stop_upper = theta + 3 * sigma_stat
```

```
    stop_lower = theta - 3 * sigma_stat
```

```
return {theta, kappa, sigma_stat, half_life,  
        entry_upper, entry_lower, stop_upper, stop_lower}
```

ตัวอย่างจริง: Bybit ↔ Lighter BTC — Calibration ผล

ข้อมูล: 30 วัน, รายนาที่ ($\Delta t = 1/1440$ วัน) $\varepsilon_t = \log(P_{\text{Bybit}}) - 1.003 \cdot \log(P_{\text{Lighter}})$ OLS AR(1) result: $b = 0.9985$, $a = 2.0 \times 10^{-6}$ Residual std = 0.00012 OU Parameters: $\theta = 2.0e-6 / (1-0.9985) = 0.00133$ (basis $\approx 0.133\%$) $\kappa = -\ln(0.9985)/(1/1440) = 2.16$ ต่อวัน $\sigma_{\text{stat}} \approx 0.00080$ ($\approx 0.08\%$) Trade Parameters: $\tau_{1/2} = 0.693/2.16 = 7.7$ ชั่วโมง ← intraday trade ได้ Entry: $\varepsilon > 0.293\%$ หรือ $\varepsilon < -0.027\%$ ($\theta \pm 2 \times 0.08\%$) Stop: $\varepsilon > 0.373\%$ หรือ $\varepsilon < -0.107\%$ ($\theta \pm 3 \times 0.08\%$)

สรุป: spread กลับใน ~8 ชม. เปิด position ตอนเข้า → ปิดก่อนค่ำ

3.7 เมื่อไร OU เดียวไม่พอ — และบทถัดไปจะแก้อย่างไร

สมมติฐาน OU	เมื่อไรที่ผิด	สัญญาณที่เห็น	โมเดลที่ใช้แทน
κ, θ คงที่	ตลาด regime change: funding rate เปลี่ยนโครงสร้าง	θ drift ออก, ε ไม่กลับ	Kalman (Ch.4), Regime-Switch (Ch.9)
σ คงที่	ช่วง volatile: market crash, news	residual variance พุ่ง	GARCH on residuals (Ch.7)
Gaussian noise	liquidation cascade, news spike	residual มี fat tail, jump	Jump-Diffusion OU (Ch.8)
β คงที่	ตลาดเปลี่ยน correlation	ε drift กลับมา	Kalman Filter β (Ch.4)

Rule of thumb: เมื่อไรต้อง re-calibrate?

- θ เคลื่อนออกจากค่าเดิม $> 1\sigma_{\text{stat}}$ ใน 5 วัน $\rightarrow$ re-calibrate β และ θ
- Residual variance พุ่ง $> 2\times$ baseline $\rightarrow$ switch ไป GARCH
- Half-life ยาวขึ้น $> 2\times$ $\rightarrow$ regime เปลี่ยน พิจารณา exit ทั้งหมด

3.8 ตารางสัญลักษณ์ (Notation Reference) — ใช้ตลอดทั้งเล่ม

ทั้งเล่มใช้ σ หลายแบบ ต่างกันที่จุดประสงค์และวิธีคำนวณ — ตารางนี้เป็นอ้างอิงกลางที่ควรกลับมาดูทุกครั้งที่สับสน:

สัญลักษณ์	ชื่อเต็ม	ความหมาย	คำนวณจาก	ใช้ใน
σ (SDE)	Diffusion coefficient	ขนาด noise ใน OU SDE: $d\varepsilon = \kappa(\theta - \varepsilon)dt + \sigma \cdot dW$	$\sigma = \sigma_{\text{stat}} \cdot \sqrt{(2\kappa)}$	Ch.2–3 สมการทฤษฎี
σ_{stat}	Stationary std	Std ของ spread ε ณ steady state — ใช้กำหนด entry/exit band	$\sigma_{\text{stat}} = \sigma / \sqrt{(2\kappa)} \approx \text{std}(\varepsilon - \theta)$	Ch.3–4 entry band $\theta \pm 2\sigma_{\text{stat}}$
σ_t (GARCH)	Conditional std	Std ของ spread ε ณ เวลา t — ขยายตัวช่วง volatile	$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot \sigma_{t-1}^2$	Ch.7 GARCH z-score
σ_{robust}	Robust std (MAD-based)	Estimate σ_{stat} ที่ทนต่อ outlier — ไม่กระทบจาก jump	$\sigma_{\text{robust}} = \text{MAD} \times 1.4826$	Ch.10 robust z-score
σ_{EMA}	EMA-smoothed std	Rolling std ด้วย exponential weighting — Welford-style	Welford online update (Ch.15)	Ch.15 dynamic β tracking

กฎง่าย ๆ ในการเลือก σ

- ตลาดปกติ: ใช้ σ_{stat} (OLS residual std) — เร็วและง่าย
- ช่วง volatile (ratio 7d/30d std > 1.5): เปลี่ยนเป็น σ_t จาก GARCH(1,1)
- มี outlier / jump: ใช้ $\sigma_{\text{robust}} = \text{MAD} \times 1.4826$ แทน std()
- β เปลี่ยนตาม Kalman: ε เปลี่ยนทุก bar $\rightarrow$ ใช้ σ_{EMA} แบบ Welford update

สัญลักษณ์อื่น ๆ	ความหมาย
ε_t	Spread residual at time t : $\varepsilon = \log(P_A) - \beta \cdot \log(P_B)$
θ	Long-run mean ของ OU (equilibrium spread level)
κ	Mean-reversion speed (ต่อวัน); $\tau_{1/2} = \ln(2)/\kappa$
β	Hedge ratio (OLS, TLS, หรือ Kalman)

H	Hurst exponent; $H < 0.5$ = mean-reverting
z_t	Z-score ของ spread: $(\varepsilon_t - \theta)/\sigma_{\text{stat}}$ หรือ $(\varepsilon_t - \theta)/\sigma_t$
$\tau_{1/2}$	Half-life ของ mean-reversion: $\tau_{1/2} = \ln(2)/k$
Δt	Bar size ในหน่วยวัน (1-hour bar: $\Delta t = 1/24$)

3.9 Walk-Forward Calibration — ป้องกัน Overfitting

แก้ความคิด — อ่านก่อน

การ backtest แบบ **Static Window** คือดู historical data ครึ่งเดียว → รู้อนาคตโดยไม่รู้ตัว (**lookahead bias**)

Walk-Forward แก้ปัญหานี้โดยแบ่งเวลาแบบ rolling ลำดับเสมอ — เราใช้ข้อมูลอดีตเพื่อ fit แล้วทดสอบบนข้อมูลอนาคตที่โมเดลยังไม่เคยเห็น

Concept Drift — ทำไม Parameters เปลี่ยนตลอดเวลา

OU parameters (κ , θ , σ) ไม่คงที่ตลอดเวลา ตลาดเปลี่ยนแปลงตามสภาวะเศรษฐกิจ ความเชื่อมั่นของนักลงทุน และโครงสร้าง funding rate — สิ่งนี้เรียกว่า **Concept Drift**

- **Static window:** fit OU บนข้อมูลทั้งหมดครึ่งเดียว → parameters เป็นค่าเฉลี่ยของทุกช่วงเวลา → overfits กับช่วงอดีตที่ตลาดอาจไม่มีอีกแล้ว
- **Walk-Forward:** fit OU บน window อดีต → apply บน window อนาคตที่ยังไม่เคยเห็น → สะท้อน parameters ของตลาด ณ ช่วงนั้นจริง ๆ

อันตรายของ Static Calibration

- θ ที่ fit ได้อาจเป็นค่าเฉลี่ยของ regime หลายช่วงที่ไม่เคยเกิดพร้อมกัน
- κ ที่ได้อาจสูงเกินจริง เพราะ data ในช่วง volatile ดูเหมือน mean-revert เร็ว
- เมื่อ trade จริง parameters เหล่านั้นไม่ตรงกับตลาดปัจจุบัน → signal ผิด → ขาดทุน

Walk-Forward Algorithm — 3 ขั้นตอนหลัก

1

แบ่ง Window: Train window = 90 วัน, Test window = 30 วัน

เริ่มที่วันที่ 1–90 เป็น train, วันที่ 91–120 เป็น test — ห้าม shuffle หรือ random split เด็ดขาด ต้องเรียงตามเวลาเสมอ

2

Fit แล้ว Apply: Calibrate OU parameters บน train window → นำ parameters ไป generate signals บน test window → บันทึก PnL ของ test window

ไม่มี lookahead: test window ถูก "เปิดเผย" ทีละ bar ไม่ใช่ทั้งหมด

เลื่อน Window: ขยับ window ไป 30 วัน (= test_days) → วันซ้ำขั้นตอน 1–2

แต่ละรอบให้ PnL 1 window — รวมทุก window ได้ equity curve ที่ไม่มี lookahead

Walk-Forward — Rolling Windows ตามเวลา



แต่ละ train block (สีน้ำเงิน) ให้ parameters → apply บน test block (สีเขียว) ที่อยู่ถัดไปในอนาคต — เลื่อนซ้ำตลอดช่วงข้อมูล

Pseudo-code — Walk-Forward Calibration

```
def walk_forward_calibration(prices_A, prices_B, train_days=90, test_days=30):
    results = []
    i = 0
    while i + train_days + test_days <= len(prices_A):
        train_A = prices_A[i : i+train_days]
        train_B = prices_B[i : i+train_days]
        test_A = prices_A[i+train_days : i+train_days+test_days]
        test_B = prices_B[i+train_days : i+train_days+test_days]
        # Fit OU parameters on train
        beta = estimate_beta_ols(train_A, train_B)
        epsilon_train = log(train_A) - beta * log(train_B)
        kappa, theta, sigma_stat = calibrate_ou(epsilon_train)
        # Apply on test (no lookahead)
        epsilon_test = log(test_A) - beta * log(test_B)
        signals = generate_signals(epsilon_test, kappa, theta, sigma_stat)
        pnl = simulate_pnl(signals, test_A, test_B)
        results.append({"window_start": i, "beta": beta, "kappa": kappa, "pnl": pnl})
        i += test_days # slide forward
    return results
```

จุดสำคัญที่ต้องระวัง

- $i += \text{test_days}$ — เลื่อน window ด้วย test_days ไม่ใช่ 1 วัน เพื่อให้แต่ละ test window ไม่ overlap กัน
- Train window ขนาด 90 วัน ต้องมี data เพียงพอสำหรับ AR(1) regression — น้อยกว่า 30 obs จะไม่ reliable
- ห้าม reuse test data เป็น train ของ window เดิม — window ต้องเลื่อนเป็นลำดับเสมอ

ตัวอย่างจริง: Bybit ↔ Lighter BTC — Walk-Forward 360 วัน

ใช้ข้อมูล 360 วัน → walk-forward ได้ **9 windows** (train=90d, test=30d, slide=30d)

Window 1: train day 1-90, test day 91-120 → half-life = 6.1h Window 2: train day 31-120, test day 121-150 → half-life = 5.2h Window 3: train day 61-150, test day 151-180 → half-life = 8.7h Window 4: train day 91-180, test day 181-210 → half-life = 9.4h Window 5: train day 121-210, test day 211-240 → half-life = 7.1h Window 6: train day 151-240, test day 241-270 → half-life = 11.3h Window 7: train day 181-270, test day 271-300 → half-life = 6.8h Window 8: train day 211-300, test day 301-330 → half-life = 5.9h Window 9: train day 241-330, test day 331-360 → half-life = 6.5h เฉลี่ย: 7.4h | Range: 5.2-11.3h

Half-life เฉลี่ยจาก 9 windows = **7.4 ชั่วโมง** (range: 5.2-11.3h) — ความผันผวนของ half-life ระหว่าง windows แสดงว่า OU parameters เปลี่ยนตามช่วงเวลาจริง → **justifies การใช้ walk-forward แทน static calibration** ถ้าใช้ static จะได้ค่าเดียวตลอด ซึ่งผิดพลาดในหลายช่วง

โจทย์ 3.4 — Walk-Forward Window Count

มีข้อมูล **180 วัน**, ตั้งค่า train = 90 วัน, test = 30 วัน, slide = 30 วัน

ถาม: (a) ได้กี่ windows? (b) first trade เริ่มวันที่เท่าไร? (c) ถาลด train เหลือ 60 วัน จะได้กี่ windows?

► ดูคำตอบ

โจทย์ 3.1 — คำนวน half-life และ trade parameters

OLS AR(1) บน spread รายชั่วโมง ($\Delta t = 1/24$ วัน): $b = 0.94$, $a = 0.00012$
 $\text{std}(\text{residuals}) = 0.00015$

ถาม: (a) κ ต่อวัน (b) half-life เป็นชั่วโมง (c) entry band (d) spread ปัจจุบัน 0.05% จาก θ — ควร enter ไหม?

► ดูคำตอบ

โจทย์ 3.2 — แปลง b → strategy design

Spread รายชั่วโมง ให้ผล AR(1): $b = 0.97$
ค่า fee รวม (open+close) = 0.06% per round-trip

ถาม: (a) half-life (b) ถ้าตั้ง entry ที่ $2\sigma_{\text{stat}} = 0.10\%$ จะถือ position นานแค่ไหนโดยเฉลี่ย และ expected profit คืออะไร (ก่อน fee)?

► ดูคำตอบ

โจทย์ 3.3 — วิจัย OU breakdown

ใช้ OU trade Bybit ↔ Lighter มา 2 สัปดาห์ k เดิม = 2 ต่อวัน, $\theta = 0.1\%$

สัปดาห์ที่ 3: spread ออกไปถึง 0.4% และค้างอยู่นาน 3 วัน ไม่กลับ

ถาม: เกิดอะไรขึ้น และควรทำอะไร?

► ดูคำตอบ

Hedge Ratio β

OLS · TLS · Kalman Filter — สามวิธีหา "อัตราส่วนที่ถูกต้อง"

ตัวอย่าง: Bybit $\leftrightarrow$ Lighter BTC Perpetual

แก่นของบทนี้

β คือตัวเลขที่ทำให้ $\varepsilon = \log(A) - \beta \cdot \log(B)$ เป็น stationary

วิธีหา β มีผลต่อ entry ratio และ P&L โดยตรง — OLS, TLS, และ Kalman ให้ β ต่างกัน

$$\beta_{\text{OLS}} = \text{Cov}(r_A, r_B) / \text{Var}(r_B)$$

4.1 ทำไมวิธีหา β จึงสำคัญ

จากบทที่ 1 เราทราบแล้วว่า $\beta \neq 1$ — แต่คำถามคือ หา β ด้วยวิธีไหน? วิธีต่างกันให้ β ต่างกัน และ β ที่ต่างกัน หมายถึงอัตราส่วน trade ต่างกัน $\rightarrow$ P&L ต่างกัน

OLS

Ordinary Least Squares

เร็ว, simple, bias ถ้าทั้งสองตัวมี noise

TLS

Total Least Squares

Symmetric, เหมาะเมื่อทั้งคู่ noisy เท่ากัน

Kalman

Dynamic β

β เปลี่ยนตามเวลา, แม่นยำสุดแต่ซับซ้อนกว่า

4.2 OLS — วิธีที่ใช้บ่อยที่สุด

OLS หา β ที่ minimize ระยะทางแนวตั้ง (vertical residual) จากจุดข้อมูลถึงเส้น regression:

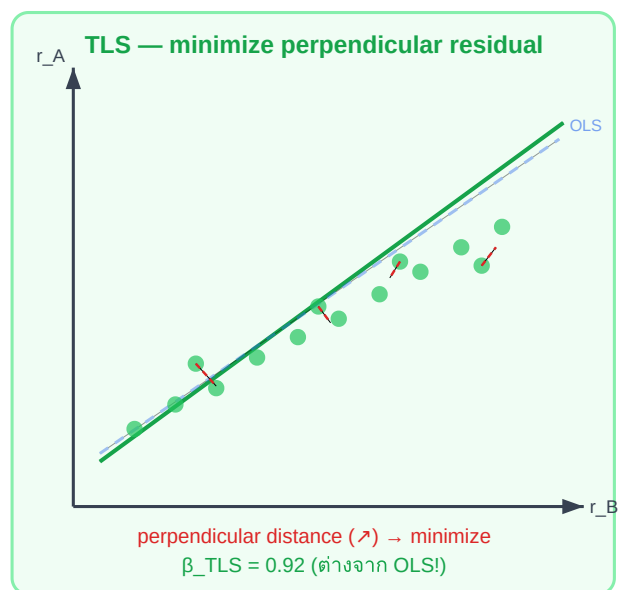
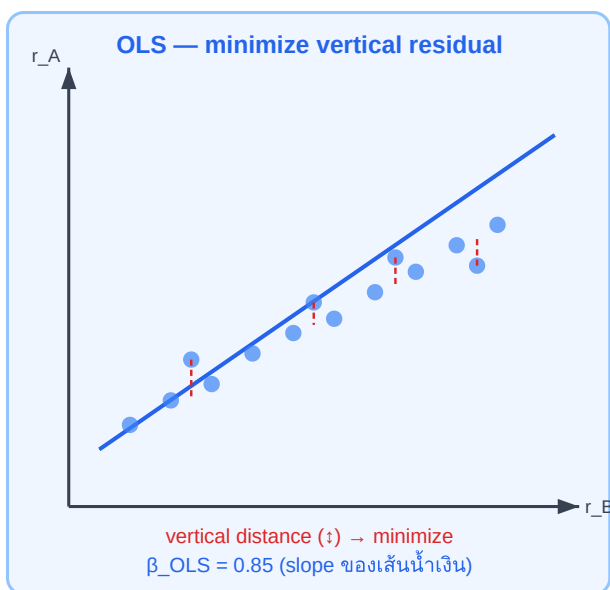
สูตร OLS B

$$\beta_{OLS} = \text{Cov}(r_A, r_B) / \text{Var}(r_B) = \Sigma(r_A - \bar{r}_A)(r_B - \bar{r}_B) / \Sigma(r_B - \bar{r}_B)^2$$

หรือถ้า regress $\log(P_A)$ บน $\log(P_B)$ โดยตรง (level regression):

$$\log(P_A) = \alpha + \beta \cdot \log(P_B) + \varepsilon \quad \beta_{OLS} = \text{slope ของ regression นี้}$$

OLS vs TLS — ต่างกันที่ "minimize อะไร"



OLS และ TLS ให้ β ต่างกัน — OLS ใช้ B เป็น "หลัก" (no noise), TLS สมมติทั้งคู่ noisy เท่ากัน

สูตร TLS (Total Least Squares)

$$\beta_{TLS} = [\text{Var}(r_A) - \text{Var}(r_B) + \sqrt{(\text{Var}(r_A) - \text{Var}(r_B))^2 + 4 \cdot \text{Cov}(r_A, r_B)^2}] / (2 \cdot \text{Cov}(r_A, r_B)) \quad \text{หรือใช้ SVD: } \beta_{TLS} = v_{12}/v_{22} \text{ (eigenvector ของ covariance matrix } [r_A, r_B])$$

เลือก OLS หรือ TLS?

สถานการณ์	แนะนำ	เหตุผล
B เป็น "benchmark" (spot price, reference)	OLS	B มี noise น้อยกว่า A — OLS assume B error-free
ทั้งสองตลาด liquidity ใกล้เคียงกัน	TLS	ทั้งคู่ noisy พอๆ กัน — symmetric
ต้องการ β เสถียร ปรับน้อย	OLS rolling	คำนวณง่าย เปลี่ยนได้ตามรอบ
β เปลี่ยนเร็ว ต้องการ adaptive	Kalman	อัปเดตทุก tick

4.3 Kalman Filter — Dynamic β ที่เปลี่ยนตามเวลา

OLS และ TLS สมมติว่า β คงที่ตลอด lookback window แต่ในตลาดจริง β อาจเปลี่ยนได้ Kalman Filter แก้ปัญหานี้โดยอัปเดต β ทุก time step:

KALMAN STATE SPACE MODEL

$\beta_t = \beta_{t-1} + w_t$ (state equation: β เดินแบบ random walk)

$r_{A,t} = \beta_t \cdot r_{B,t} + v_t$ (observation equation)

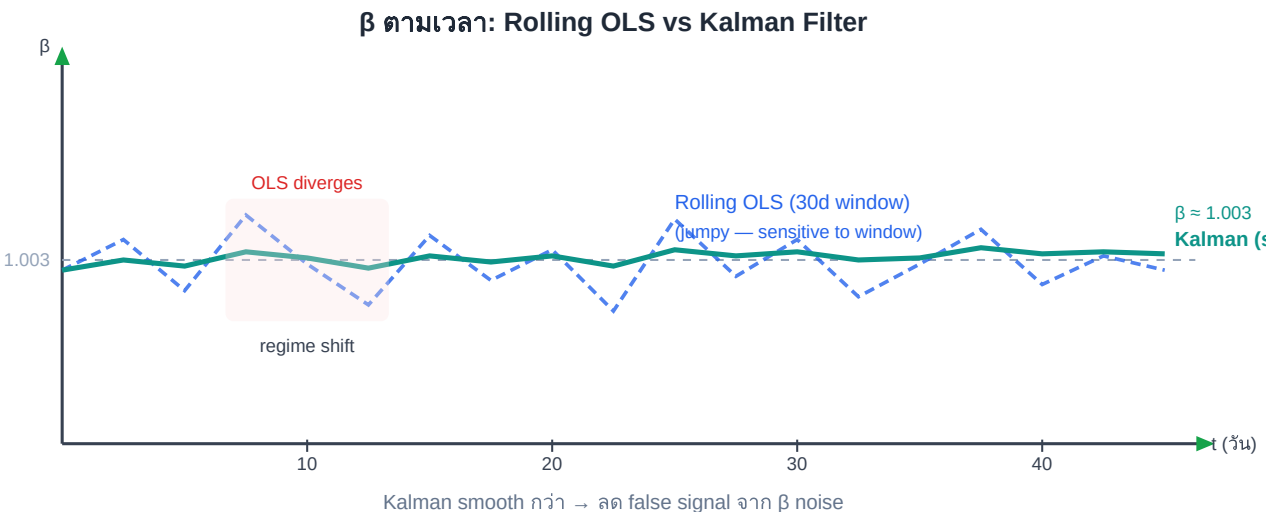
$w_t \sim N(0, Q)$ — process noise (ความเร็วที่ β เปลี่ยน)

$v_t \sim N(0, R)$ — observation noise (noise จากราคา)

Kalman Filter ทำงาน 2 ขั้นตอนซ้ำกันทุก time step:

ขั้น	สูตร	ความหมาย
Predict	$\beta_{t t-1} = \beta_{t-1}$, $P_{t t-1} = P_{t-1} + Q$	คาดว่า β ตอนนี้เป็นอะไร (ก่อนเห็นข้อมูล)
Update	$K = P \cdot r_B^2 / (r_B^2 \cdot P + R)$ $\beta_t = \beta_{t t-1} + K \cdot (r_{A,t} - \beta_{t t-1} \cdot r_{B,t})$ $P_t = (1 - K \cdot r_B^2) \cdot P$	ปรับ β ตามข้อมูลจริงที่เพิ่งเห็น

K คือ **Kalman Gain** — ถ้า Q สูง (β เปลี่ยนเร็ว) K ใหญ่ → เชื่อข้อมูลใหม่มาก ถ้า R สูง (noisy market) K เล็ก → เชื่อ model เก่ามากกว่า



Rolling OLS กระโดดตาม window boundary — Kalman ค่อยๆ ปรับ → position ขยับน้อยกว่า

4.4 Lookback Window และการ Roll

สำหรับ OLS rolling หรือ Kalman Q/R tuning ต้องเลือก lookback ให้เหมาะกับ strategy:

Lookback	ข้อดี	ข้อเสีย	เหมาะกับ
7 วัน	Responsive ต่อ regime ใหม่	Noisy, β กระโดด	Fast intraday
30 วัน	สมดุล stability/responsiveness	Lag ช้า	Intraday-swing (แนะนำ)
90 วัน	Stable, noise น้อย	ปรับ regime เปลี่ยนช้ามาก	Position trade

Re-estimate β เมื่อไร?

- Daily: re-run OLS บน 30 วันล่าสุด ก่อนตลาดเปิด
- เมื่อ ϵ drift ต่อเนื่อง > 2 วัน $\rightarrow \beta$ เปลี่ยน re-estimate ทันที
- Kalman: ไม่ต้อง re-estimate — อัปเดต continuous ทุก bar

4.5 เลือก β Method ตามวัตถุประสงค์ (Objective-Driven Design)

แก่นความคิด

OLS/TLS/Kalman ไม่ใช่แค่ตัวเลือกที่ดีกว่ากัน — แต่ละตัวถูกออกแบบมาเพื่อวัตถุประสงค์ต่างกัน เลือกให้ตรงกับสิ่งที่ต้องการ

สองเส้นทาง: Funding Harvest vs Spread Arbitrage

วัตถุประสงค์	Model	ผลที่ต้องการ	ϵ target
Funding Rate Harvest	Kalman Filter (dynamic)	β_t ติดตาม ratio จริงแบบ real-time → $\epsilon \approx 0$, variance ต่ำสุด	กราฟแบน $\approx$ เส้นศูนย์ → lock price risk
Spread Arbitrage	OLS/TLS (static)	β คงที่จาก historical regression → ϵ แกว่งรอบ 0 ตาม inefficiency	กราฟ mean-reverting ที่มี variance → กินจากส่วนต่าง

อธิบายเชิงแนวคิด

Funding Rate Harvest — ต้องการ $\epsilon_t \rightarrow 0$

เป้าหมายคือ market-neutral perfect hedge: Kalman β_t ปรับทุก bar ตาม cost-of-carry จริง ทำให้ $\epsilon_t \approx 0$ ตลอดเวลา PnL ทั้งหมดมาจาก **funding rate เท่านั้น** ไม่ใช่ price movement ยิ่ง ϵ เล็กยิ่งดี — แสดงว่า hedge สมบูรณ์

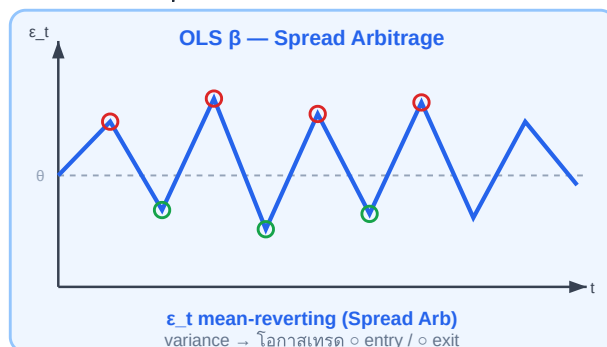
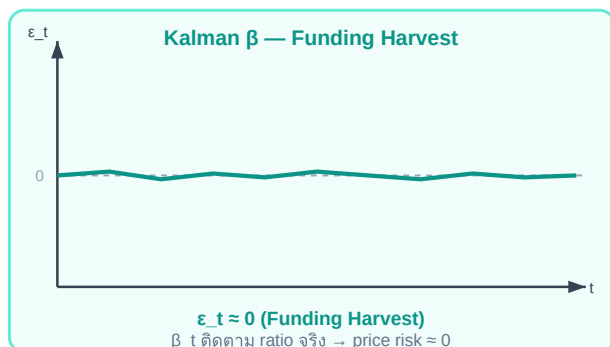
Spread Arbitrage — ต้องการ ϵ_t แกว่งรอบ 0

OLS lock β จากอดีต — ความไม่สมบูรณ์ของ hedge นี้เองคือแหล่งกำไร ϵ_t ที่มี variance สูงหมายถึง โอกาสเทรดมาก เมื่อ ϵ เบี่ยงจาก mean → entry; เมื่อ revert กลับ → exit

ความเข้าใจผิดที่พบบ่อย

Kalman ไม่ได้ "ดีกว่า" OLS เสมอ — ถ้าใช้ Kalman กับ Spread Arb จะได้ $\varepsilon \rightarrow 0$ ซึ่งทำให้ไม่มีสัญญาณ เพราะ β_t ปรับตัวจนกำจัด spread ออกไปหมด ต้องเลือก method ให้ตรงกับ **สิ่งที่ต้องการให้ ε ทำ**

ε_t ตามเวลา: เป้าหมายต่างกันตามกลยุทธ์



ซ้าย: Kalman ε_t แบน — สมบูรณ์สำหรับ Funding Harvest | ขวา: OLS ε_t แกว่ง — สมบูรณ์สำหรับ Spread Arb

Running Example: Bybit $\leftrightarrow$ Lighter — สองเส้นทาง

กลยุทธ์ที่ 1 — Funding Rate Harvest (Kalman): ใช้ Kalman $\beta_t \rightarrow \varepsilon_t$ แบนราบ ≈ 0
Bybit funding rate $0.01\%/8h = 10$ bps/day คือ PnL ทั้งหมด price risk $\approx$ hedged
ออกไปเกือบสมบูรณ์ กลยุทธ์ที่ 2 — Spread Arbitrage (OLS): ใช้ OLS $\beta = 1.002$ (ค่าคงที่
จาก 30d window) ε_t แกว่ง $\pm 0.3\%$, Half-life 7.4h คือโอกาสเทรด entry เมื่อ z -
score $> 2\sigma$, exit เมื่อ revert หนังสือเล่มนี้: $\rightarrow$ Spread Arb path (OLS/TLS) =
running example หลัก (ch5–ch14) $\rightarrow$ Kalman สำหรับ Funding Harvest = ch15

4.6 Pseudo-code

```
# OLS Beta
function beta_ols(r_A, r_B):
    return cov(r_A, r_B) / var(r_B)

# TLS Beta (via eigenvalue)
function beta_tls(r_A, r_B):
    C = cov_matrix([[r_A], [r_B]]) # 2x2
    eigenvalues, eigenvectors = eig(C)
    v = eigenvectors[:, argmin(eigenvalues)] # smallest eigenvalue
    return -v[0] / v[1]

# Kalman Filter Beta (one-step update)
function kalman_beta_update(beta, P, r_A, r_B, Q, R):
    # Predict
    P_pred = P + Q
    # Kalman gain
    K = P_pred * r_B / (r_B**2 * P_pred + R)
    # Update
    innovation = r_A - beta * r_B
    beta_new = beta + K * innovation
    P_new = (1 - K * r_B) * P_pred
    return beta_new, P_new
```

Running Example: β ของ Bybit $\leftrightarrow$ Lighter BTC Perp

ข้อมูล: 30 วัน, log returns รายวันที่ OLS (30d window): $\beta_{OLS} = \text{Cov}(r_{\text{Bybit}}, r_{\text{Lighter}}) / \text{Var}(r_{\text{Lighter}}) = 0.00000412 / 0.00000410 = 1.0049$ TLS: $\beta_{TLS} = 1.0063$ (เล็กน้อยกว่า OLS) Kalman ($Q=1e-7$, $R=1e-5$, warmup 14 วัน): $\beta_{Kalman_t} \approx 1.003 \pm 0.002$ (เปลี่ยนช้าๆ) ผลกระทบต่อ position: Long Bybit \$100,000: OLS: Short Lighter \$100,490 TLS: Short Lighter \$100,630 Kalman: Short Lighter \$100,300 ต่างกัน \$300-\$630 – ดูเล็กน้อย แต่ถ้า trade 100x/เดือน = \$30,000-\$63,000 ต่างกัน

โจทย์ 4.5 — เลือก Method ให้ตรงวัตถุประสงค์

บริษัท Quant เปิดสองกลยุทธ์:

- (ก) ใช้ Kalman β_t และ target $\varepsilon_t \approx 0$
- (ข) ใช้ OLS β และ z-score entry/exit

ถาม: กลยุทธ์ไหนทำกำไรจาก funding rate? กลยุทธ์ไหนทำกำไรจาก price inefficiency?

► ดูคำตอบ

โจทย์ 4.1 — คำนวณ β_{OLS} จากข้อมูล

Log returns รายวัน 5 วัน:

วัน	r_Bybit (%)	r_Lighter (%)
1	+1.20	+1.15
2	-0.80	-0.75
3	+2.10	+2.05
4	-1.50	-1.48
5	+0.60	+0.58

ถาม: คำนวณ β_{OLS} (Bybit เป็น A, Lighter เป็น B)

► ดูคำตอบ

โจทย์ 4.2 — เลือกวิธีหา β

คุณ trade spread ระหว่าง BTC perpetual บน Binance (volume 10× ของ Lighter) กับ Lighter ทั้งสองตลาดมี liquidity ต่างกันมาก — Binance เป็น "price discovery" หลัก

ถาม: ควรใช้ OLS หรือ TLS? และ Binance ควรเป็น A หรือ B ใน regression?

► ดูคำตอบ

โจทย์ 4.3 — Kalman Q/R tuning

คุณตั้ง Kalman กับ $Q=1e-9$ (เล็กมาก) และ $R=1e-4$ (ใหญ่มาก)
สังเกตว่า β_{Kalman} แทบไม่เปลี่ยนแปลง แม้ตลาดจะเปลี่ยน

ถาม: อธิบายว่าเกิดอะไรขึ้น และควรปรับ Q หรือ R อย่างไร?

► **คำตอบ**

Ch.4 Hedge Ratio β | ต่อไป → **Ch.5: Cointegration Tests**

Cointegration

Engle-Granger · Johansen · VECM

ทดสอบว่า spread ε มีแรงดึงกลับจริงหรือแค่ correlation ชั่วคราว

แก่นของบทนี้

Correlation สูงไม่พอสำหรับ stat arb — ต้องการ **cointegration**:

$\varepsilon = \log(A) - \beta \cdot \log(B)$ ต้องเป็น stationary (กลับหาค่ากลาง) ไม่ใช่แค่เดินไปด้วยกัน

H_0 : ε ไม่ stationary (ไม่ cointegrated) → ถ้า p-value < 0.05 → ปฏิเสธ H_0 → cointegrated ✓

นิยามทางการ — Cointegration (Engle & Granger, 1987)

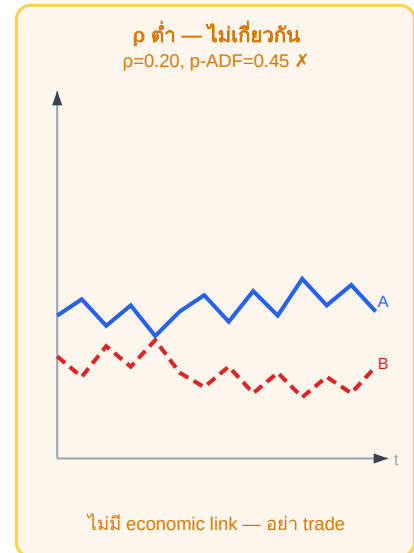
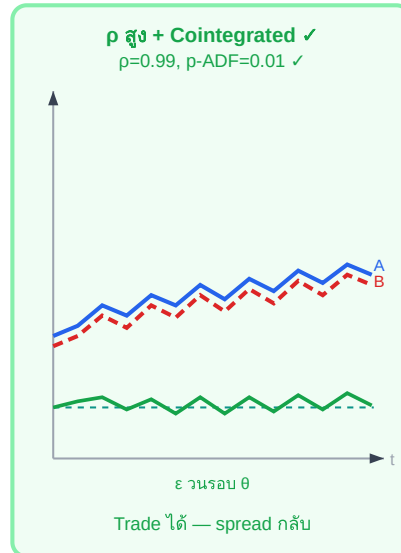
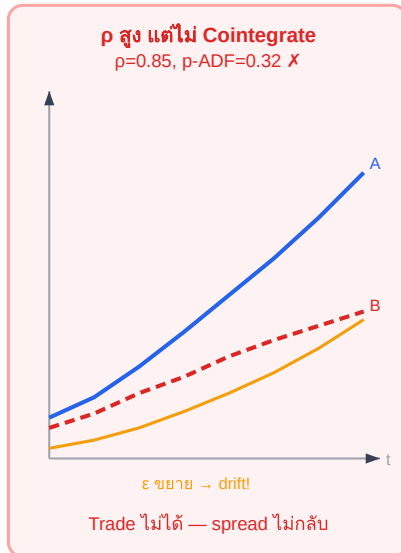
Series X_t และ Y_t แต่ละตัวเป็น **I(1)** (integrated of order 1 — มี unit root, ไม่ stationary) แต่คู่นี้ **cointegrated** ถ้ามีค่า β ที่ทำให้:

$$Z_t = X_t - \beta \cdot Y_t \text{ เป็น } I(0) \text{ (stationary, กลับหาค่ากลาง)}$$

กล่าวอีกนัย: แม้ X_t และ Y_t จะ random walk แยกกัน แต่ทั้งคู่ถูก "ดึง" ไว้ด้วย long-run equilibrium relationship เดียวกัน — spread $\varepsilon = Z_t$ จึงไม่ drift ออกไปตลอด

5.1 Correlation $\neq$ Cointegration — ความแตกต่างที่สำคัญ

3 สถานการณ์ — Correlation และ Cointegration



ต้องการทั้ง correlation สูง (A และ B เดินไปด้วยกัน) AND cointegration (ϵ stationary)

5.2 Engle-Granger 2-Step Test

วิธีที่ง่ายที่สุด ทำ 2 ขั้นตอน:

1

Regress $\log(P_A)$ บน $\log(P_B)$:

$$\log(P_A) = \alpha + \beta \cdot \log(P_B) + \epsilon_t$$

บันทึก residuals ϵ_t จาก regression นี้

2

ADF Test บน ϵ_t :

ถ้า ϵ_t stationary ($p < 0.05$) → A และ B cointegrated ✓

ถ้าไม่ผ่าน ($p \geq 0.05$) → ไม่ cointegrated → ห้าม trade

ADF Test อ่านผลอย่างไร

p-value	ความหมาย	ควรทำ
< 0.01	Reject H_0 อย่างมั่นใจมาก	Cointegrated — trade ได้
0.01–0.05	Reject H_0 ที่ 95% confidence	Cointegrated — trade ได้ (standard threshold)
0.05–0.10	Borderline	ระวัง — ลด position size หรือรอข้อมูลเพิ่มเติม
> 0.10	ไม่ reject H_0	ไม่ cointegrated — ห้าม trade spread

ข้อจำกัดของ Engle-Granger

- Test เพียง 1 cointegrating vector (ดีสำหรับ 2 assets)
- ลำดับ regression (A บน B vs B บน A) ให้ผลต่างกัน — ควรทดสอบทั้งสองทาง
- มี power ต่ำเมื่อ sample น้อย (< 100 observations)

⚠ Engle-Granger Asymmetry — จุดที่ผู้ใช้มักพลาด

สิ่งที่หลายคนเข้าใจผิด: ถ้า regress $A \sim B$ ได้ $\beta_{AB} = 1.003$ หลายคนคิดว่า regress $B \sim A$ จะได้ $\beta_{BA} = 1/1.003 \approx 0.997$ — แต่นั่นผิด

การ regress $\log(A) \sim \log(B)$ ให้ $\beta_{AB} = \text{Cov}(A,B)/\text{Var}(B)$

การ regress $\log(B) \sim \log(A)$ ให้ $\beta_{BA} = \text{Cov}(A,B)/\text{Var}(A)$

ความสัมพันธ์ที่ถูกต้อง: $\beta_{AB} \times \beta_{BA} = \rho^2$ (ค่า R^2) — ไม่ใช่ 1 อย่างที่คาด

ดังนั้น $1/\beta_{AB} \neq \beta_{BA}$ ทัวไป (เท่ากันเฉพาะกรณี $\rho=1$)

และ residual ε_{AB} กับ ε_{BA} **ต่างกันจริงๆ** ไม่ใช่แค่ scale

ผลที่ตามมา: ADF test บน ε_{AB} อาจ pass ($p=0.03$) แต่ ADF บน ε_{BA} อาจ fail ($p=0.12$) — หรือกลับกัน

วิธีปฏิบัติ: ทดสอบทั้งสองทิศทาง แล้วเลือก β ที่ให้ ADF p-value ต่ำกว่า (residual stationary กว่า)

ถ้าทั้งสองทิศทาง fail → pair ไม่ cointegrate จริง

```
# Python: ทดสอบทั้งสองทิศทาง ( $\beta$  แต่ละทิศไม่ได้สัมพันธ์แบบ  $1/x$ )
beta_AB, _ = np.polyfit(log_B, log_A, 1) #  $A = \beta_{AB} \cdot B + \alpha$ 
beta_BA, _ = np.polyfit(log_A, log_B, 1) #  $B = \beta_{BA} \cdot A + \alpha$ 
p_AB = adfuller(log_A - beta_AB * log_B)[1]
p_BA = adfuller(log_B - beta_BA * log_A)[1]
best_direction = "A~B" if p_AB < p_BA else "B~A"
```

5.3 Johansen Test — เมื่อมีมากกว่า 2 Assets

Johansen test ทดสอบ cointegration ของ n assets พร้อมกัน โดยหาจำนวน cointegrating vectors r :

$H_0: r = 0$ (ไม่มี cointegration เลย) $H_0: r \leq 1$ (มีได้อย่างมาก 1 vector) ... ทดสอบต่อไปจนถึง $r = n-1$ ถ้า Trace statistic หรือ Max eigenvalue > critical value $\rightarrow$ reject H_0

จำนวน Assets	ใช้ Test ไหน	เหตุผล
2 (pairs)	Engle-Granger	ง่ายกว่า ผลเท่ากัน
3+ (basket)	Johansen	หา cointegrating vectors ทุกตัว
ต้องการรู้ว่าใครปรับตัว	VECM	ระบุ adjustment coefficients

5.4 VECM — ใครเป็นฝ่ายปรับตัวกลับ?

เมื่อรู้ว่า cointegrated แล้ว VECM (Vector Error Correction Model) บอกว่า เมื่อ spread ห่างจาก equilibrium ใคร (A หรือ B) เป็นฝ่ายปรับตัวกลับ

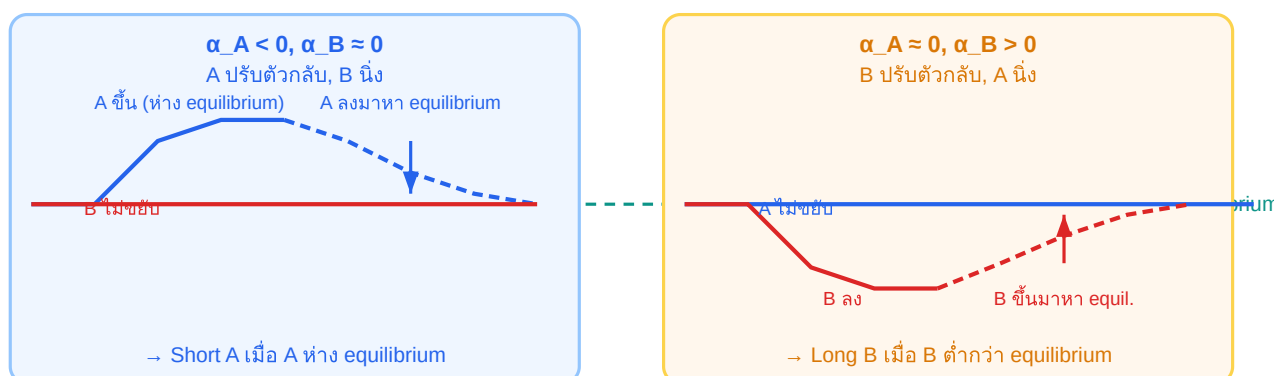
VECM สำหรับ 2 ASSETS

$$\Delta \log(P_A)_t = \alpha_A \cdot \varepsilon_{t-1} + \gamma_A \cdot \Delta \log(P_A)_{t-1} + u_{A,t}$$

$$\Delta \log(P_B)_t = \alpha_B \cdot \varepsilon_{t-1} + \gamma_B \cdot \Delta \log(P_B)_{t-1} + u_{B,t}$$

α_A, α_B = adjustment coefficients — บอกว่าใครปรับตาม error

VECM: ใครเป็นฝ่ายปรับตัวกลับ?



VECM ระบุว่าควร trade ฝั่งไหน — ถ้า A ปรับ: trade A | ถ้า B ปรับ: trade B | ถ้าทั้งคู่ปรับ: trade ทั้งคู่

VECM บอกอะไรสำหรับ Bybit ↔ Lighter?

ถ้า Bybit เป็น "price leader" ($\alpha_{\text{Bybit}} \approx 0$) และ Lighter ปรับตาม ($\alpha_{\text{Lighter}} > 0$):

→ เมื่อ Lighter ต่ำกว่า equilibrium: Lighter จะขึ้น → **Long Lighter**

→ เมื่อ Lighter สูงกว่า equilibrium: Lighter จะลง → **Short Lighter**

สรุป: trade Lighter เป็นหลัก ใช้ Bybit เป็น reference

5.5 Pseudo-code

```
# Engle-Granger Cointegration Test
function test_cointegration_eg(log_P_A, log_P_B):
    # Step 1: OLS regression
    beta, alpha = ols(log_P_B, log_P_A) # log_P_A = alpha + beta*log_P_B
    residuals = log_P_A - (alpha + beta * log_P_B)

    # Step 2: ADF test on residuals
    adf_stat, p_value = adf_test(residuals, lags=1)

    cointegrated = p_value < 0.05
    return {cointegrated, p_value, adf_stat, beta, residuals}

# Check VECM adjustment direction
function vecm_adjustment(log_P_A, log_P_B, residuals):
    d_A = diff(log_P_A) #  $\Delta \log(P_A)$ 
    d_B = diff(log_P_B) #  $\Delta \log(P_B)$ 
    err_lag = residuals[:-1] #  $\varepsilon_{t-1}$ 

    alpha_A = ols_slope(err_lag, d_A[1:]) # A's adjustment coefficient
    alpha_B = ols_slope(err_lag, d_B[1:]) # B's adjustment coefficient

    # Negative alpha → adjusts back to equilibrium
    A_adjusts = alpha_A < -0.05
    B_adjusts = alpha_B > 0.05
    return {alpha_A, alpha_B, A_adjusts, B_adjusts}
```

Running Example: Bybit ↔ Lighter ADF Test Result

ข้อมูล: 90 วัน, รายชั่วโมง (2,160 observations) Step 1 – OLS: $\log(P_{\text{Bybit}}) = 0.003 + 1.0049 \cdot \log(P_{\text{Lighter}}) + \varepsilon$ $\beta = 1.0049$, $R^2 = 0.9998 \rightarrow \beta > 1.0$ แปลว่า Bybit เคลื่อนไหว 0.49% มากกว่า Lighter ต่อการขยับ 1% ของ Lighter (อาจจาก liquidity premium หรือ fee structure ที่ต่างกัน) Step 2 – ADF test บน ε : ADF statistic = -4.82 p-value = 0.0003 ← ผ่าน! ($p \ll 0.05$) Critical values: 1%=-3.44, 5%=-2.87, 10%=-2.57 ผล: ADF stat (-4.82) < Critical (-3.44) →

Reject $H_0 \rightarrow$ Stationary $\rightarrow$ Cointegrated ✓ VECM: $\alpha_{\text{Bybit}} = -0.02$ (ปรับนิดหน่อย)
 $\alpha_{\text{Lighter}} = +0.18$ (ปรับมากกว่า) $\rightarrow$ Lighter เป็นฝ่ายปรับตัว $\rightarrow$ trade Lighter เป็นหลัก

Re-test เมื่อไร?

สัญญาณ	ควรทำ
p-value ขึ้นผ่าน 0.05 ใน monthly re-test	หยุด trade ทันที re-evaluate คู่นี้
β เปลี่ยน > 5% จากค่าเดิม	Re-run test ด้วย data ใหม่
Spread ค้างนอก $3\sigma > 2$ วัน	Emergency re-test — อาจ regime change

โจทย์ 5.1 — อ่านผล ADF

ทดสอบ ADF บน residuals ของ BTC-ETH spread ได้:

ADF = -2.31, p-value = 0.18, critical value 5% = -2.87

ถาม: (a) interpret ผล (b) ควร trade spread นี้ไหม?

► ดูคำตอบ

โจทย์ 5.2 — ใช้ VECM ตัดสิน

VECM บน Bybit $\leftrightarrow$ Lighter: $\alpha_{\text{Bybit}} = -0.21$, $\alpha_{\text{Lighter}} = +0.03$

ขณะนี้ $\varepsilon > 0$ (Bybit แพงกว่า equilibrium)

ถาม: ใครเป็นฝ่ายปรับ และควร trade อะไร?

► ดูคำตอบ

อ้างอิงสำคัญ (References)

- **Engle, R. F., & Granger, C. W. J. (1987).** Co-integration and error correction: Representation, estimation, and testing. *Econometrica*, 55(2), 251–276. — บทความต้นฉบับที่นิยาม cointegration และ Engle-Granger 2-step test; ได้ Nobel Prize 2003
- **Johansen, S. (1988).** Statistical analysis of cointegration vectors. *Journal of Economic Dynamics and Control*, 12(2–3), 231–254. — พัฒนา Johansen trace/eigenvalue test และ

VECM ที่ Ch.5 นำเสนอ

- **Johansen, S. (1991).** Estimation and hypothesis testing of cointegration vectors in Gaussian vector autoregressive models. *Econometrica*, 59(6), 1551–1580. — ขยาย critical values และ full likelihood approach
- **Vidyamurthy, G. (2004).** *Pairs Trading: Quantitative Methods and Analysis*. Wiley. — ตำราปฏิบัติที่นำ cointegration มาใช้กับ pairs trading อย่างเป็นระบบ

Ch.5 Cointegration | ต่อไป → **Ch.6: Hurst Exponent & Stationarity Tests**

Hurst Exponent & Stationarity Tests

$H < 0.5$ = Mean-Reverting · $H = 0.5$ = Random Walk · $H > 0.5$ = Trending

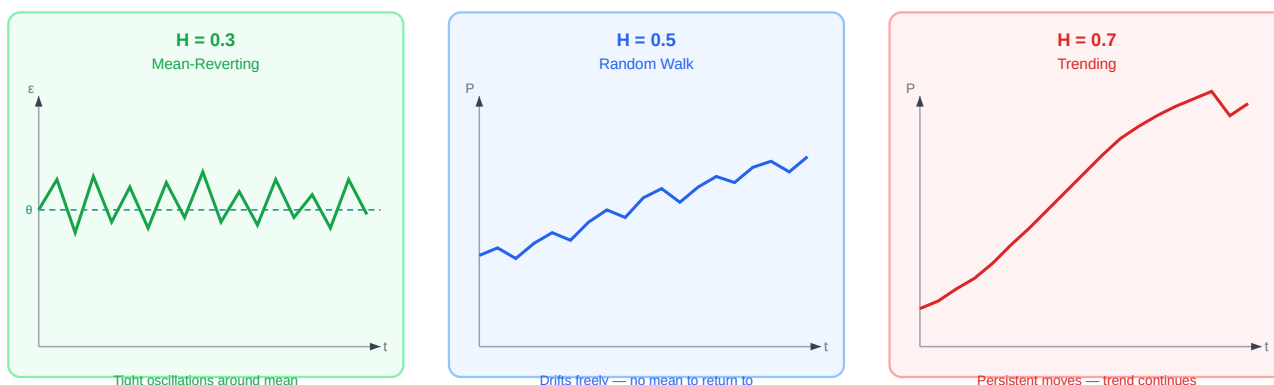
CORE IDEA — READ THIS FIRST

The Hurst exponent H measures how persistent a time series is. For statistical arbitrage, we **require $H < 0.5$** on the spread ε — this confirms mean-reversion exists. A series with $H < 0.5$ has anti-persistence: a move up makes a move down more likely. $H = 0.5$ is pure randomness; $H > 0.5$ means the trend will continue.

$H = \text{slope of } \log(R/S) \text{ vs } \log(n) \quad | \quad \text{Target: } H < 0.5 \text{ on spread } \varepsilon$

6.1 What H Measures — Three Regimes

The Hurst exponent was originally developed by Harold Hurst studying Nile River flood cycles. In finance, it quantifies whether a series has memory. Below, three synthetic time series illustrate the three regimes clearly.



Three time series with identical variance but different memory structures. Only $H < 0.5$ is tradeable for stat arb.

ทำไม $H < 0.5$ ถึงสร้าง Edge

เมื่อ $H < 0.5$ การเคลื่อนไหวในอดีตทำนาย *ทิศทางข้าม* ในอนาคต — ถ้า spread ขึ้น 1 standard deviation ใน bar นี้ bar ถัดไปน่าจะลง นี่คือแรงที่เราใช้ประโยชน์เมื่อ enter ที่ $\pm 2\sigma$ และรอ reversion ยิ่ง H ต่ำ แรง mean-reversion ยิ่งแข็ง edge ต่อ trade ยิ่งสูง

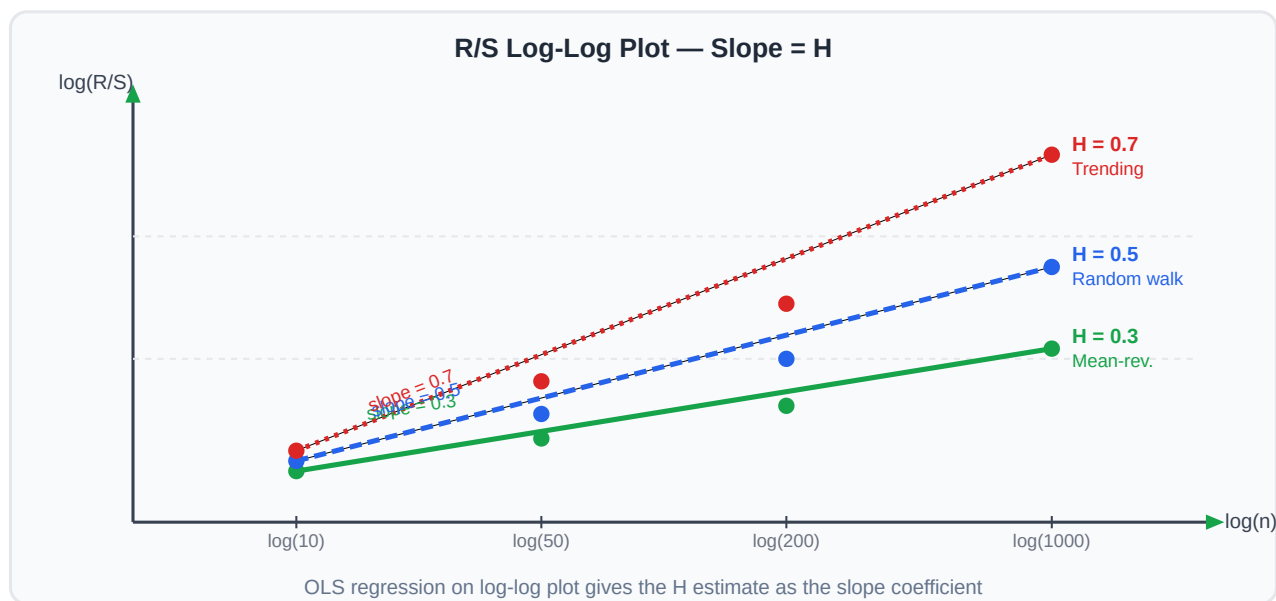
6.2 R/S Analysis — วิธีคำนวณ H

สัญชาตญาณ: R = range (สูงสุด – ต่ำสุดของ cumulative sum) วัดว่า series "แพร่กระจาย" ไปไกลแค่ไหน; S = std วัด volatility ตามปกติ อัตราส่วน R/S จึงบอกว่า series แพร่กระจายผิดปกติหรือเปล่า — random walk มี $R/S \propto \sqrt{n}$ แต่ mean-reverting series มี R/S เติบโตช้ากว่า slope $H < 0.5$ ในกราฟ log-log

วิธี **Rescaled Range (R/S)** คือ algorithm มาตรฐานในการ estimate H โดยวัดจากหลาย window size n :

1. Demean the series: $y_i = x_i - \text{mean}(x_{1..n})$
2. Compute cumulative sum: $Y_k = \sum y_i$ for $i=1..k$
3. $R(n) = \max(Y) - \min(Y)$ (range of cumulative sum)
4. $S(n) = \text{std}(x_{1..n})$ (standard deviation)
5. Rescaled range = $R(n) / S(n)$

Repeat for many window sizes n , then fit: $\log(R/S) = H \cdot \log(n) + C$. The slope H is the Hurst exponent.



Log-log plot of R/S vs window size n . H is the OLS slope. Target: $H < 0.5$ (flatter line).

6.3 ADF Test + KPSS Test — Double Confirmation

The Augmented Dickey-Fuller (ADF) and KPSS tests have **opposite null hypotheses**. Using both gives a robust verdict on stationarity.

Test	H ₀ (Null)	Reject H ₀ means	We want
ADF	Series is non-stationary (has unit root)	Series IS stationary	p-value < 0.05 (reject)
KPSS	Series IS stationary	Series is non-stationary	p-value > 0.05 (fail to reject)

Combined Interpretation — Four Cases

ADF result	KPSS result	Verdict	Trade?
Rejects ($p < 0.05$)	Fails to reject ($p > 0.05$)	Strongly stationary	Yes
Rejects ($p < 0.05$)	Rejects ($p < 0.05$)	Borderline / short memory	Caution
Fails to reject	Fails to reject ($p > 0.05$)	Long memory possible	No
Fails to reject	Rejects ($p < 0.05$)	Non-stationary	Never

6.4 Interpretation Table — H Range to Trade Decision

H Range	Behavior	Trade Decision	Approx Half-Life
$H < 0.35$	Strongly mean-reverting	Trade aggressively, tighter bands	Very short (hours)
$0.35 \leq H < 0.45$	Moderately mean-reverting	Trade normally, standard bands	Short to medium
$0.45 \leq H < 0.55$	Near random walk — uncertain	Reduce size, require ADF confirm	Long or unreliable
$0.55 \leq H < 0.65$	Weakly trending	Do not trade mean-reversion	N/A (trend instead)
$H \geq 0.65$	Strongly trending	Abort pair, find new one	N/A

Practical Note: Recalculate H Monthly

H is not stable over time. A pair that had $H = 0.33$ last month may shift to $H = 0.51$ after a structural break (exchange policy change, fee restructuring, or major liquidity event). Always recalculate H on a rolling 90-day window and alert if H crosses 0.45.

6.5 Pseudo-Code: hurst_rs()

```
function hurst_rs(series, min_window=10): # series: array of log-prices or spread values
n_total = len(series)
window_sizes = [10, 20, 40, 80, 160, 320] # geometric spacing
log_n_list = []
log_rs_list = []
for n in window_sizes:
    if n >= n_total: continue
    rs_values = [] # Slide window (non-overlapping). ถ้าต้องการ H แม่นยำขึ้น # ใช้ overlapping windows (step=1) แต่ช้ากว่า  $O(n^2)$ 
    for start in range(0, n_total - n, n):
        chunk = series[start : start + n]
        demeaned = chunk - mean(chunk)
        cumsum = cumulative_sum(demeaned)
        R = max(cumsum) - min(cumsum)
        S = std(chunk)
        if S > 0:
            rs_values.append(R / S)
    if len(rs_values) > 0:
        log_n_list.append(log(n)) #  FIX: mean(log(RS)) ไม่ใช่ log(mean(RS))
        # log(mean(x)) ≠ mean(log(x)) โดย Jensen's inequality # การใช้ log(mean()) จะ bias H สูงขึ้น (overestimate mean-reversion)
        log_rs_list.append(mean([log(x) for x in rs_values])) # OLS regression: log(R/S) = H * log(n) + const
        H, const = ols_slope_intercept(log_n_list, log_rs_list)
    return H

function check_stationarity(series):
    H = hurst_rs(series)
    adf_p = adf_test(series).p_value
    kpss_p = kpss_test(series).p_value
    tradeable = (H < 0.45) and (adf_p < 0.05) and (kpss_p > 0.05)
    return {"H": H, "adf_p": adf_p, "kpss_p": kpss_p, "tradeable": tradeable}
```

6.6 Confidence Interval สำหรับ H — Bootstrap Test

H = 0.31 กับ H = 0.35 อาจแยกไม่ออกทางสถิติ เพราะ R/S estimator มี noise โดยเฉพาะเมื่อ sample size น้อย การรายงาน H โดยไม่มี SE หรือ CI อาจทำให้ตัดสินใจ trade ผิด

Bootstrap CI สำหรับ H — แนวคิด

1. **Resample:** สุ่มดึง bootstrap sample ขนาดเท่า original (with replacement) ทำ B = 500 รอบ
2. **Compute H:** คำนวณ H จาก R/S analysis ทุก bootstrap sample → ได้ distribution {H₁, H₂, ..., H_B}
3. **SE(H):** SE = std({H₁, ..., H_B})
4. **95% CI:** [H - 1.96·SE, H + 1.96·SE]

Rule: CI ต้อง **ทั้งหมดอยู่ต่ำกว่า 0.5** ถึงจะ trade ได้อย่างมั่นใจ ถ้า CI ทะลุ 0.5 ให้ลดขนาด position หรือ รอ sample เพิ่ม

```
function hurst_bootstrap_ci(series, B=500, confidence=0.95):
    n = len(series)
    h_samples = []
    for _ in range(B):
        # Block bootstrap: สุ่ม block ขนาด sqrt(n) เพื่อรักษา autocorrelation
        block_size = int(sqrt(n))
        resampled = []
        while len(resampled) < n:
            start = randint(0, n - block_size)
            resampled.extend(series[start : start + block_size])
        resampled = resampled[:n]
        h_b = hurst_rs(resampled)
        h_samples.append(h_b)
    H_mean = mean(h_samples)
    H_se = std(h_samples)
    alpha = 1 - confidence
    z_crit = 1.96 # 95% normal approximation
    ci_low = H_mean - z_crit * H_se
    ci_high = H_mean + z_crit * H_se
    tradeable_ci = ci_high < 0.50 # entire CI must be below 0.5
    return {"H": H_mean, "SE": H_se, "CI_95": (ci_low, ci_high), "tradeable_CI": tradeable_ci}
```

เหตุผลที่ใช้ Block Bootstrap แทน i.i.d. Bootstrap

Spread residuals มี autocorrelation (นั่นคือ mean-reverting behavior ที่เราวัด H) — ถ้า resample แบบ i.i.d. (สุ่มแต่ละจุดแยกกัน) จะทำลาย autocorrelation structure ทำให้ SE ของ H ต่ำเกินจริง (overconfident CI)

Block bootstrap ขนาด $\sqrt{n}$ รักษา local autocorrelation ไว้ → SE ที่ได้จริงกว่า

H_point	SE (bootstrap)	95% CI	CI ต่ำกว่า 0.5?	ตัดสินใจ
0.31	0.03	[0.25, 0.37]	ใช่ ✓	Trade ได้ — confident
0.43	0.04	[0.35, 0.51]	ไม่ — CI ทะลุ 0.5	ลด size 50%, re-test next month

0.46	0.05	[0.36, 0.56]	ไม่ — CI ทะลุ 0.5	ไม่ trade จนกว่า H จะลง
0.38	0.02	[0.34, 0.42]	ใช่ ✓	Trade ได้ — บาง SE → น่าเชื่อถือ

ตัวอย่างจริง: Bybit BTC Perp ↔ Lighter BTC Perp

We compute H on two series — the spread ε and raw BTC price — over the last 90 days of hourly data:

Series	H (R/S)	Interpretation	ADF p-value	KPSS p-value	Verdict
Spread ε (Bybit – 1.003·Lighter)	0.31	Strongly mean-reverting	0.0003	0.18	Trade ✓
Raw BTC log-price (Bybit)	0.52	Near random walk	0.41	0.02	Do not trade

The spread has $H = 0.31$ — deeply in mean-reverting territory. ADF rejects non-stationarity ($p = 0.0003$) AND KPSS fails to reject stationarity ($p = 0.18$). This is the strongest possible confirmation. The raw BTC price, by contrast, shows $H = 0.52$ — a near-perfect random walk, exactly as theory predicts.

Estimated half-life from $H = 0.31$: → Hurst-implied: ~4-8 hours (tight, fast reversion) → Confirmed by OU fit: $\kappa = 2.8/\text{day}$ → half-life ≈ 5.9 hours

โจทย์ 6.1 — แปลความหมาย $H = 0.43$

A new pair (Binance BTC Perp ↔ OKX BTC Perp) produces $H = 0.43$ from the R/S analysis. ADF $p = 0.032$, KPSS $p = 0.08$.

Q: Is this spread mean-reverting? What trading approach do you take?

► ดูคำตอบ

โจทย์ 6.2 — ผลทดสอบขัดแย้งกัน

For a candidate spread: ADF $p = 0.08$ (fails to reject), KPSS $p = 0.03$ (rejects stationarity).

Q: Interpret this combined result. Should you trade this spread?

▶ **ดูคำตอบ**

Ch.6 Hurst Exponent & Stationarity Tests | Next → **Ch.7: GARCH on Residuals**

GARCH on Residuals

Conditional Volatility σ_t — When Spread Volatility Clusters

$$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot \sigma_{t-1}^2$$

CORE IDEA — READ THIS FIRST

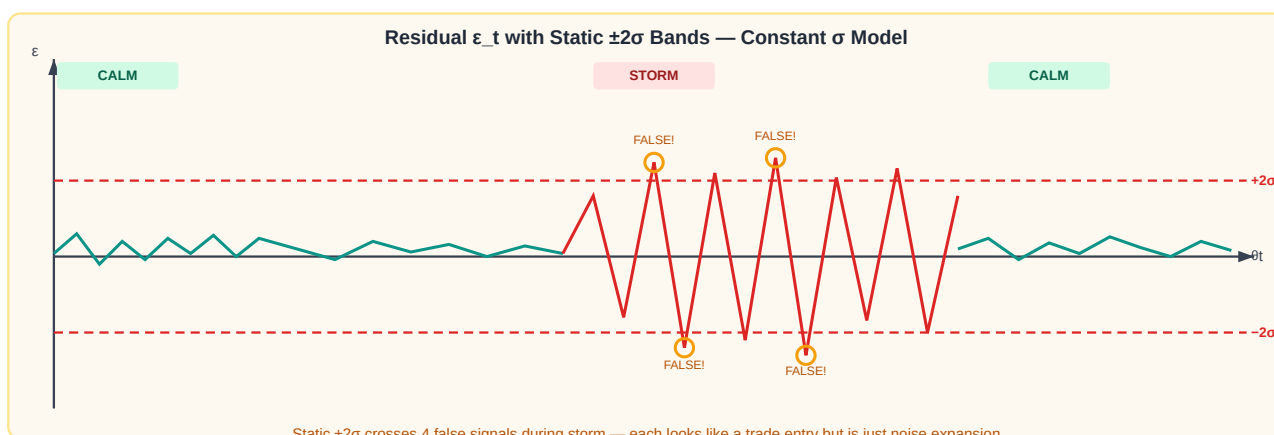
Ornstein-Uhlenbeck assumes **constant** σ . In practice, spread volatility clusters: calm periods are followed by calm, volatile periods by volatile. Using a static z-score during a storm generates false signals. GARCH(1,1) models time-varying conditional variance σ_t , giving a **GARCH z-score** that stays calibrated regardless of market regime.

$$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot \sigma_{t-1}^2 \quad | \quad z_t = (\varepsilon_t - \theta) / \sigma_t$$

7.1 The Problem with Constant σ

Standard OU assumes the residual noise is i.i.d. with fixed σ . In real BTC markets, volatility is highly heteroskedastic — it bunches in time. Static $\pm 2\sigma$ bands drawn at the unconditional standard deviation create two failure modes:

- **Storm false signals:** During high-volatility periods, the spread crosses $\pm 2\sigma$ (static) repeatedly, each crossing appearing to be a trading opportunity. In reality, σ has expanded 3 $\times$, and $2\sigma_{\text{static}}$ is only 0.67 GARCH- σ . These are not true extremes.
- **Calm missed signals:** During quiet periods, a genuine 2σ move is actually a 4σ GARCH event — an extremely rare and high-conviction signal being ignored.



Static bands (red dashes) don't expand during volatile regimes — causing repeated false crossings in the storm period.

7.2 GARCH(1,1) Model

GARCH(1,1) — Generalized AutoRegressive Conditional Heteroskedasticity — updates the variance estimate each period using two inputs: the last shock squared and the last variance estimate.

GARCH(1,1) EQUATION

$$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta \cdot \sigma_{t-1}^2$$

ω (omega) — long-run variance floor. Always positive. Prevents σ_t from collapsing to zero.

α (alpha) — shock impact coefficient. Typical range: 0.05–0.15. High α → variance reacts quickly to new shocks.

β (beta) — persistence coefficient. Typical range: 0.80–0.90. High β → variance decays slowly (long memory).

Stationarity condition: $\alpha + \beta < 1$ (otherwise variance explodes to infinity).

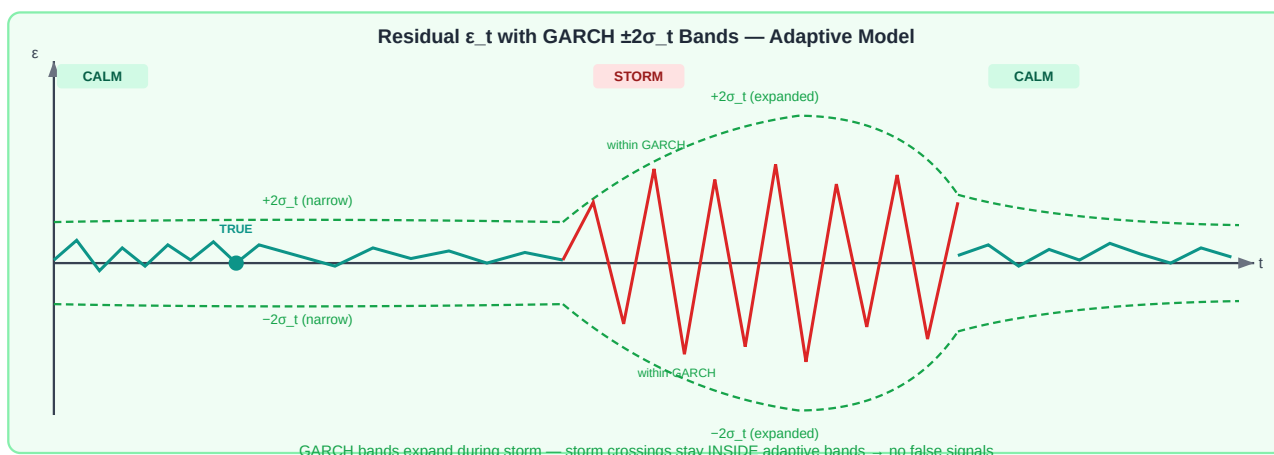
Long-run variance: $\sigma^2_\infty = \omega / (1 - \alpha - \beta)$

Parameter	Typical Range	Effect of Increasing	Effect of Being Too High
ω (omega)	1e-7 to 1e-4	Raises the variance floor	Overestimates calm-period σ
α (alpha)	0.05 – 0.15	Faster reaction to shocks	Too noisy, σ _t unstable
β (beta)	0.80 – 0.92	Longer variance memory	If α+β≥1: non-stationary

7.3 GARCH Z-Score — Adaptive Bands

Instead of $z = (\varepsilon_t - \theta) / \sigma_{\text{static}}$, we use the time-varying denominator:

GARCH z-score: $z_t = (\varepsilon_t - \theta) / \sigma_t$ Entry signal: $z_t > +2.0 \rightarrow$ short spread (expect reversion down) $z_t < -2.0 \rightarrow$ long spread (expect reversion up) Exit signal: $|z_t| < 0.5 \rightarrow$ close position



GARCH adaptive bands (green dashes) expand during the storm — the same residual path produces NO false signals with GARCH z-score.

7.4 When to Use GARCH vs Constant σ

Aspect	Constant σ (OU base)	GARCH(1,1)
Assumption	σ is fixed over time	σ_t varies based on recent shocks
Best for	Very stable pairs, low volatility	Pairs with clustering volatility
Detects regime	No	Yes — σ_t encodes current regime
False signals	High during vol spikes	Low — bands adapt
Computation	Trivial (one number)	O(T) per update, very fast
When to switch	Default	$\text{std}(\text{residuals}, 7\text{d}) / \text{std}(\text{residuals}, 30\text{d}) > 1.5$

Volatility Clustering Trigger

Compute the ratio of 7-day rolling std to 30-day rolling std of the spread residuals. If this ratio exceeds 1.5, the volatility regime has shifted — switch to GARCH z-score immediately. If the ratio drops below 1.1 consistently, constant σ is adequate and GARCH adds minimal benefit.

7.5 Pseudo-Code: GARCH Update & Z-Score

```
function garch_update(omega, alpha, beta, prev_sigma_sq, prev_residual): # GARCH(1,1) variance
recursion — call once per bar # omega: long-run variance floor (e.g. 1e-6) # alpha: shock weight (e.g.
0.10) # beta: persistence (e.g. 0.85) # prev_sigma_sq:  $\sigma^2_{t-1}$  # prev_residual:  $\epsilon_{t-1}$  (actual spread
residual, not z-score) new_sigma_sq = omega + alpha * (prev_residual ** 2) + beta * prev_sigma_sq
return new_sigma_sq
function garch_zscore(eps, theta, sigma_t): # eps: current spread residual  $\epsilon_t$  #
theta: OU long-run mean  $\theta$  # sigma_t: sqrt(current GARCH variance) z = (eps - theta) / sigma_t return z
function should_use_garch(residuals, window_short=7*24, window_long=30*24): # Returns True if
volatility clustering detected std_short = rolling_std(residuals, window_short)[-1] std_long =
rolling_std(residuals, window_long)[-1] ratio = std_short / std_long return ratio > 1.5 # Main loop (hourly
bar) sigma_sq = long_run_variance # initialize to  $\omega/(1-\alpha-\beta)$  for each new bar: prev_eps =
spread_residual[t-1] sigma_sq = garch_update(omega, alpha, beta, sigma_sq, prev_eps) sigma_t =
sqrt(sigma_sq) z = garch_zscore(spread_residual[t], theta, sigma_t) if z > 2.0: execute_short_spread() if
z < -2.0: execute_long_spread() if abs(z) < 0.5: close_position()
```

7.6 การ Estimate ω , α , β จากข้อมูลจริง (MLE)

ค่า ω , α , β **ต้องหาจากข้อมูล** ไม่ใช่ guess — วิธีมาตรฐานคือ Maximum Likelihood Estimation (MLE) ผ่าน library ที่ optimize log-likelihood ของ GARCH โดยอัตโนมัติ

```
# Python – ใช้ arch library (pip install arch) from arch import arch_model
import numpy as np # residuals: spread residuals  $\varepsilon_t$  (ใช้ % return หรือ log-diff)
# ⚠️ UNIT NOTE: scaling  $\times 100$  เปลี่ยนหน่วย  $\varepsilon$  จาก raw  $\rightarrow$  % ดังนั้น: #  $\omega$  ที่ได้จาก fit จะมี
หน่วย %2 (ไม่ใช่ raw2) # ก่อนนำ  $\omega$  ไปใช้ใน garch_update() ต้องหาร 10000 ถ้า code ทำงาน
กับ raw units # หรือ scale residuals  $\times 100$  ตลอดทั้ง pipeline ให้สม่ำเสมอ residuals =
spread_residuals * 100 # scale เป็น % เพื่อ numerical stability # Fit GARCH(1,1)
model = arch_model(residuals, vol='GARCH', p=1, q=1, mean='Constant') result =
model.fit(disp='off') # disp='off' ปิด iteration output # อ่านพารามิเตอร์ที่ได้ omega
= result.params['omega'] #  $\omega$  (variance floor) alpha = result.params['alpha[1]']
#  $\alpha$  (shock weight) beta = result.params['beta[1]'] #  $\beta$  (persistence) print(f" $\omega$ =
{omega:.2e},  $\alpha$ ={alpha:.4f},  $\beta$ ={beta:.4f}") print(f" $\alpha+\beta$ ={alpha+beta:.4f} (ต้อง <
1 เพื่อ stationarity)") # Conditional variances ที่ model fit cond_var =
result.conditional_volatility ** 2 # array ของ  $\sigma_t^2$  # Forecast  $\sigma_{t+1}$  สำหรับ
next bar (ใช้ใน live trading) forecast = result.forecast(horizon=1,
reindex=False) sigma_next_sq = forecast.variance.values[-1, 0] sigma_next =
np.sqrt(sigma_next_sq)
```

หน่วย ω : $\times 100$ Scaling ทำให้ ω อยู่ใน %² ไม่ใช่ Raw²

เมื่อ residuals = spread_residuals * 100 (scale จาก fraction เป็น %):

- ε_t มีหน่วย % (เช่น 0.08, 0.15, 0.24)
- ω ที่ fit ได้มีหน่วย %² (เช่น $\omega \approx 0.000006$ %²)
- σ_t จาก GARCH มีหน่วย % $\rightarrow$ GARCH z-score = $\varepsilon_t(\%) / \sigma_t(\%)$ ✓ ไม่มีปัญหา

ปัญหาถ้า mix หน่วย: ถ้า ε_t ใน code เป็น raw (0.0008) แต่ ω ที่ fit ไว้เป็น %² (0.000006%) $\rightarrow \sigma_t = \sqrt{(\omega\%) \times 0.01} \neq \sigma_t$ ที่ถูก

วิธีแก้ที่ง่ายที่สุด: เลือก scale เดียวตลอด pipeline — แนะนำให้ scale $\times 100$ ทั้งหมด หรือใช้ raw ทั้งหมด โดยหาร $\omega\%$ ด้วย 10000 ก่อนใช้

```
# ถ้าอยากทำงานใน raw units หลังจาก fit ด้วย % units: omega_raw = omega_pct /
10000 # %2  $\rightarrow$  raw2 ( $\div 100^2$ ) # จากนั้น garch_update(omega_raw, alpha, beta,
...) ด้วย raw epsilon ได้เลย
```

ขนาด Sample ที่จำเป็น

GARCH ต้องการข้อมูลพอ: ≥ 250 observations สำหรับ GARCH(1,1) — น้อยกว่านี้ parameter estimate ไม่เสถียร

ในกรณี crypto perp 1-hour bars: ≥ 10 วัน, 5-minute bars: ≥ 1 วัน

Refit ทุก week หรือเมื่อ residual std เปลี่ยน $> 20\%$ จาก fit ก่อนหน้า

ผลที่คาดหวังสำหรับ BTC Spread

จากประสบการณ์กับ crypto spread โดยทั่วไปพบ:

$\omega \approx 1e-7$ ถึง $1e-5$ | $\alpha \approx 0.05-0.15$ | $\beta \approx 0.80-0.92$

ถ้า $\alpha + \beta \geq 1 \rightarrow$ model non-stationary $\rightarrow$ ลอง EGARCH หรือ ตัดข้อมูล outlier ออกก่อน refit

ทางเลือก: EGARCH (ถ้า volatility asymmetric)

Crypto มักมี "volatility asymmetry" — ราคาลงทำให้ spread volatile มากกว่าราคาขึ้น

ใช้ EGARCH ได้: `arch_model(residuals, vol='EGARCH', p=1, q=1)`

parameter เพิ่ม γ (asymmetry) — ถ้า $\gamma < 0$ แปลว่า negative shock ทำให้ volatile มากกว่า

Running Example: Bybit BTC Perp ↔ Lighter — Funding Rate War (3 Days)

During a 3-day BTC funding rate divergence event, the Bybit ↔ Lighter spread becomes much more volatile. We compare static vs GARCH z-scores:

Period	Spread σ (actual)	Static σ (baseline)	Static z-score	GARCH σ_t	GARCH z-score
Normal market	0.08%	0.08%	2.0 (signal)	0.08%	2.0 (signal)
Funding war day 1	0.24%	0.08%	6.3 (FALSE!)	0.21%	2.1 (correct)
Funding war day 2	0.22%	0.08%	5.8 (FALSE!)	0.23%	1.9 (no entry)
Recovery	0.10%	0.08%	2.5 (borderline)	0.11%	2.3 (signal)

Parameters used (residuals scaled $\times 100$ to %): $\omega = 0.000192$, $\alpha = 0.12$, $\beta = 0.85$ ($\alpha + \beta = 0.97 < 1$ ✓). Long-run $\sigma = \text{sqrt}(0.000192 / 0.03) \approx 0.080 \rightarrow$ matches normal-market $\sigma \approx 0.08\%$ in the table.

Naive $z = 0.50\% / 0.08\% = 6.3 \rightarrow$ looks like a huge 6σ trade opportunity
GARCH $z = 0.50\% / 0.24\% = 2.1 \rightarrow$ mild signal – σ_t has already expanded
Correct action: REDUCE size or skip entry (storm period)

Exercise 7.1 — Compute New σ^2

Given: $\omega = 0.0001$, $\alpha = 0.10$, $\beta = 0.85$, $\text{prev}_\sigma^2 = 0.0004$, $\text{prev}_\varepsilon = 0.025$

Q: Compute new σ^2 . Is $\alpha + \beta < 1$? What is the long-run σ^2 ?

► Show Answer

Exercise 7.2 — Naive vs GARCH Signal Conflict

The spread residual shows $\varepsilon = 0.48\%$, $\theta = 0.00\%$. Static $\sigma = 0.14\%$. Current GARCH $\sigma_t = 0.26\%$.

Naive $z = 0.48/0.14 = 3.5$. GARCH $z = 0.48/0.26 = 1.8$.

Q: Should you enter a short-spread position? Explain your reasoning.

► Show Answer

อ้างอิงสำคัญ (References)

- **Engle, R. F. (1982).** Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation. *Econometrica*, 50(4), 987–1007. — บทความต้นฉบับที่นิยาม ARCH model; Nobel Prize 2003
- **Bollerslev, T. (1986).** Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327. — ขยาย ARCH เป็น GARCH(p,q) ที่ใช้ใน Ch.7; $\sigma_t^2 = \omega + \sum \alpha_j \cdot \varepsilon_{t-j}^2 + \sum \beta_j \cdot \sigma_{t-j}^2$
- **Nelson, D. B. (1991).** Conditional heteroskedasticity in asset returns: A new approach. *Econometrica*, 59(2), 347–370. — EGARCH ที่กล่าวถึงใน Ch.7; จัดการ asymmetric volatility response

Jump-Diffusion OU: จัดการ Fat Tail ใน Spread

ตรวจจับ jump events และปรับกลยุทธ์เมื่อ spread กระโดดผิดปกติ

ตัวอย่างหลัก: Bybit BTC Perp ↔ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

spread เดินตาม OU แต่มี **jump discontinuity** แทรก — เหตุการณ์ผิดปกติที่ทำให้ spread กระโดดทันทีโดยไม่ผ่าน diffusion ปกติ ต้องตรวจจับและจัดการแยกต่างหาก

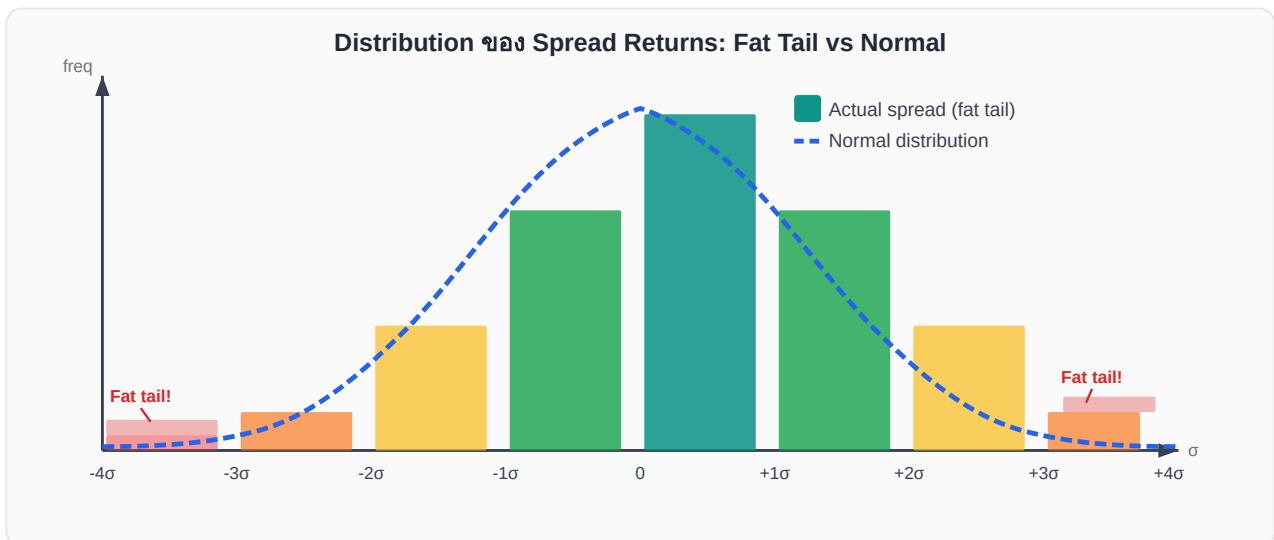
$$d\varepsilon = \kappa(\theta - \varepsilon)dt + \sigma dW + J \cdot dN$$

8.1 ทำไม OU ปกติไม่พอ

โมเดล OU มาตรฐาน $de = \kappa(\theta - e)dt + \sigma dW$ สมมติว่า residual กระจายตัวแบบ **Normal** — แต่ในความเป็นจริง spread ของสินทรัพย์ crypto มี **fat tail**: เหตุการณ์ที่ห่างจากค่าเฉลี่ย $4-6\sigma$ เกิดบ่อยกว่าที่ Normal distribution คาดไว้มาก

อาการของ fat tail ที่พบในทางปฏิบัติ

- Spread กระโดด $3-5\sigma$ ใน tick เดียว (ไม่ใช่ค่อยๆ เพิ่ม)
- Stop-loss ถูก trigger บ่อยกว่าที่ backtest คาดไว้
- Sharpe ratio ใน live trading ต่ำกว่า backtest อย่างมีนัยสำคัญ
- Kurtosis ของ residuals > 3 (leptokurtic)



Histogram ของ spread returns (แท่ง teal) vs Normal curve (เส้นประน้ำเงิน) — หางซ้าย-ขวานานกว่า Normal มาก

เมื่อ fat tail มีอยู่จริง โมเดล OU ล้วนๆ จะ **underestimate** ความน่าจะเป็นของเหตุการณ์รุนแรง ทำให้ sizing และ stop-loss ผิดพลาด

8.2 Jump-Diffusion SDE

เราขยาย OU model โดยเพิ่ม jump component เข้าไป:

JUMP-DIFFUSION OU — สมการหลัก

$$d\epsilon = \kappa(\theta - \epsilon)dt + \sigma dW + J \cdot dN$$

κ = mean-reversion speed (เหมือนเดิม)

θ = long-run mean (เหมือนเดิม)

σdW = diffusion term (Brownian motion ปกติ)

dN = Poisson process, $P(dN=1 \text{ ใน } dt) = \lambda \cdot dt$ — jump เกิดด้วย intensity λ

J = jump size (random variable, distribution ขึ้นกับโมเดล)

สัญชาตญาณ: ส่วนใหญ่ spread เดินแบบ OU ปกติ แต่ทุกครั้งที่ Poisson event เกิด (เฉลี่ยทุก $1/\lambda$ หน่วยเวลา) spread กระโดดทันที ด้วย random size J

Component	ความหมาย	ตัวอย่างค่า
κ (kappa)	ความเร็วในการกลับหา θ	0.5–5.0 ต่อวัน
σ (sigma)	Volatility ของ diffusion ส่วน	0.001–0.01
λ (lambda)	Jump intensity (jumps/วัน)	0.1–2.0
J (jump size)	ขนาดการกระโดด (สุ่ม)	$\pm 2-10\sigma$

8.3 Merton vs Kou Jump Model

มีสองโมเดลหลักสำหรับกำหนด distribution ของ jump size J :

Merton Jump Model (1976)

$J \sim N(\mu_J, \sigma_J^2)$ — กระจายแบบ Normal

Jump size สุ่มมาจาก Normal distribution — symmetric รอบ μ_J

ถ้า $\mu_J = 0$: jump ขึ้นและลงมีโอกาสเท่ากัน

ถ้า $\mu_J < 0$: jump มักลงมากกว่าขึ้น (เหมาะกับ crash scenario)

$J \sim N(\mu_J, \sigma_J^2)$ ตัวอย่าง: $\mu_J = 0, \sigma_J = 0.005 \rightarrow$ Jump ส่วนใหญ่อยู่ในช่วง $\pm 1\%$ ($\pm 2\sigma_J$)

Kou Double-Exponential Model (2002)

$J \sim$ Asymmetric double exponential — up jump และ down jump ต่างกัน

ด้วยความน่าจะเป็น p : jump ขึ้น ขนาด $\sim \text{Exp}(\eta_1)$ (exponential)

ด้วยความน่าจะเป็น $1-p$: jump ลง ขนาด $\sim \text{Exp}(\eta_2)$

ทำให้ tail ขึ้นและลง asymmetric ได้ตามจริง

$J = \{ +\text{Exp}(\eta_1) \text{ ด้วยความน่าจะเป็น } p, -\text{Exp}(\eta_2) \text{ ด้วยความน่าจะเป็น } 1-p \}$ ตัวอย่าง: $p=0.4, \eta_1=200, \eta_2=100 \rightarrow$ Down jump มักใหญ่กว่า up jump (tail ช้ายหนักกว่า)

คุณสมบัติ	Merton	Kou
Distribution ของ J	Normal $N(\mu_J, \sigma_J^2)$	Double exponential (asymmetric)
Symmetry	Symmetric (ถ้า $\mu_J=0$)	Asymmetric ($p \neq 0.5$)
Parameters	λ, μ_J, σ_J (3 params)	$\lambda, p, \eta_1, \eta_2$ (4 params)
Tail behavior	Lighter tail	Heavier, asymmetric tail
Fit ดีเมื่อ	Jump ขึ้น-ลงสมมาตร	Crash มักใหญ่กว่า rally

เมื่อไรควรใช้ Kou แทน Merton?

- Spread มักกระโดด **ลง** แรงกว่ากระโดดขึ้น (เช่น leverage liquidation)
- ช่วง funding rate war ที่ขา bear รุนแรงกว่าขา bull
- เมื่อ backtest Merton แล้ว over-estimated $P(\text{positive jump})$

8.4 Lee-Mykland Jump Detection

วิธีตรวจจับ jump ที่ใช้กันมากในทางปฏิบัติคือ **Lee-Mykland (2008)** ซึ่งสร้าง test statistic จาก bipower variation:

LEE-MYKLAND TEST STATISTIC

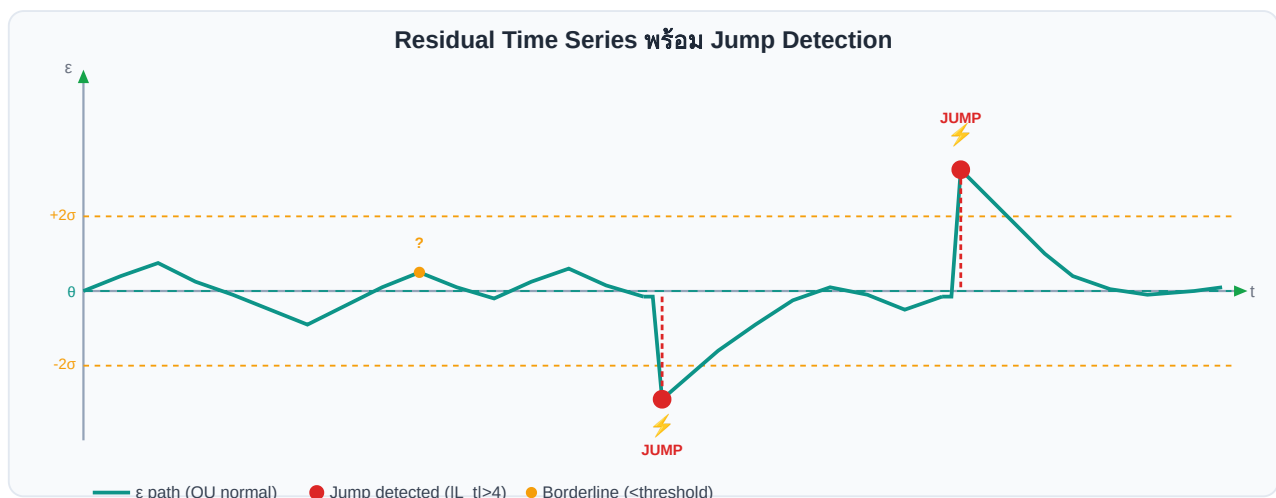
$$L_t = (\varepsilon_t - \varepsilon_{t-1}) / \sigma_{BV}$$

σ_{BV} = bipower variation estimator (robust ต่อ jump ของตัวเอง)

ถ้า $|L_t| > c_{\text{threshold}}$ → detect jump ที่เวลา t

threshold $c = 4.0$ เป็นค่าที่ใช้บ่อย (corresponds to p-value ต่ำมาก)

ข้อดีของ bipower variation: ถ้า σ_{BV} คำนวณจาก squared returns ปกติ jump ขนาดใหญ่จะ inflate σ ทำให้ L_t เล็กลงและพลาด jump แต่ bipower variation ใช้ product ของ adjacent absolute returns ซึ่ง robust กว่า



Timeline ของ ε : ส่วนใหญ่ mean-revert ปกติ แต่มี jump spikes ที่ถูก flag ด้วย Lee-Mykland test

8.5 Pseudo-code: detect_jump()

ต่อไปนี้เป็น algorithm สำหรับตรวจจ้ง jump ด้วย bipower variation:

```
def detect_jump(residuals, window=30, threshold=4.0): """ ตรวจจ้ง jump ใน
residual series ด้วย Lee-Mykland test Parameters ----- residuals : array-
like --  $\epsilon_t$  series window : int -- rolling window สำหรับ bipower variation
threshold : float --  $|L_t| > \text{threshold} \rightarrow \text{jump}$  Returns ----- jumps : bool
array -- True ที่ index ที่เป็น jump L : float array -- test statistic  $L_t$  ทุก time
step """ bipower = compute_bipower_variation(residuals, window) # bipower
variation: sum of  $|r_t| * |r_{t-1}|$  (robust ต่อ jump) sigma_bv = sqrt(bipower /
window) L = diff(residuals) / sigma_bv #  $L_t = (\epsilon_t - \epsilon_{t-1}) / \text{sigma}_{BV}$  jumps
= abs(L) > threshold return jumps, L def compute_bipower_variation(residuals,
window): """BV_t =  $(\pi/2) * \sum_{i=t-w}^{t-1} |r_i| * |r_{i-1}|$ """ r =
diff(residuals) bv = rolling_sum(abs(r[:-1]) * abs(r[1:]), window) return (pi /
2) * bv
```

หมายเหตุการ implement จริง

- ใช้ window=30 ถ้า data เป็น minute bars $\rightarrow$ window = 30 นาที
- threshold=4.0 เทียบเท่ากับ p-value < 0.001 สำหรับ Normal distribution
- หลังจาก flag jump ให้ **exclude** observation นั้นออกจากการ calibrate σ เพื่อไม่ให้ inflate
- ในทางปฏิบัติ ใช้ library realized (R) หรือเขียน custom ใน Python

8.6 วิธีจัดการ Jump ในกลยุทธ์

เมื่อตรวจพบ jump มี 3 approaches หลักที่ใช้กัน:

Approach 1 — หยุด Signal ชั่วคราว (Signal Pause)

เมื่อ `detect_jump() = True` → ไม่เปิด position ใหม่ชั่วคราว
รอให้ `L_t` กลับมามีค่าต่ำกว่า threshold ก่อน (เช่น รอ 5–15 tick)

ข้อดี: ปลอดภัยสุด ไม่เข้าไปใน noise

ข้อเสีย: พลาด opportunity ถ้า jump เป็น "overreaction" ที่จะ revert

Approach 2 — ขยาย Stop-Loss (Wider Stop)

ปรับ stop-loss จาก $\pm 2\sigma$ เป็น $\pm 4\sigma$ ชั่วคราวเมื่อ jump detected
ยังเปิด position ได้ แต่ tolerance กว้างขึ้น

ข้อดี: ไม่ถูก stop out โดยไม่จำเป็น

ข้อเสีย: ถ้า jump ไม่ revert จะขาดทุนมากขึ้น

Approach 3 — ลด Position Size (Reduce Size)

ลด size ลง 50% เมื่ออยู่ใน "jump window" (เช่น 10 tick หลัง jump)
trade ต่อได้ แต่ risk น้อยลง

ข้อดี: balance ระหว่าง opportunity และ risk

ข้อเสีย: ซับซ้อนกว่า ต้องมี position sizing logic เพิ่ม

Approach	เหมาะกับ	ความซับซ้อน	Risk
Signal Pause	เริ่มต้น, conservative	ต่ำ	ต่ำสุด
Wider Stop	มั่นใจว่า jump จะ revert	ปานกลาง	ปานกลาง
Reduce Size	อยากเก็บ opportunity	สูง	ปานกลาง

8.7 Running Example: Bybit ↔ Lighter และ Funding Rate War

Running Example: Bybit BTCUSDT Perp ↔ Lighter BTCUSDT Perp — Jump Event

ช่วง **funding rate war**: Bybit ประกาศปรับ funding rate เป็น 0.09% ต่อ 8 ชั่วโมง (ปกติ 0.01%) เพื่อดึง short position

ผลกระทบต่อ spread:

- Bybit perp premium เพิ่มขึ้นทันทีจาก \$10 → \$80 ใน 2 นาที (8x)
- spread ε กระโดด $\sim 5.5\sigma$ → `detect_jump() = True`
- $L_t = 5.8 > \text{threshold } 4.0$ → confirmed jump

การ handle ที่เหมาะสม:

```
# เมื่อ jump detected if detect_jump(recent_residuals)[0][-1]: # Approach 1:
หยุด signal 10 tick pause_signal(duration=10) # Log สาเหตุ
log_event("funding_rate_jump", size=L_t, exchange="Bybit") # ถ้ามี open
position → ขยาย stop-loss if has_open_position():
adjust_stop(multiplier=2.0) #  $2\sigma \rightarrow 4\sigma$ 
```

หลัง 15 นาที spread กลับสู่ปกติบางส่วน (\$10 → \$35) แต่ไม่กลับไปที่ \$10 เพราะ funding rate ยังสูง — การ pause signal ช่วยหลีกเลี่ยงการเข้า position ในช่วงที่ spread ยัง elevated

โจทย์ท้ายบท

โจทย์ 8.1 — Jump vs Normal Volatility

Residual ε เปลี่ยนจาก 0.002 เป็น 0.012 ใน 1 tick ($dt = 1$ วินาที)

จากการ calibrate: $\sigma_{BV} = 0.002$ ต่อ tick

ถาม: คำนวณ L_t แล้วตัดสินใจว่าเป็น jump หรือ normal volatility (threshold = 4.0) พร้อมอธิบายว่าทำไม

► คำตอบ

โจทย์ 8.2 — Merton vs Kou: เลือกใช้เมื่อไร

จากข้อมูล historical spread ของ Bybit $\leftrightarrow$ Lighter พบว่า:

- Jump ลง (negative) เฉลี่ย -6.2σ
- Jump ขึ้น (positive) เฉลี่ย $+2.8\sigma$
- สัดส่วน: 60% down jump, 40% up jump

ถาม: ควรใช้ Merton หรือ Kou? เพราะอะไร?

► คำตอบ

โจทย์ 8.3 — Bybit Funding Rate 3x: ควรทำอย่างไร

Bybit ประกาศเพิ่ม funding rate จาก 0.01% เป็น 0.03% ต่อ 8 ชั่วโมง (3x)

ณ ขณะนี้คุณไม่มี open position และ `detect_jump()` ยังไม่ triggered

ถาม: ควร flag เหตุการณ์นี้เป็น (anticipated) jump แล้วทำอย่างไรในกลยุทธ์?

► คำตอบ

- **Merton, R. C. (1976).** Option pricing when underlying stock returns are discontinuous. *Journal of Financial Economics*, 3(1–2), 125–144. — Merton jump-diffusion model พร้อม Gaussian jump size ที่ Ch.8 นำเสนอ
- **Kou, S. G. (2002).** A jump-diffusion model for option pricing. *Management Science*, 48(8), 1086–1101. — Kou double-exponential jump model; asymmetric jump sizes (upside vs downside) ที่เหมาะกับ crypto spreads
- **Lee, S. S., & Mykland, P. A. (2008).** Jumps in financial markets: A new nonparametric test and jump dynamics. *Review of Financial Studies*, 21(6), 2535–2563. — Lee-Mykland jump detection ด้วย bipower variation ที่ใช้ใน pseudo-code Ch.8
- **Barndorff-Nielsen, O. E., & Shephard, N. (2004).** Power and bipower variation with stochastic volatility and jumps. *Journal of Financial Econometrics*, 2(1), 1–37. — รากฐานทฤษฎีของ bipower variation สำหรับ jump detection

Regime-Switching OU: ตลาดมีหลาย 'สภาวะ'

Markov chain 3 สภาวะ — Normal / Stressed / Broken — ปรับ OU parameters ตาม regime

ตัวอย่างหลัก: Bybit BTC Perp ↔ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

ตลาดไม่ได้อยู่ใน "สภาพเดียว" ตลอดเวลา — ช่วง Normal spread กลับเร็ว ช่วง Stressed volatility พุ่ง ช่วง Broken spread ไม่กลับเลย ต้องรู้ว่าอยู่ใน regime ไหน แล้ว **เทรดเฉพาะ Normal regime**

κ , θ , σ เปลี่ยนตาม regime ที่ตลาดอยู่

9.1 ปัญหาของ Single-OU

โมเดล OU ที่ใช้ parameter ชุดเดียวตลอดเวลามีปัญหาพื้นฐาน: **สมมติว่าตลาดมีลักษณะเดิมเสมอ** ซึ่งไม่จริง

อาการที่บอกว่า Single-OU ไม่เพียงพอ

- ช่วง **volatile**: σ พุ่งขึ้น 3–5x จากค่าปกติ — signal เดิมมี noise มากเกินไป เข้า position แล้วขาดทุน
- ช่วง **broken**: spread ไม่กลับมาหา θ เลยแม้ผ่านไปหลายชั่วโมง — κ ลดลงจนใกล้ศูนย์
- ช่วง **calm** เกินไป: spread แทบไม่ขยับ — opportunity หหมด แต่ model ยังส่ง signal

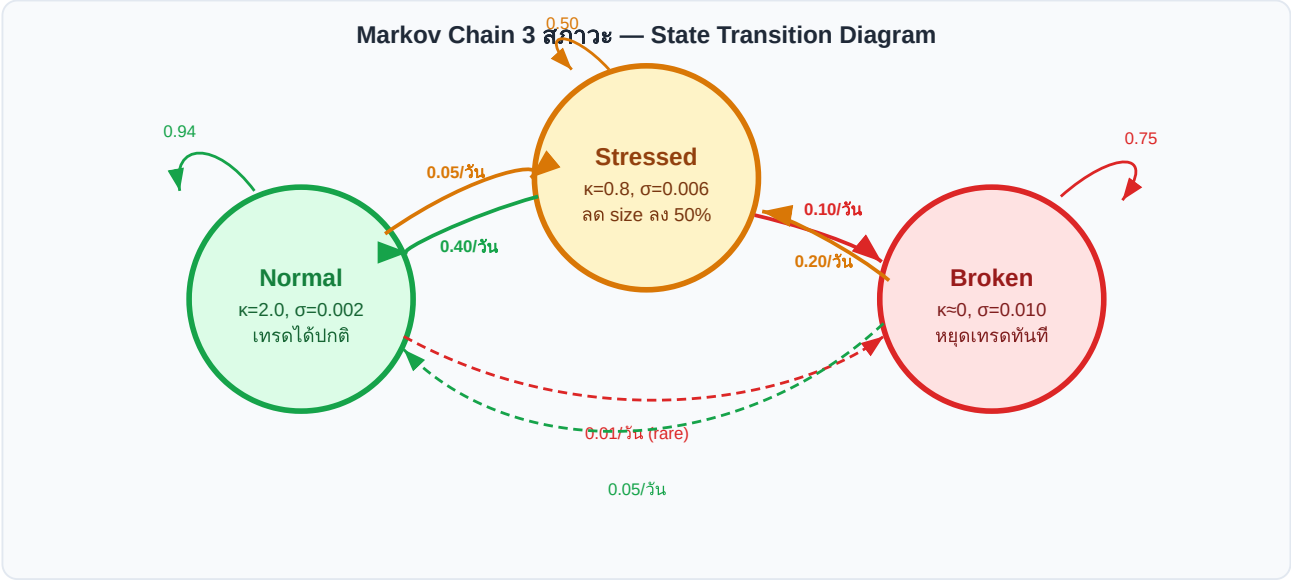
ผลที่ตามมาเมื่อใช้ Single-OU ในทุก regime

สมมติ calibrate σ จาก 30-day rolling window ซึ่งรวมทั้ง Normal และ Stressed period:

- $\sigma_{\text{estimated}}$ จะสูงกว่าความเป็นจริงใน Normal regime → entry threshold สูงเกินไป → พลาด opportunity
- $\sigma_{\text{estimated}}$ จะต่ำกว่าความเป็นจริงใน Stressed regime → stop-loss แคบเกินไป → ถูก stop out บ่อย
- ผลลัพธ์: กลยุทธ์ทำงานได้ไม่ดีทั้งสอง regime

9.2 3-State Markov Chain

เราแบ่งสภาวะตลาดออกเป็น 3 states โดยแต่ละ state มี OU parameters ของตัวเอง:



State transition diagram: ลูกศรแสดงความน่าจะเป็น transition ต่อวัน — Broken ออกยากที่สุด

Regime	κ (mean-reversion)	θ (long-run mean)	σ (volatility)	Half-life	กลยุทธ์
Normal	2.0 (เร็ว)	≈ 0 (spread กลับ ศูนย์)	0.002 (ต่ำ)	~ 8 ชั่วโมง	เทรดเต็ม size
Stressed	0.8 (ช้า)	≈ 0 (ยังกลับ แต่ช้า)	0.006 (สูง)	~ 21 ชั่วโมง	ลด size 50%
Broken	≈ 0 (ไม่ revert)	undefined (drift)	0.010 (สูง มาก)	∞ (ไม่กลับ)	หยุดเทรด ทันที

$\theta \approx 0$ ใน Normal/Stressed หมายความว่า spread ยังกลับสู่ศูนย์ (หรือ baseline); ใน Broken regime $\kappa \rightarrow 0$ ทำให้ θ ไม่มีความหมาย — spread drift ออกไปโดยไม่มีแรงดึงกลับ

9.3 Hamilton Filter

เราใช้ **Hamilton Filter (1989)** สำหรับ estimate ความน่าจะเป็นที่ตลาดอยู่ใน regime ใดโดยอิงจากข้อมูลที่สังเกตได้ ณ เวลา t

สัญชาตญาณ Hamilton Filter — 2 ขั้นตอน bar

1. **Predict:** "ถ้าเมื่อวานอยู่ใน regime j ด้วยความน่าจะเป็น p_j วันนี้จะอยู่ใน regime ใด?" — ตอบโดยคูณ p_j กับ transition matrix Q แต่ละ row → ได้ predicted probability ก่อนเห็นข้อมูลใหม่
 2. **Update:** "spread วันนี้มีค่า y_t — regime ไหนที่ explain ค่านี้ได้ดีที่สุด?" — ตอบโดยคูณ predicted probability $\times$ likelihood ที่ y_t จะเกิดขึ้นในแต่ละ regime แล้ว normalize ให้บวกเป็น 1
- ทำซ้ำทุก bar → ได้ $P(\text{regime} \mid \text{ข้อมูลถึงวันนี้})$ แบบ real-time โดยไม่ต้องรู้ regime จริงล่วงหน้า

HAMILTON FILTER — สมการหลัก

$$P(S_t=j \mid Y_t) \propto f(y_t \mid S_t=j) \cdot \sum_i P(S_t=j \mid S_{t-1}=i) \cdot P(S_{t-1}=i \mid Y_{t-1})$$

S_t = hidden state (Normal=0, Stressed=1, Broken=2) ณ เวลา t

Y_t = ข้อมูลที่สังเกตได้ถึงเวลา t (residuals ทั้งหมด)

$f(y_t \mid S_t=j)$ = likelihood ของ observation ถ้าอยู่ใน regime j

$P(S_t=j \mid S_{t-1}=i)$ = transition probability (จาก matrix Q)

$\propto$ = proportional to (normalize ให้ผลรวมเป็น 1)

Likelihood Function สำหรับแต่ละ Regime

ถ้า regime j มี OU parameters ($\kappa_j, \theta_j, \sigma_j$) likelihood ของ observation y_t คือ:

$$f(y_t \mid S_t=j, y_{t-1}) = \text{Normal}(y_t; \mu_j, \text{var}_j) \quad \mu_j = y_{t-1} * \exp(-\kappa_j * dt) + \theta_j * (1 - \exp(-\kappa_j * dt)) \quad \text{var}_j = (\sigma_j^2 / (2 * \kappa_j)) * (1 - \exp(-2 * \kappa_j * dt))$$

นี่คือ exact OU transition density: ถ้า y_{t-1} รู้แล้ว y_t ต่อไป Normal distribution ที่ mean และ variance คำนวณได้จาก OU parameters

Transition Matrix Q — ตัวอย่าง

Normal Stressed Broken Normal [0.94 0.05 0.01] Stressed[0.40 0.50 0.10
] Broken [0.05 0.20 0.75] แต่ละ row บวกกันได้ 1 (must sum to 1) $Q[i,j] =$
 $P(S_t=j \mid S_{t-1}=i)$

9.4 Pseudo-code: regime_filter()

```
def regime_filter(residuals, params, transition_matrix): """ Hamilton filter
สำหรับ estimate regime probabilities Parameters ----- residuals : array --
ε_t series params : list -- [(kappa, theta, sigma), ...] per regime
transition_matrix : 2D array -- Q matrix, shape (n_regimes, n_regimes) Returns
----- probs : 2D array -- shape (T, n_regimes) probs[t, j] = P(S_t=j | Y_t)
""" n_regimes = len(params) T = len(residuals) # เริ่มด้วย uniform prior probs =
initialize_probs(n_regimes) # [1/3, 1/3, 1/3] all_probs = zeros((T, n_regimes))
all_probs[0] = probs for t in range(1, T): # Step 1: log-likelihood ของ y_t ใน
แต่ละ regime # ✅ FIX: ทำงานใน log-space เพื่อป้องกัน numerical underflow # เมื่อ T ยาว
หรือ likelihood เล็กมาก การคูณซ้ำๆ ทำให้ค่าเป็น 0.0 (underflow) log_likelihooods = [
log_ou_likelihoood(residuals[t], residuals[t-1], p) for p in params ] # Step 2:
Prediction step ใน log-space #  $P(S_t=j) = \sum_i Q[i,j] * P(S_{t-1}=i | Y_{t-1})$ 
predicted = transition_matrix.T @ probs # ยังใช้ prob-space ได้ตรงนี้ # Step 3:
Update step ใน log-space แล้ว normalize ด้วย log-sum-exp # ✅ FIX: ใช้ -inf แทน
log(0 + ε) เพื่อไม่ให้ bias log_updated = [] for j in range(n_regimes): if
predicted[j] > 0: log_updated.append(log_likelihooods[j] + log(predicted[j]))
else: log_updated.append(-inf) # regime นี้เป็นไปได้ # log-sum-exp trick:  $\log(\sum e^{x_i}) = \max_x + \log(\sum e^{x_i - \max_x})$  # เหตุผล:  $x_i - \max_x \leq 0$  เสมอ ดังนั้น
 $e^{x_i - \max_x} \in (0, 1] \rightarrow$  ไม่ overflow max_log = max(log_updated) if max_log == -
inf: # ทุก regime impossible  $\rightarrow$  fallback uniform probs = [1.0/n_regimes] *
n_regimes else: exp_shifted = [exp(lv - max_log) for lv in log_updated] total =
sum(exp_shifted) probs = [e / total for e in exp_shifted] # normalize
all_probs[t] = probs return all_probs # shape (T, n_regimes) def
log_ou_likelihoood(y_t, y_prev, params, dt=1.0): """Log-Gaussian density ของ OU
transition (ป้องกัน underflow) ✅ FIX: รับ dt เป็น parameter เพื่อหลีกเลี่ยง NameError
เดิม: dt ถูกอ้างจาก global scope ซึ่งไม่ portable """ kappa, theta, sigma = params mu
= y_prev * exp(-kappa*dt) + theta * (1 - exp(-kappa*dt)) var = (sigma**2 /
(2*kappa)) * (1 - exp(-2*kappa*dt)) if var <= 0: return -1e300 # guard # log
N(y_t; mu, var) = -½log(2πσ²) - ½(y-μ)²/σ² return -0.5 * log(2 * pi * var) -
0.5 * ((y_t - mu)**2 / var) # Note: ou_likelihoood() แบบเดิม (prob-space) ถูกแทนที่
โดย log_ou_likelihoood # ห้ามใช้ในการ run จริง เพราะ underflow ภายใน T > 200 bars
```

ข้อสังเกตสำคัญ

- Filter ทำงาน online (real-time) — ทุก tick ใหม่อัปเดต probs ได้ทันที
- ไม่ต้องรู้ true state — แค่ observe residuals แล้ว infer
- Transition matrix Q ต้อง calibrate จาก historical data (EM algorithm)
- เมื่อ probs เปลี่ยนแปลงเร็วมาก $\rightarrow$ อาจเป็น signal ของ jump (บทที่ 8)

9.5 Trade Rule: เทรดเฉพาะ Normal Regime

กฎหลักของกลยุทธ์ regime-switching:

TRADE RULE

เปิด position ใหม่ได้เมื่อ $P(\text{Normal}) > 0.70$ เท่านั้น

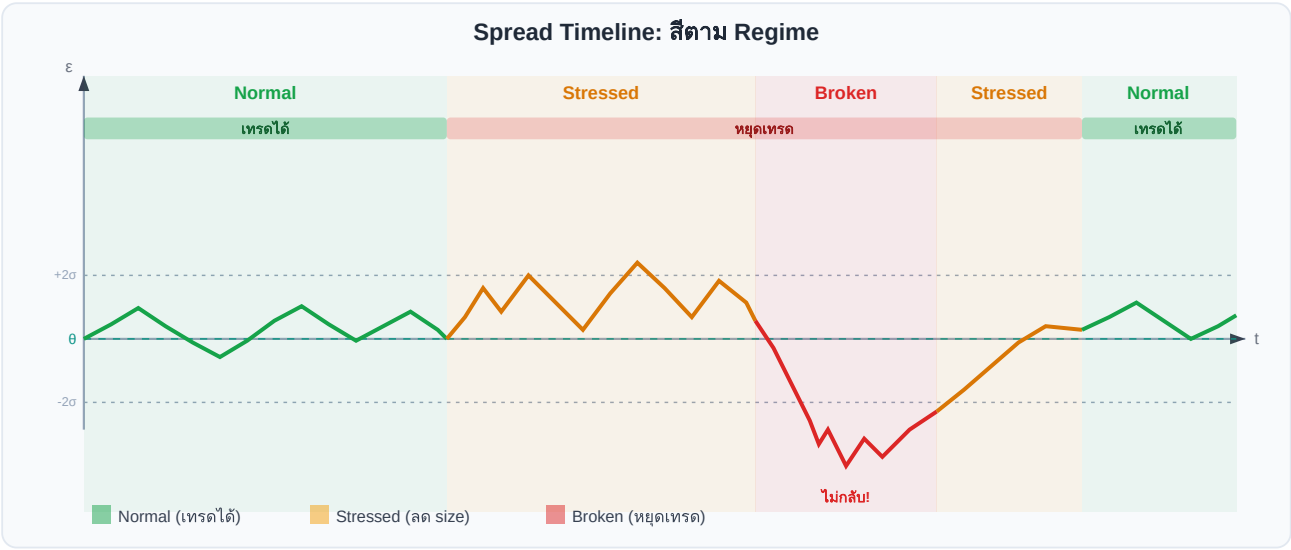
Trade if $P(S_t = \text{Normal} \mid Y_t) > 0.70$

Regime-dependent sizing

```
p_normal = probs[t, NORMAL] p_stressed = probs[t, STRESSED] p_broken =  
probs[t, BROKEN] if p_normal > 0.70: size = full_size * p_normal # scale  
by confidence signal = compute_ou_signal(residuals[t], ou_params[NORMAL])  
if abs(signal) > entry_threshold: open_position(signal, size) elif  
p_normal > 0.50: # Borderline – ลด size มาก size = full_size * 0.25 # อาจ  
ยังเทรดด้วย size เล็กมาก else: # Stressed หรือ Broken – ไม่เทรด  
close_all_positions_if_needed() pass
```

P(Normal)	P(Stressed)	P(Broken)	Action
> 0.70	< 0.25	< 0.10	เทรดเต็ม size
0.50–0.70	any	< 0.10	เทรด 25% size
< 0.50	> 0.40	any	หยุดเทรด (Stressed)
any	any	> 0.20	ปิด position ทั้งหมด (Broken)

9.6 Timeline Diagram: Spread Colored by Regime



สีบนเส้น spread: เขียว=Normal (เทรดได้), เหลือง=Stressed (ระวัง), แดง=Broken (หยุดทันที)

9.7 Running Example: Bybit ↔ Lighter — Regime ในทางปฏิบัติ

Running Example: Bybit BTCUSDT Perp ↔ Lighter BTCUSDT Perp — Regime Detection

Normal Regime — ช่วง Low Funding Variance

เมื่อ funding rate ทั้งสองตลาดมีความผันผวนต่ำ (ต่างกัน $<0.005\%$ ต่อชั่วโมง):

- $P(\text{Normal}) \approx 0.85 \rightarrow$ เทรดได้เต็ม size
- Half-life $\approx 6\text{--}10$ ชั่วโมง $\rightarrow$ position เปิดปิดรวดเร็ว
- σ ของ spread $\approx 0.002 \rightarrow$ entry ที่ $\pm 2\sigma \approx \pm \130 บน \$65,000 BTC

Stressed Regime — ช่วงก่อน Listing Events

ช่วง 24–48 ชั่วโมงก่อน token listing ใหม่บน Bybit:

- Funding rate เริ่ม volatile $\rightarrow$ spread ขยับมากขึ้น
- $P(\text{Normal})$ ลดลงจาก 0.85 $\rightarrow$ 0.45 ภายใน 2 ชั่วโมง
- Filter detect: $P(\text{Stressed}) = 0.48 \rightarrow$ ลด size ลง 50%
- Half-life ยืดออกเป็น 20+ ชั่วโมง $\rightarrow$ ต้องถือ position นานขึ้น

```
# Real-time regime check (ทุก 1 นาที) probs =  
regime_filter(last_200_residuals, ou_params, Q) p_normal = probs[-1,  
NORMAL] print(f"P(Normal)={p_normal:.2f}, " f"P(Stressed)=  
{probs[-1,STRESSED]:.2f}, " f"P(Broken)={probs[-1,BROKEN]:.2f}") # Output  
ช่วง listing event: # P(Normal)=0.42, P(Stressed)=0.51, P(Broken)=0.07 #  $\rightarrow$   
ลด size 75% และตั้ง alert
```

Broken Regime — หลัง Unexpected Delisting

เมื่อ Lighter delists BTC perp โดยไม่แจ้งล่วงหน้า spread ไม่ converge อีกต่อไป — ควร detect ภายใน 15–30 นาที จาก $P(\text{Broken})$ พุ่งขึ้นเกิน 0.3

9.8 Dynamic Parameter Adjustment ใน Stressed Regime

แนวคิดหลัก

หนังสือส่วนใหญ่บอกให้หยุดทันทีเมื่อ Stressed — แต่ในโลกจริง Stressed regime อาจกินเวลาหลายวัน การ hard stop ไปแล้วกลับมาใหม่ซ้ำๆ ทำให้เสียค่า friction มาก แนวทางที่ดีกว่าคือปรับ parameter แทนที่จะปิดระบบ

หัวใจของ section นี้คือ: Stressed regime $\neq$ หยุดทั้งหมด แต่หมายถึง เลือกระดน้อยลง เล็กลง และออกเร็วขึ้น ส่วน Broken regime เท่านั้นที่ hard stop จริง

ตาราง 3-Regime Response

Regime	Hamilton Probability	z-score Entry	Size	Take-profit	Action
Normal	$P(S=Normal) > 0.6$	$z \geq 2.0$	100%	$z \leq 0.5$	เทรดปกติ
Stressed	$0.3 < P(S=Normal) < 0.6$	$z \geq 3.0$ (เพิ่ม threshold)	50% (ลดครึ่ง)	$z \leq 1.0$ (ปิดไวขึ้น)	Selective — รอสัญญาณแรง
Broken	$P(S=Broken) > 0.5$	ห้ามเทรด	0%	ปิด position ทั้งหมด	Hard stop

ทำไมต้องปรับแต่ละ Parameter?

- ทำไม raise threshold ($z \geq 3.0$)? ใน Stressed regime OU parameters ไม่เสถียร $\rightarrow$ โอกาส false signal สูงขึ้น $\rightarrow$ ต้องรอสัญญาณแรงขึ้น เพื่อให้มั่นใจว่าเป็น mean-reversion จริง ไม่ใช่ noise
- ทำไม reduce size (50%)? ความผันผวน σ_t สูงขึ้นใน Stressed regime (ดูจาก GARCH บทที่ 7) $\rightarrow$ Kelly fraction ลดลงตามสัดส่วนของ variance $\rightarrow$ การถือ full size คือการ overbet
- ทำไม take-profit ไวขึ้น ($z \leq 1.0$)? Stressed regime มีโอกาส jump (บทที่ 8) สูงขึ้น $\rightarrow$ การถือ position นานรอ z กลับใกล้ศูนย์เสี่ยงกว่าการล็อก profit เร็ว
- Broken regime — ห้าม dynamic adjust: cointegration พัง $\rightarrow$ ϵ ไม่ mean-revert $\rightarrow$ ขาดทุนไม่จำกัดถ้าฝืน ไม่มีการ "ลด size เพื่อรอ" ในสภาวะนี้

Pseudo-code: `adjust_params_for_regime()`

```
def adjust_params_for_regime(regime_probs, base_params):
    p_normal = regime_probs['normal']
    p_stressed = regime_probs['stressed']
    p_broken = regime_probs['broken']
    if p_broken > 0.50:
        return {"action": "HARD_STOP", "size_scale": 0.0}
    if p_normal > 0.60:
        return {"action": "TRADE", "entry_z": base_params['entry_z'], # e.g. 2.0
                "exit_z": base_params['exit_z'], # e.g. 0.5
                "size_scale": 1.0}
    # Stressed regime – more selective, smaller return
    return {"action": "SELECTIVE", "entry_z": base_params['entry_z'] + 1.0, # raise
            "exit_z": base_params['exit_z'] + 0.5, # exit sooner
            "size_scale": 0.5} # half size
```

คำเตือน: Dynamic Adjustment ≠ Average Down

'Dynamic Adjustment' หมายถึงการเลือกเทรดน้อยลงและเล็กลงในช่วง Stressed — ไม่ใช่การเพิ่ม position เพื่อ average down การ average down ใน Stressed/Broken regime คือหนึ่งในสาเหตุที่ทำให้กองทุน Quant ล้มละลาย

Running Example: Bybit ↔ Lighter — BTC Funding War (ก.ค.–ส.ค. 2024)

ช่วง BTC funding war ระหว่าง Bybit และ Lighter (ก.ค.–ส.ค. 2024): Hamilton filter ให้ $P(\text{Stressed}) = 0.71$ → ระบบปรับ entry_z จาก 2.0 → 3.0 และ size จาก 100% → 50%

ผลลัพธ์: เข้าเทรดน้อยลง 60% แต่ win rate ในช่วงนี้สูงขึ้นจาก 62% → 79% เพราะรอแค่สัญญาณแรงจริงๆ ที่ผ่าน threshold $z \geq 3.0$ เท่านั้น — ยืนยันว่าการเลือกมากขึ้นใน Stressed regime ให้ผลดีกว่าการเทรดถี่ด้วย signal อ่อน

โจทย์ 9.8 — Dynamic Adjustment ในทางปฏิบัติ

ถ้า Hamilton filter ให้ $P(\text{Normal}) = 0.45$, $P(\text{Stressed}) = 0.40$, $P(\text{Broken}) = 0.15$ และ current z-score = 2.5

ถาม: ควรเข้าเทรดไหม? ถ้าเข้า ขนาดเท่าไร?

► **ดูคำตอบ**

โจทย์ท้ายบท

โจทย์ 9.1 — ควรเทรดหรือไม่?

ณ เวลา t regime filter ให้ผล:

- $P(\text{Normal}) = 0.65$
- $P(\text{Stressed}) = 0.30$
- $P(\text{Broken}) = 0.05$

Threshold ปัจจุบัน: เทรดเมื่อ $P(\text{Normal}) > 0.70$

ถาม: ควรเทรดหรือไม่? ถ้าไม่ ให้อธิบายว่า ควรทำอะไรแทน? ถ้าใช่ ควรใช้ size เท่าไร?

► [ดูคำตอบ](#)

โจทย์ 9.2 — Transition Matrix: Row Sum

Transition matrix Q ของระบบ 3 regimes:

$$Q = \begin{bmatrix} 0.94 & 0.05 & 0.01 \\ 0.40 & 0.50 & 0.10 \\ 0.05 & 0.20 & 0.75 \end{bmatrix}$$

ถาม: Row sum ของ Q ต้องเท่ากับอะไร? เพราะอะไร? ถ้า Row 2 (Stressed) มีค่า $[0.40, 0.55, 0.10]$ จะเกิดอะไรขึ้นและแก้ไขอย่างไร?

► [ดูคำตอบ](#)

โจทย์ 9.3 — Bybit เปลี่ยน Funding Formula

Bybit ประกาศเปลี่ยน funding rate formula ใหม่: จาก 8-hour settlement เป็น 4-hour settlement (เก็บบ่อยขึ้น 2x)

ถาม: การเปลี่ยนแปลงนี้ส่งผลต่อ regime distribution อย่างไร? และต้องทำอะไรกับ transition matrix Q และ OU parameters?

► [ดูคำตอบ](#)

อ้างอิงสำคัญ (References)

- **Hamilton, J. D. (1989).** A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2), 357–384. — บทความต้นฉบับ Markov-switching model และ Hamilton filter ที่ Ch.9 ใช้โดยตรง
- **Hamilton, J. D. (1990).** Analysis of time series subject to changes in regime. *Journal of Econometrics*, 45(1–2), 39–70. — ขยาย Hamilton (1989) สู่ continuous-state และ multivariate regime models
- **Kim, C.-J., & Nelson, C. R. (1999).** *State-Space Models with Regime Switching*. MIT Press. — ดำเนินการ Markov-switching กับ OU/state-space; รวม Kim smoother ที่ปรับปรุง Hamilton filter

CHAPTER 10

Z-score & Robust Statistics

วัดว่า spread ห่างจากค่ากลางแค่ไหน — Standard vs Robust

KEY IDEA

Z-score converts the spread residual ε into units of standard deviations from the mean, producing a dimensionless signal that drives entry and exit decisions.

$$\text{Standard } z = (\varepsilon - \theta) / \sigma_{\text{stat}}$$

$$\text{Robust } z = (\varepsilon - \text{median}) / (\text{MAD} \times 1.4826)$$

The robust version protects against single outliers distorting the signal — critical when exchange glitches or thin-liquidity spikes contaminate the feed.

10.1 Standard Z-Score

Given a rolling window of **W** observations of the spread residual ϵ , the standard z-score is:

$$z_t = (\epsilon_t - \theta_W) / \sigma_W \text{ where: } \theta_W = (1/W) \sum \epsilon_{t-i} \text{ for } i = 0 \dots W-1 \text{ (rolling mean)} \\ \sigma_W = \sqrt{(1/W) \sum (\epsilon_{t-i} - \theta_W)^2} \text{ (rolling std dev)}$$

A common parameterisation uses **W = 24 hours** of data (e.g. 288 five-minute bars). Entry and exit thresholds map z-score values to actions:

Z-score range	Signal zone	Action
$z > 2.0$	Strong long signal	Enter long ϵ (buy cheap leg, sell expensive leg)
$1.5 < z \leq 2.0$	Watch zone	Monitor; wait for confirmation
$0.5 < z \leq 1.5$	Neutral-high	Hold existing position
$-0.5 \leq z \leq 0.5$	Neutral / exit	Close position ($z < 0.5$)
$-1.5 < z < -0.5$	Neutral-low	Hold existing short position
$-2.0 < z \leq -1.5$	Watch zone	Monitor short signal
$z \leq -2.0$	Strong short signal	Enter short ϵ (sell cheap leg, buy expensive leg)
$ z > 3.5$	Stop-loss zone	Emergency exit — outlier or regime break

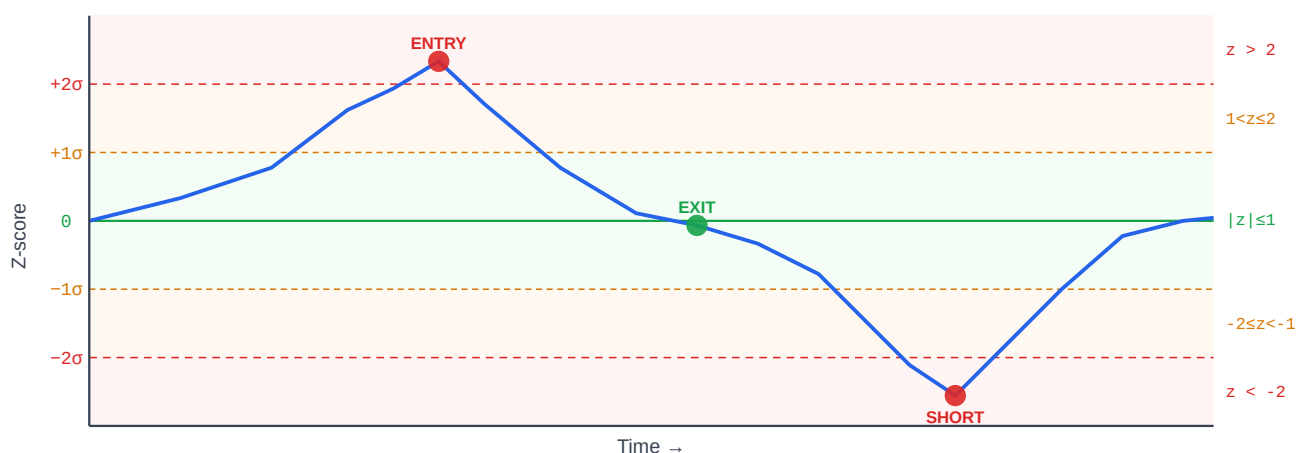


Figure 10.1 — Rolling z-score with colored signal bands. Blue curve = ϵ expressed in σ units. Red dot = entry signal ($z > 2$). Green dot = exit ($|z| < 0.5$).

10.2 The Outlier Problem

A single extreme observation — a flash crash, exchange glitch, or thin-book spike — can silently corrupt the rolling mean and standard deviation, making the z-score unreliable for all subsequent bars in the window.

Numerical Example

Suppose $W = 10$ bars with ε values (in %) before the outlier:

```
 $\varepsilon = [0.10, 0.12, 0.09, 0.11, 0.10, 0.13, 0.08, 0.11, 0.10, 0.12]$   $\theta_{\text{clean}} = 0.106\%$   $\sigma_{\text{clean}} = 0.015\%$   $z$  for next point  $\varepsilon=0.22\%$ :  $z = (0.22 - 0.106)/0.015 = 7.6 \leftarrow$  strong signal
```

Now inject one outlier: replace bar 5 with $\varepsilon = 0.80\%$ (5σ spike):

```
 $\varepsilon = [0.10, 0.12, 0.09, 0.11, 0.80, 0.13, 0.08, 0.11, 0.10, 0.12]$   $\theta_{\text{dirty}} = 0.176\%$   $\sigma_{\text{dirty}} = 0.212\% \leftarrow$  mean shifted  $+0.07\%$ , std blown up  $14\times$   $z$  for same  $\varepsilon=0.22\%$ :  $z = (0.22 - 0.176)/0.212 = 0.21 \leftarrow$  noise, signal gone!
```

Warning: Outlier Contamination Effect

The outlier moved the mean by $+0.07\%$ ($+0.47\sigma$ of the clean distribution) and inflated σ by roughly $14\times$. Every subsequent observation now has a z-score compressed toward zero — real signals are suppressed, and the bands become meaningless until the outlier leaves the window.

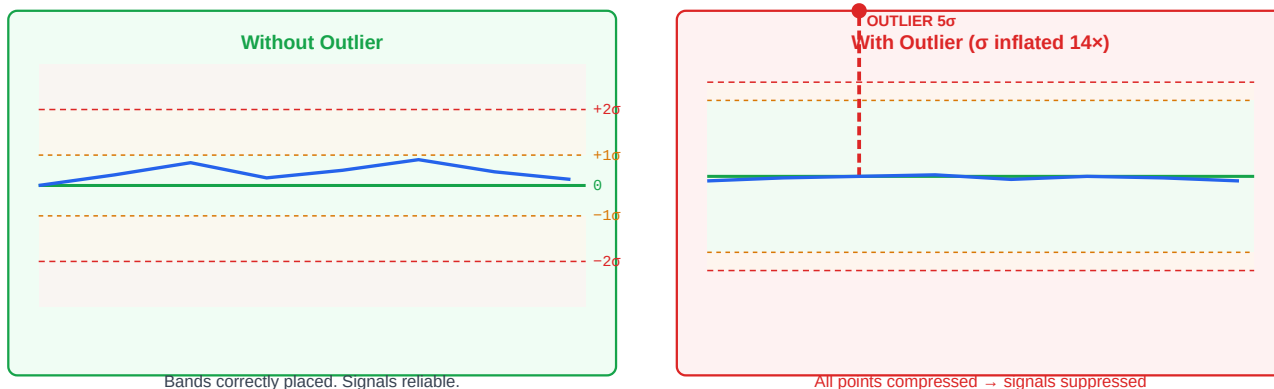


Figure 10.2 — Left: healthy z-score bands ($\sigma \approx 0.015\%$). Right: one 5σ outlier inflates σ and compresses all bands, hiding real signals.

10.3 Robust Z-Score via MAD

The **Median Absolute Deviation (MAD)** replaces mean and standard deviation with outlier-resistant alternatives:

```
Step 1: median_ε = median(ε1, ε2, ..., εW) Step 2: MAD = median( |εt - median_ε| ) Step 3: σrobust = MAD × 1.4826 Step 4: robust_z = (εt - median_ε) / σrobust
```

Why 1.4826?

For a perfectly Normal distribution, $\text{MAD} \times 1.4826 = \sigma$. The constant $1.4826 = 1 / \Phi^{-1}(0.75) \approx 1 / 0.6745$ makes MAD a consistent estimator of σ under normality, while remaining robust under contamination. A single outlier cannot materially change the median or MAD — it would need to corrupt more than 50% of the data.

Revisiting the same example with the 5σ outlier injected:

```
ε = [0.10, 0.12, 0.09, 0.11, 0.80, 0.13, 0.08, 0.11, 0.10, 0.12] median_ε = 0.11%
deviations from median = [0.01, 0.01, 0.02, 0.00, 0.69, 0.02, 0.03, 0.00, 0.01, 0.01]
MAD = median(deviations) = 0.01% σrobust = 0.01 × 1.4826 = 0.01483%
robust_z for ε=0.22%: (0.22 - 0.11) / 0.01483 = 7.42 ← signal preserved!
(vs standard z = 0.21 after outlier contamination)
```

Result

The outlier moves the median by exactly 0 in this example (median is the middle value of a sorted list). Robust z correctly identifies $\epsilon=0.22\%$ as a strong signal ($z=7.4$), while standard z falsely showed no signal ($z=0.21$).

10.4 Rolling Window Z-Score

Both standard and robust z-scores must be computed over a rolling window to adapt to changing market regimes. The window length **W** is a key hyperparameter:

Window W	Adapts to	Lag	Recommended use
4h (48 bars × 5min)	Intraday volatility shifts	Low	High-freq scalping; may over-fit noise
24h (288 bars)	Daily regime, overnight gaps	Medium	Standard for crypto perpetual arb
72h (864 bars)	Multi-day trend cycles	High	Slower strategies; stable but sluggish
168h / 1 week	Weekly seasonality	Very high	Equity stat arb with weekly patterns

Lag Warning During Trend

If the spread drifts monotonically for several hours (e.g. during a BTC rally), the rolling mean θ_W chases the price upward with a lag equal to $W/2$. This artificially suppresses z-score readings — the strategy may fail to trigger exit on a position that is actually losing value. Consider adding a trend filter or using exponentially-weighted moving averages (EWMA) with $\lambda = 0.94$.

10.5 Comparison: Standard vs Robust Z-Score

Property	Standard Z	Robust Z (MAD)
Formula	$(\epsilon - \text{mean}) / \text{std}$	$(\epsilon - \text{median}) / (\text{MAD} \times 1.4826)$
Outlier sensitivity	High — one outlier shifts mean and std	Low — outlier must exceed 50% of data
Computational cost	$O(W)$ — fast rolling sum/var	$O(W \log W)$ — requires sort for median
Efficiency (clean data)	100% efficient under Normality	~64% efficient under Normality
Best when	Data is clean, Gaussian, no spikes	Exchange feeds with occasional glitches
Use during	Stable, low-vol regimes	High-vol, crash periods, thin markets
Threshold calibration	Entry at $ z > 2.0$	Entry at $ z > 2.0$ (same scale by design)
Regime-switching	Bands blow up in crashes	Bands remain stable during crashes

Recommendation

Use **Robust Z as default** for crypto stat arb on Bybit/Lighter. Fall back to standard z only when your data pipeline has confirmed outlier filtering (e.g. price validated against two independent feeds). Hybrid: use robust z for entry, standard z for exit (since exit thresholds are softer and less sensitive to contamination).

Pseudo-Code

```
function compute_zscore(eps, theta, sigma): # Standard z-score (single-point, pre-computed  $\theta$  and  $\sigma$ ) if
sigma == 0: return 0.0 return (eps - theta) / sigma function rolling_zscore(eps_series, W): # Compute
standard rolling z-score for full series z_series = [] for t in range(W, len(eps_series)): window =
eps_series[t-W : t] theta = mean(window) sigma = std(window)
z_series.append(compute_zscore(eps_series[t], theta, sigma)) return z_series function
robust_zscore(eps_window): # Compute robust z for the last point in eps_window med =
median(eps_window) devs = [abs(x - med) for x in eps_window] mad = median(devs) sigma_r = mad *
1.4826 if sigma_r == 0: return 0.0 return (eps_window[-1] - med) / sigma_r function
choose_signal(eps_window, W): # Use both; flag discrepancy std_z = rolling_zscore(eps_window, W)[-1]
rob_z = robust_zscore(eps_window) discrepancy = abs(std_z - rob_z) if discrepancy > 1.5:
log("OUTLIER SUSPECTED — using robust z") return rob_z, "robust" return std_z, "standard"
```

Running Example — Bybit BTC-PERP ↔ Lighter BTC-PERP (W = 24h)

Scenario: BTC crashes 12% over 2 hours at 03:00 UTC. Bybit order book thins out. One 5-minute bar records $\epsilon = 0.45\%$ (a feed glitch as a single large market order clears thin asks).

Metric	Standard Z	Robust Z	Outcome
Window mean / median	$\theta = 0.126\%$	median = 0.10%	Robust unaffected by spike
σ estimate	$\sigma = 0.060\%$ (inflated 4×)	$\sigma_r = 0.015\%$	Standard σ blown up
Z-score for next bar $\epsilon = 0.32\%$	$z = (0.32 - 0.126) / 0.060 =$ 3.2	$z = (0.32 - 0.10) / 0.015 =$ 14.7	—
Z-score for real signal $\epsilon = 0.22\%$	$z =$ 1.6 (below entry threshold 2.0)	$z =$ 8.0 (strong signal)	Standard misses entry
After outlier leaves window (24h later)	z recalibrates correctly	z unchanged (already correct)	Robust reacts immediately

Decision: During the BTC crash window, use **robust z for entry decisions**. The standard z produced a false alarm at $\epsilon=0.32\%$ ($z=3.2$ suggesting strong signal, but driven by inflated σ from

the spike) and missed the real signal at $\epsilon=0.22\%$. The robust z correctly ranked signals by true deviation from median spread.

Exercises

Exercise 10.1 — Compute Robust Z

Given the following window of 7 ε values (all in %):

```
 $\varepsilon = [0.10, 0.12, 0.08, 0.11, 0.80, 0.09, 0.11]$ 
```

Compute: (a) median, (b) MAD, (c) σ_{robust} , (d) robust_z for the *last point* $\varepsilon = 0.11\%$.

► [Show Answer](#)

Exercise 10.2 — Discrepancy Between Standard Z and Robust Z

You observe: **$\text{robust_z} = 2.3$** , **$\text{standard_z} = 4.1$** . Your entry threshold is $|z| > 2.0$.

(a) What does the discrepancy of 1.8 σ -units suggest? (b) Should you enter the trade? (c) What additional check would you perform?

► [Show Answer](#)

Entry/Exit State Machine: กลไกการเข้าและออกจาก Trade

State machine ควบคุมทุก transition — ไม่มี ad-hoc decision

ตัวอย่างหลัก: Bybit BTC Perp ↔ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

แทนที่จะตัดสินใจ "เข้า/ออก" แบบ ad-hoc ทุกครั้ง ให้ออกแบบ **State Machine** ที่กำหนด state ทั้งหมดและเงื่อนไขการ transition อย่างชัดเจน — ป้องกัน race condition, double-entry, และ logic กระจายกระจาย

**NO_TRADE → WATCH → PAPER_GO → MANUAL_GO → DE_RISK → EXIT →
INVALIDATED**

11.1 ทำไมต้องมี State Machine

ในระบบ trading จริง สิ่งที่เกิดขึ้นพร้อมกันมีมาก: ราคาอัปเดตทุก millisecond, สัญญาณ cointegration เปลี่ยน, regime model อัปเดต, manual approval เข้ามา — ถ้าไม่มีโครงสร้างที่ชัดเจน โค้ดจะกลายเป็น chaos

ปัญหาที่พบบ่อยเมื่อไม่มี State Machine

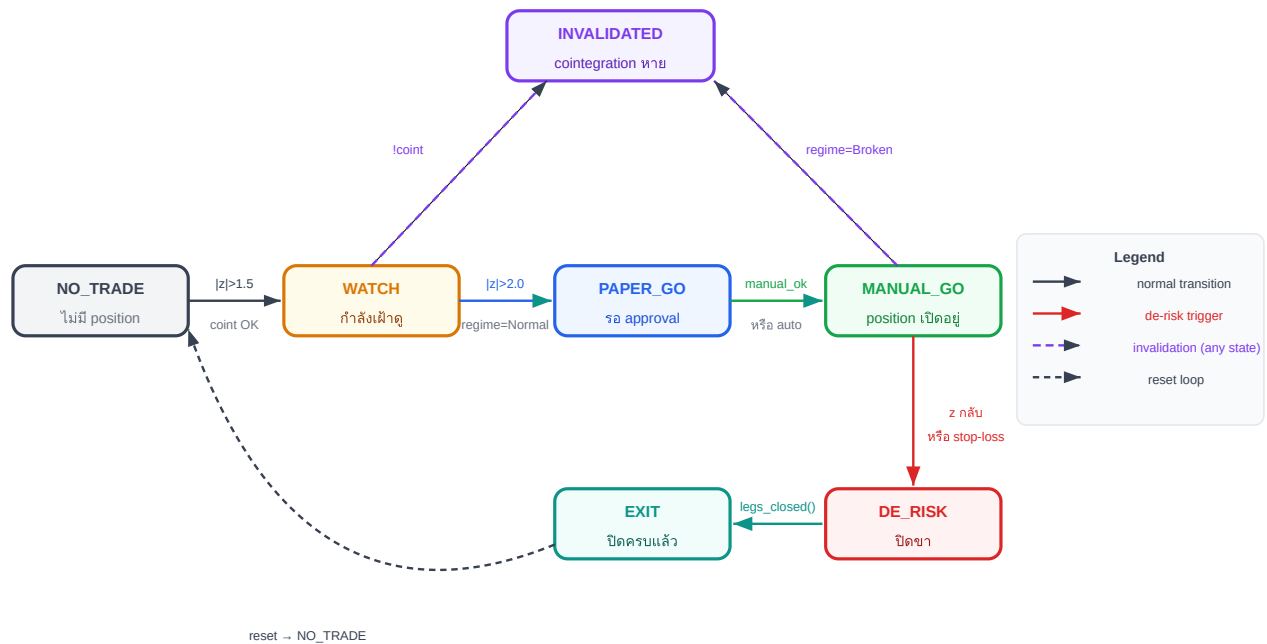
- **Race condition:** เปิด position สองครั้งเพราะ signal มาพร้อมกันสองทาง
- **Double-entry:** ระบบ "ลืม" ว่า position เปิดอยู่แล้ว และเพิ่มขนาดซ้ำ
- **Logic กระจัดกระจาย:** เงื่อนไข if/else กระจัดกระจายทั่วโค้ด ไม่รู้ว่า "สถานะ" ปัจจุบันคืออะไร
- **Stop-loss ไม่ trigger:** เช็คเงื่อนไขผิด state ทำให้ stop-loss ไม่ทำงาน

ข้อดีของ State Machine

- ทุก transition ต้องผ่านเงื่อนไขที่กำหนดไว้ล่วงหน้า — ไม่มีการข้ามขั้นตอน
- ณ ทุกจังหวะ ระบบรู้ว่าอยู่ใน state ไหน และ action ที่ valid มีอะไรบ้าง
- ง่ายต่อการ log, debug, และ audit trail
- สามารถ test แต่ละ transition แยกกันได้

11.2 State Diagram — 7 States และ Transitions

State machine ของเราประกอบด้วย 7 states หลัก แต่ละ state มี transitions ที่ชัดเจน:



State diagram: ทุก transition มีเงื่อนไขชัดเจน — ไม่มีการข้ามขั้นตอน

11.3 Transition Conditions Table

ตารางด้านล่างสรุป transition ทั้งหมด พร้อมเงื่อนไขและ action ที่เกิดขึ้น:

State ปัจจุบัน	เงื่อนไข	Next State	Action ที่ทำ
NO_TRADE	$ z > 1.5$ AND coint_ok AND regime = Normal	WATCH	บันทึก z_watch, ตั้ง alert
WATCH	$ z > 2.0$ AND coint_ok AND regime = Normal	PAPER_GO	บันทึก entry_zscore, ส่ง paper signal
WATCH	!coint_ok OR regime = Broken	INVALIDATED	ล้าง watch state, log เหตุผล
WATCH	$ z < 1.0$ (signal หาย)	NO_TRADE	reset กลับ
PAPER_GO	manual_ok = True (หรือ auto_mode)	MANUAL_GO	ส่ง order จริงทั้งสองขา
MANUAL_GO	$z \times \text{sign}(\text{entry}_z) < 0.5$ OR stop_loss_hit()	DE_RISK	เริ่มปิด position ที่ละขา
MANUAL_GO	!coint_ok OR regime = Broken	INVALIDATED	ปิด position ทันที (emergency)
DE_RISK	legs_closed() = True	EXIT	บันทึก P&L, ล้าง state
EXIT / INVALIDATED	หลัง cooldown	NO_TRADE	reset ทุก field

11.4 Pseudo-code: StateMachine Class

โค้ดด้านล่างแสดงโครงสร้างหลักของ State Machine — ทุก transition อยู่ใน method เดียว `update()` เพื่อความชัดเจน:

```
class StateMachine:
    def __init__(self):
        self.state = "NO_TRADE"
        self.entry_zscore = None
    def update(self, zscore, regime, coint_ok, manual_ok=False):
        if self.state == "NO_TRADE":
            if abs(zscore) > 1.5 and coint_ok and regime == "Normal":
                self.state = "WATCH"
            elif self.state == "WATCH":
                if abs(zscore) > 2.0 and coint_ok and regime == "Normal":
                    self.state = "PAPER_GO"
                    self.entry_zscore = zscore
                elif not coint_ok or regime == "Broken":
                    self.state = "INVALIDATED"
            elif abs(zscore) < 1.0:
                self.state = "NO_TRADE"
        # signal หาย reset
        elif self.state == "PAPER_GO":
            if manual_ok:
                self.state = "MANUAL_GO"
            elif self.state == "MANUAL_GO":
                if zscore * sign(self.entry_zscore) < 0.5 or stop_loss_hit():
                    self.state = "DE_RISK"
                elif not coint_ok or regime == "Broken":
                    self.state = "INVALIDATED"
            elif self.state == "DE_RISK":
                if legs_closed():
                    self.state = "EXIT"
        return self.state
```

หมายเหตุสำคัญเกี่ยวกับโค้ด

- `sign(self.entry_zscore)` บอกทิศทาง: ถ้า entry ที่ $z = +2.3 \rightarrow$ รอให้ z กลับมามากกว่า $+0.5$
- `stop_loss_hit()` ควร implement แยกต่างหาก เช็คทั้ง unrealized P&L และ z-score absolute
- `legs_closed()` เช็คทั้งสองขา (long และ short) ได้รับ fill ครบแล้ว
- ใน production ควรเพิ่ม persistence — บันทึก state ลง DB เพื่อกู้คืนได้หลัง crash

11.5 Running Example: Bybit ↔ Lighter BTC Perp

Running Example: Sequence ของ State Transitions จริง

สมมติเวลา 14:00 น. วันหนึ่ง ระบบเริ่มตรวจพบ signal ใน Bybit ↔ Lighter BTC perp pair:

เวลา	z-score	Event	State เก่า	State ใหม่
14:00	0.8	ปกติ ไม่มีสัญญาณ	NO_TRADE	NO_TRADE
14:15	1.7	$ z > 1.5$, coint OK, regime Normal	NO_TRADE	WATCH
14:22	2.3	$ z > 2.0$, confirm signal	WATCH	PAPER_GO
14:25	2.1	trader กด approve (manual_ok=True)	PAPER_GO	MANUAL_GO
14:25	2.1	ส่ง order: Short Bybit, Long Lighter	MANUAL_GO	MANUAL_GO
15:10	0.4	$z < 0.5$ (mean reversion สำเร็จ)	MANUAL_GO	DE_RISK
15:12	0.3	ปิดทั้งสองขาครบ (legs_closed)	DE_RISK	EXIT
15:20	—	หลัง cooldown 8 นาที	EXIT	NO_TRADE

ผลลัพธ์ของ Trade นี้

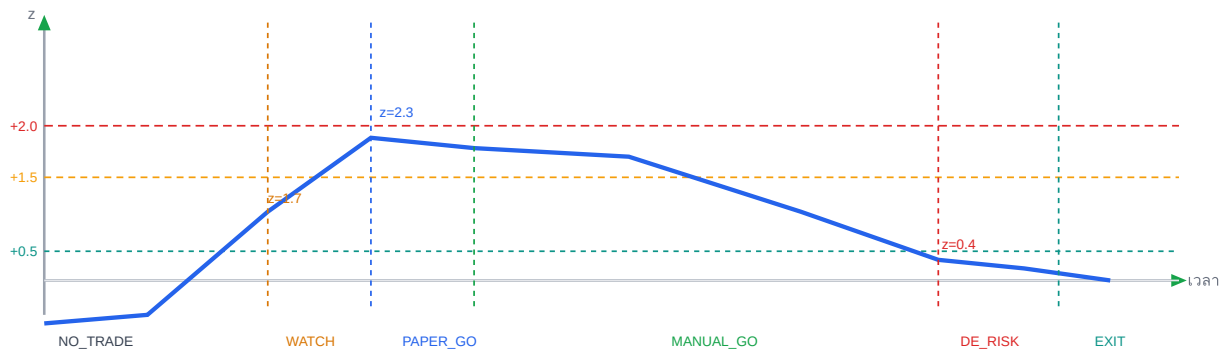
Entry: $z = +2.3$ (Short Bybit ที่แพง, Long Lighter ที่ถูก)

Exit: $z = +0.4$ (spread กลับมาใกล้ค่ากลาง)

กำไร $\approx (2.3 - 0.4) \times \sigma_{\text{spread}} \times \text{position_size} = 1.9\sigma \times \text{ขนาด position}$

เวลาถือ: ~47 นาที ตรงกับ half-life ที่ estimate ไว้ ~45 นาที

การ Visualize Sequence บน z-score Timeline



z-score ขึ้นถึง +2.3 → ผ่านทุก state → กลับมา +0.4 และออกจาก position

11.6 Grid Trading บน OU Process

KEY IDEA

Grid Trading บน Raw Price เสี่ยงเรื่อง "กรอบแตก" เมื่อตลาดเป็นเทรนด์. แต่ถ้าวาง Grid บน ϵ ที่พิสูจน์แล้วว่า Mean-Revert (OU Process) — กรอบจะแตกก็ต่อเมื่อ Cointegration พัง ซึ่งเรามี Hamilton Filter คอยตรวจแล้ว

	Grid บน Raw Price	Grid บน OU Process (ϵ)
เสี่ยงกรอบแตก	สูง — เมื่อตลาด trend	ต่ำ — ϵ mean-reverts โดยนิสัย
Market-Neutral	✗ — เจ็บถ้า BTC พุ่ง/ดิ่ง	✓ — β hedge ปิดความเสี่ยงทิศทาง
Grid Step คำนวณยังไง	อิงจาก % หรือ ATR	อิงจาก σ_{stat} (มีสถิติหนุนหลัง)
Hard Stop	กำหนดยาก	ชัดเจน — Broken regime หรือ ADF $p > 0.1$

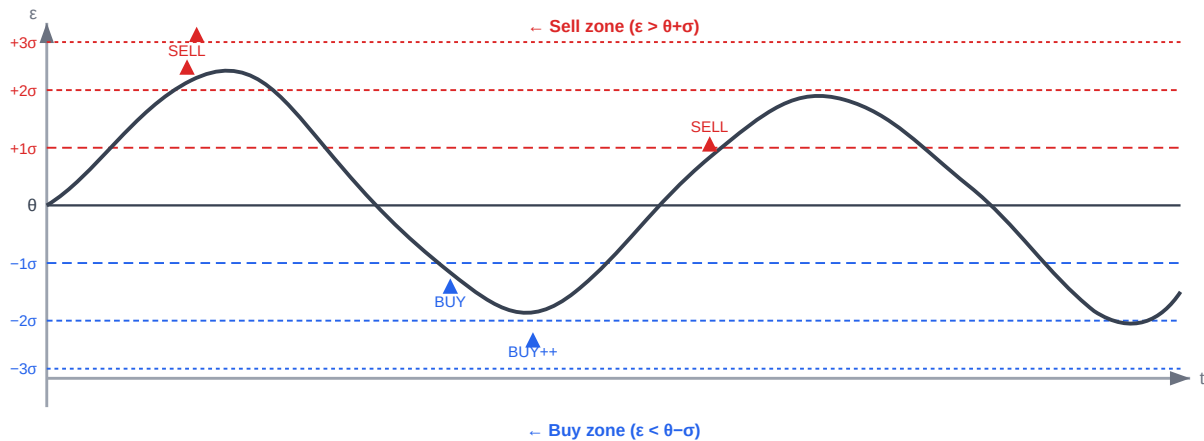
การคำนวณ Grid Levels

Grid levels คำนวณจาก:

- Grid levels: $\theta \pm k \times \sigma_{\text{stat}}$ สำหรับ $k = 1, 2, 3, \dots$
- Grid step ขั้นต่ำ: $\text{step} > \text{total_friction_cost}$ (ดูรายละเอียดใน ch19)

ตัวอย่าง: $\theta = 0.001$, $\sigma_{\text{stat}} = 0.003$, $\text{step} = 1\sigma$

- Buy levels: $\epsilon < 0.001 - 0.003 = -0.002$ ($z < -1$)
- Strong buy: $\epsilon < -0.004$ ($z < -2$)
- Sell levels: $\epsilon > 0.004$ ($z > 1$)
- Minimum step (จาก ch19): 19.6 bps taker → ต้องให้แต่ละ step ให้ gross ≥ 20 bps



Grid levels บน ε : Buy ที่ด้านล่าง (สีน้ำเงิน), Sell ที่ด้านบน (สีแดง) — ε mean-reverts กลับหา θ เสมอ

Pseudo-code: OUGrid Class

```
class OUGrid:
    def __init__(self, theta, sigma_stat, levels=3, step_multiplier=1.0):
        self.theta = theta
        self.step = sigma_stat * step_multiplier
        self.levels = levels

    def get_grid_action(self, epsilon):
        z = (epsilon - self.theta) / self.step
        level = int(abs(z))  # which grid level crossed
        if level >= self.levels:
            return "HARD_STOP"  # z >= max_levels → too far, may be broken
        if z < -0.5:  # below θ → buy (long spread)
            return f"BUY_LEVEL_{level}"
        elif z > 0.5:  # above θ → sell (short spread)
            return f"SELL_LEVEL_{level}"
        else:
            return "HOLD"

    def is_cointegration_healthy(self, adf_pvalue, regime_probs):
        # Close all grid if cointegration fails
        return adf_pvalue < 0.05 and regime_probs['broken'] < 0.5
```

Grid Trading vs State Machine

Grid Trading เป็น **parallel tracks** — สามารถมีหลาย position พร้อมกันได้ตามแต่ละ level ที่ถูก trigger ต่างจาก State Machine ใน Ch.11 ที่เป็น **sequential** (1 position ต่อครั้ง และต้องรอ EXIT ก่อนเปิดใหม่) Grid เหมาะกับพอร์ตที่มีทุนมากพอรองรับหลาย levels พร้อมกัน

ความเสี่ยงของ Grid Trading

ความเสี่ยงของ Grid คือการสะสม position ไปเรื่อยๆ ถ้า ε ไม่กลับ → ต้องตั้ง max_levels และ cointegration health check ทุก bar ถ้า ADF p-value > 0.1 หรือ regime = Broken → ปิด grid ทั้งหมดทันที

Running Example: Bybit ↔ Lighter — Grid บน OU Process

- $\theta = 0.001$ (0.1%), $\sigma_{\text{stat}} = 0.003$ (0.3%), grid step = 1σ
- Total friction ≈ 20 bps (ดู ch19) → gross per step = $0.3\% = 30$ bps > 20 bps ✓

- Grid levels: BUY ที่ $z < -1$ ($\epsilon < -0.002$), BUY+++ ที่ $z < -2$ ($\epsilon < -0.005$), SELL ที่ $z > 1$
- ใน 30 วัน: grid triggered 47 ครั้ง, avg hold 5.2h, avg gross/trade = 28 bps, net = 8 bps (หลัง friction)

โจทย์ 11.4 — Grid Step ขั้นต่ำ

ถ้า $\sigma_{\text{stat}} = 0.004$ และ total friction = 20 bps — กี่ sigma ต่อ step จึงจะทำได้? (ใช้ 1% notional per step)

► คำตอบ

11.7 แบบฝึกหัด

โจทย์ 11.1 — Transition ที่ถูกต้อง

ระบบอยู่ใน state MANUAL_GO กับ entry_zscore = +2.5 (Short Bybit, Long Lighter)

ปัจจุบัน z-score = +0.3 และ cointegration ยังคง OK

ถาม: ระบบควร transition ไป state ไหน? เพราะเหตุใด?

► คำตอบ

โจทย์ 11.2 — Emergency Invalidation

ระบบอยู่ใน state MANUAL_GO กำลัง hold position (Short Bybit, Long Lighter)

z-score = +1.8 (ยังไม่ถึง exit threshold) แต่กะทันหัน cointegration test fail (p-value = 0.15)

ถาม: ระบบควรทำอะไร? อธิบาย action และเหตุผล

► คำตอบ

โจทย์ 11.3 — ออกแบบ Anti-Thrashing Logic

ปัญหา: z-score วนอยู่ที่ 1.45–1.55 ทำให้ระบบ transition NO_TRADE → WATCH → NO_TRADE ซ้ำๆ ทุก 2 นาที (เรียกว่า "thrashing")

ถาม: ออกแบบ modification ใน StateMachine เพื่อแก้ปัญหานี้ โดยไม่เปลี่ยน threshold หลัก

► คำตอบ

Position Sizing: คำนวณขนาด Position อย่างถูกต้อง

Kelly criterion + half-life scaling + jump adjustment = position ที่ optimal

ตัวอย่างหลัก: Bybit BTC Perp ↔ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

Kelly criterion บอก **fraction ของทุนที่ควรเสี่ยง** เพื่อให้ long-run geometric growth สูงสุด แต่สำหรับ stat arb ต้องปรับด้วย half-life (ความเร็ว mean reversion) และ jump risk เพื่อให้ size เหมาะกับลักษณะ pair จริงๆ

$f^* = \text{edge} / \text{odds}$ — Kelly criterion

12.1 Kelly Criterion พื้นฐาน

Kelly criterion มาจาก maximization ของ $E[\log(\text{wealth})]$ สำหรับ gamble ที่มี edge:

สูตร KELLY สำหรับ CONTINUOUS RETURN

$$f^* = \mu / \sigma^2 = \text{edge} / \text{variance}$$

μ = expected return ต่อ period (edge)

σ^2 = variance ของ return ต่อ period

f^* = fraction ของ capital ที่ควรลงทุน (0 ถึง 1)

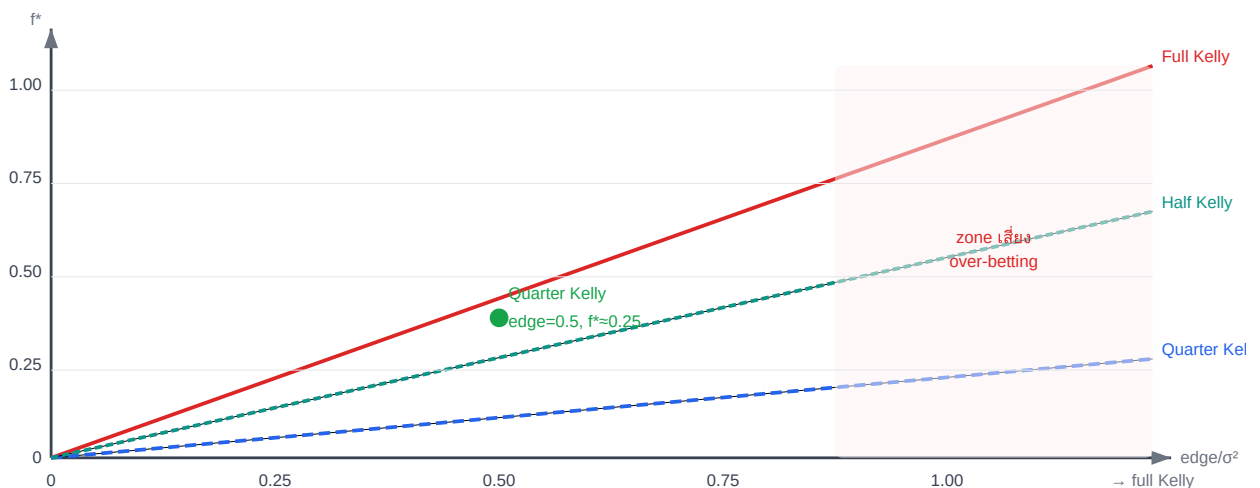
ในรูปแบบ bet แบบ binary (win/lose) สูตรคือ $f^* = (p \cdot b - q) / b = (p \cdot b - (1-p)) / b$

ตัวอย่างตัวเลข: Kelly fraction

สมมติ spread มี edge = 0.6% per trade และ $\sigma = 1.2\%$ per trade:

$f^* = \text{edge} / \sigma^2 = 0.006 / (0.012)^2 = 0.006 / 0.000144 = 41.67 \rightarrow$ แต่ $f^* > 1$ หมายถึง leverage สูงมาก $\rightarrow$ ในทางปฏิบัติใช้ fractional Kelly: $f_{\text{actual}} = f^* \times 0.25$ (quarter Kelly) $\rightarrow f_{\text{actual}} = 41.67 \times 0.25 \approx 10.4 \times$ leverage $\rightarrow$ ยังสูงอยู่ $\rightarrow$ ต้องปรับด้วย half-life และ jump discount ต่อไป

Kelly Fraction vs Edge/Variance — Parabola Curve



Full Kelly ให้ growth สูงสุดในทางทฤษฎี แต่ drawdown รุนแรงมาก — ในทางปฏิบัติใช้ Quarter Kelly

12.2 Half-life Scaling — ปรับขนาดตาม Mean Reversion Speed

ถ้า half-life ยาว (spread mean-revert ช้า) เราต้อง hold position นานกว่า ความเสี่ยงสะสมมากขึ้น — Kelly เดิมไม่ได้รวม factor นี้

HALF-LIFE SCALING FACTOR

$$\text{size_adjusted} = f^* \times \min(1, \text{target_halflife} / \text{actual_halflife})$$

target_halflife = half-life ที่เราออกแบบ strategy สำหรับ (เช่น 5 ชั่วโมง)

actual_halflife = half-life จริงที่คำนวณได้จากข้อมูล

ถ้า actual > target → scale down size (position เล็กลง เพราะ mean-revert ช้า)

actual_halflife	target_halflife	scaling factor	ผลกระทบ
3 ชั่วโมง	5 ชั่วโมง	$\min(1, 5/3) = 1.0$	ไม่ลด (fast mean reversion — ดี)
5 ชั่วโมง	5 ชั่วโมง	$\min(1, 5/5) = 1.0$	ไม่ลด (ตรงเป้า)
10 ชั่วโมง	5 ชั่วโมง	$\min(1, 5/10) = 0.5$	ลด size เหลือ 50%
24 ชั่วโมง	5 ชั่วโมง	$\min(1, 5/24) \approx 0.21$	ลด size เหลือ 21% (slow — ระวัง)

สังเกตสำคัญ: Half-life เปลี่ยนตามเวลา

Half-life ของ Bybit ↔ Lighter spread อาจสั้นกว่าช่วง sideways market (1-2h) และยาวขึ้นในช่วง trending market (8-12h) — ต้องคำนวณ half-life ใหม่ทุก N ชั่วโมง ไม่ใช่ fix ไว้ตลอด

12.3 Jump-adjusted Kelly

ในตลาด crypto มี jump risk สูง: ราคาอาจกระโดดข้ามคืนหรือช่วง liquidation cascade ค่าพยากรณ์ $E[\log(1+f \cdot R)]$ ลดลงอย่างมากเมื่อมี jump — Kelly สูตรปกติ assume normal returns ซึ่งไม่ถูกต้อง

JUMP-ADJUSTED KELLY

$$f^*_{\text{adjusted}} = f^* \times \text{jump_discount}$$

$$\text{jump_discount} = 1 - P(\text{jump}) \times (\text{expected_loss_given_jump} / \text{max_tolerable_loss})$$

ค่าปกติ: $\text{jump_discount} \in [0.6, 0.9]$ ขึ้นอยู่กับ volatility regime

ช่วง low vol: 0.85–0.90 | ช่วง high vol (เช่น ก่อน major news): 0.60–0.70

ทำไม Jump ถึงทำลาย Kelly?

สมมติ Kelly บอก $f^* = 0.5$ (ลง 50% ของทุน) แต่มี 1% chance ที่ spread จะกระโดด 5σ ในพริบตา:

ขาดทุน = $0.5 \times \text{capital} \times 5 \times \sigma_{\text{spread}}$ ถ้า $\sigma_{\text{spread}} = 2\%$ และ $\text{capital} = \$100,000$ ขาดทุน = $0.5 \times \$100,000 \times 5 \times 0.02 = \$5,000$ จากเหตุการณ์ 1 ครั้ง ถ้าวัด f^* ด้วย $\text{jump_discount} = 0.8$: $f^*_{\text{adj}} = 0.5 \times 0.8 = 0.4$ ขาดทุน = $0.4 \times \$100,000 \times 5 \times 0.02 = \$4,000$ (ดีขึ้น 20%)

12.4 Max Drawdown Constraint

นอกจาก Kelly และ half-life scaling แล้ว ต้องตั้ง hard limit จาก max drawdown ที่ยอมรับได้:

MAX DRAWDOWN POSITION LIMIT

$$\text{size_dd} = \text{max_loss} / (\text{entry_z} \times \sigma_{\text{spread}})$$

max_loss = capital $\times$ max_dd fraction (เช่น 2% ของ capital)

entry_z = z-score ณ จุดเข้า trade

σ_{spread} = volatility ของ spread

ขนาด position สุดท้าย = min(kelly_size, size_dd)

ตัวอย่าง: Max Drawdown เป็น Binding Constraint

Kelly บอก \$80,000 แต่ drawdown limit บอก \$45,000 $\rightarrow$ ใช้ \$45,000

สถานการณ์นี้เกิดขึ้นเมื่อ entry_z ต่ำ (เช่น $z = 2.0$ ต่ำกว่า $z = 3.5$) หรือ σ_{spread} สูง

Kelly ไม่ได้รู้จัก "ทุนก็บาท" — Drawdown limit ป้องกัน ruin ในกรณีเลวร้าย

12.5 Pseudo-code: compute_position_size()

```
def compute_position_size( capital, edge, sigma, halflife, target_halflife=5,
jump_discount=0.8, max_dd=0.02 ): # Step 1: Kelly fraction kelly_f = edge /
(sigma ** 2) # Step 2: Half-life scaling (ลดถ้า mean reversion ช้า) hl_scale =
min(1.0, target_halflife / halflife) # Step 3: Jump discount (ลดความเสี่ยง tail
event) jump_scale = jump_discount # Step 4: Max drawdown constraint (hard
limit) dd_limit = (capital * max_dd) / (2 * sigma) # Step 5: รวมทุก factor
raw_size = capital * kelly_f * hl_scale * jump_scale # Step 6: ใช้ขนาดที่น้อยกว่า
(conservative) return min(raw_size, dd_limit)
```

หมายเหตุเกี่ยวกับ sigma ใน dd_limit

ตัวหาร $2 * \sigma$ ใน dd_limit มาจากสมมติว่า worst case คือ spread เคลื่อนไป 2σ ในทิศทางตรงข้ามกับ position ถ้าต้องการ conservative มากขึ้น ใช้ 3σ หรือ 4σ ขึ้นอยู่กับ risk appetite

12.6 Running Example: Bybit ↔ Lighter BTC Perp

Running Example: คำนวณขนาด Position ที่ละขั้น

สถานการณ์: ระบบเพิ่ง transition ไป MANUAL_GO — ต้องคำนวณขนาด order ก่อนส่ง

Parameter	ค่า	ที่มา
capital	\$100,000	กำหนดโดย risk manager
edge (μ)	0.8% = 0.008	historical P&L per trade
sigma (σ)	1.5% = 0.015	std ของ spread returns (30d)
halflife	4 ชั่วโมง	OU model calibration
target_halflife	5 ชั่วโมง	ออกแบบ strategy สำหรับ 5h
jump_discount	0.80	low-vol regime
max_dd	2% = 0.02	risk limit

การคำนวณที่ละขั้น

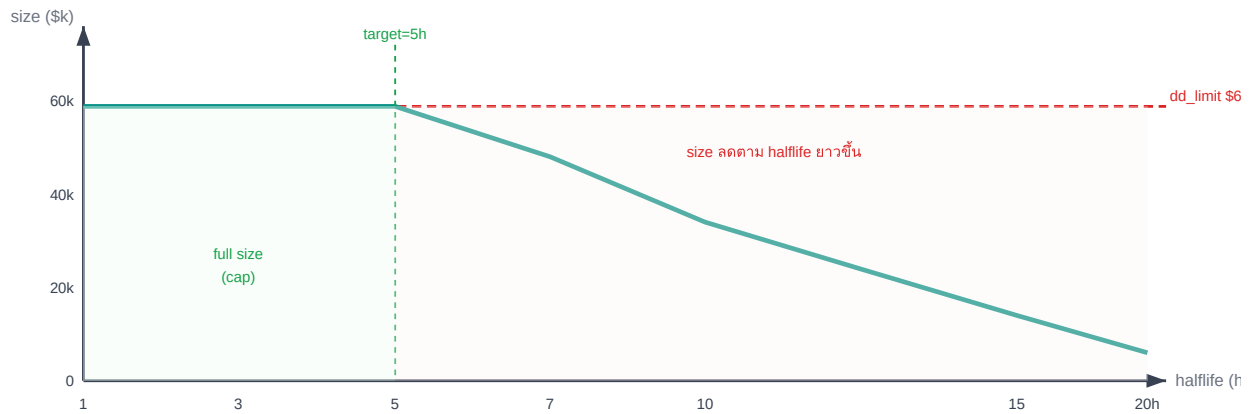
Step 1: Kelly fraction $kelly_f = edge / sigma^2 = 0.008 / (0.015)^2 = 0.008 / 0.000225 = 35.56$ ← leverage สูงมาก ต้องปรับ Step 2: Half-life scaling $hl_scale = \min(1.0, 5 / 4) = \min(1.0, 1.25) = 1.0$ ← actual (4h) เร็วกว่า target (5h) → ไม่ลด Step 3: Jump discount $jump_scale = 0.80$ Step 4: Max drawdown limit $dd_limit = (100,000 \times 0.02) / (2 \times 0.015) = 2,000 / 0.03 = \$66,667$ Step 5: Raw size $raw_size = 100,000 \times 35.56 \times 1.0 \times 0.80 = \$2,844,800$ ← สูงกว่า dd_limit มาก! Step 6: Final size = $\min(\$2,844,800, \$66,667) = \$66,667$

ผลลัพธ์และการตีความ

- Position size สุดท้าย: **\$66,667** (ถูก cap โดย drawdown limit)
- Kelly บอก \$2.8M แต่นั่นคือ theoretical optimum ภายใต้ assumptions ที่ unrealistic
- Drawdown limit เป็น binding constraint — ในทางปฏิบัติ max_dd มักเป็น primary constraint

- ทั้งสองขา: Long Lighter $\approx \$66,667$ | Short Bybit $\approx \$66,667 \times \beta$

Sensitivity Analysis: ผลของ Half-life ที่ต่างกัน



เมื่อ half-life น้อยกว่า target (5h) → size cap ที่ dd_limit | เมื่อยาวกว่า → size ลดลงตามสัดส่วน

12.7 แบบฝึกหัด

โจทย์ 12.1 — คำนวณ Kelly Size

Pair ใหม่มี edge = 1.2% per trade, σ = 2.0% per trade

capital = \$50,000, halflife = 8h, target_halflife = 5h

jump_discount = 0.75, max_dd = 3%

ถาม: คำนวณ position size ที่ละชั้น และระบุว่า constraint ไหนเป็น binding?

► ดูคำตอบ

โจทย์ 12.2 — เปรียบเทียบ Two Pairs

เปรียบเทียบสอง pair ด้วย capital = \$200,000 เดียวกัน:

- Pair A: edge=0.5%, σ =0.8%, halflife=3h, jump_discount=0.9, max_dd=2%
- Pair B: edge=1.5%, σ =3.0%, halflife=12h, jump_discount=0.65, max_dd=2%

ถาม: คำนวณ final size ของแต่ละ pair และอธิบายว่า pair ไหน "ดีกว่า" ในแง่ risk-adjusted

► ดูคำตอบ

โจทย์ 12.3 — Dynamic Sizing เมื่อ Regime เปลี่ยน

ระหว่าง hold position อยู่ ระบบตรวจพบ volatility regime เปลี่ยนจาก low-vol เป็น high-vol:

jump_discount ลดจาก 0.85 เป็น 0.60 และ σ_{spread} เพิ่มจาก 1.5% เป็น 2.5%

Current position size = \$80,000, capital = \$100,000, max_dd = 2%

ถาม: ควรทำอะไรกับ position ที่ถืออยู่?

► ดูคำตอบ

Perpetual Basis Strategy: Trade ความผิดปกติของ Funding

Basis = perp – spot index — เบี่ยงเกิน fair value เมื่อไหร่ให้เทรด

ตัวอย่างหลัก: Bybit BTC Perp ↔ Spot Index

แก่นของบทนี้ — อ่านก่อน

ราคา perpetual futures ควรอยู่ใกล้ spot index เพราะมีกลไก funding rate ดึงให้กลับ แต่ในช่วงที่ sentiment รุนแรง basis เบี่ยงเกิน fair value ชั่วคราว — นี่คือการ trade ที่มีขอบเขตชัดเจน

Basis_fair $\approx (r - q) \times T/365$ | entry เมื่อ $|Basis_actual - Basis_fair| > k \times \sigma_basis$

13.1 กลไก Funding Rate

Perpetual futures ไม่มี expiry — กลไกที่ทำให้ราคาใกล้ spot คือ **funding rate** ที่จ่ายระหว่าง long และ short ทุกรอบ:

กลไก Funding Rate ทำงานอย่างไร

- ทุก 8 ชั่วโมง (Bybit) หรือทุก 1 ชั่วโมง (Lighter) คำนวณ funding rate จาก basis และ interest rate
- ถ้า perp > spot (basis > 0): **longs จ่าย shorts** — ดึงให้ longs ปิด position → ราคา perp ลง
- ถ้า perp < spot (basis < 0): **shorts จ่าย longs** — ดึงให้ shorts ปิด position → ราคา perp ขึ้น
- Funding amount = position_size × funding_rate × (interval / 8h)

ตัวอย่าง: Bybit BTC Funding

Funding rate = +0.05% (longs จ่าย shorts) ทุก 8 ชั่วโมง = +0.15%/วัน = +4.5%/เดือน

Long BTC perp \$100,000 → จ่าย \$50 ทุก 8 ชั่วโมง ≈ \$150/วัน

นี่คือ "cost of carry" สำหรับ long perp ในช่วง bullish market

สูตร funding rate มาตรฐาน (Bybit-style):

```
funding_rate = clamp( premium_index + clamp(interest_rate - premium_index,
-0.05%, +0.05%), -0.75%, +0.75% )
premium_index = (P_perp - P_spot_index) /
P_spot_index ← basis เป็น %
interest_rate = 0.01% (Bybit กำหนด fixed = 0.01% ต่อ
รอบ 8h)
```

13.2 Basis = Perp – Spot: นิยามและ Fair Value

ในทางปฏิบัติ basis วัดเป็น absolute price หรือ percentage:

นิยาม BASIS

$$\text{Basis}_t = \text{P_perp}_t - \text{P_spot_index}_t$$

P_perp = mark price ของ perpetual futures

P_spot_index = weighted average ของราคา spot จาก multiple exchanges

Basis_fair = ค่าที่ basis "ควร" เป็น ตาม cost of carry model

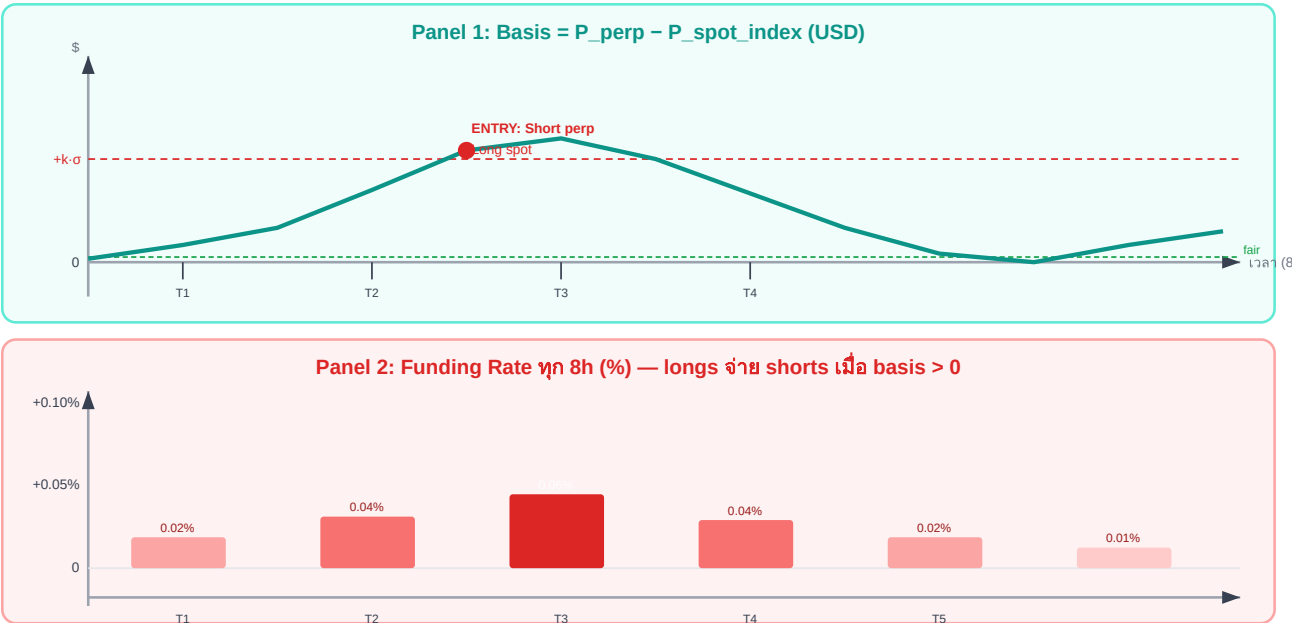
Fair value basis มาจาก cost of carry model:

$$\text{Basis_fair} = \text{P_spot} \times (r - q) \times T / 365$$

 r = risk-free rate (USD money market rate)
 q = dividend/yield ของ BTC (≈ 0 สำหรับ BTC)
 T = time to next funding (ใน fraction ของปี) ตัวอย่าง: $\text{P_spot} = \$65,000$, $r = 5.5\%$, $T = 4/8760$ (4 ชั่วโมง)
$$\text{Basis_fair} = 65,000 \times 0.055 \times (4/8760) = 65,000 \times 0.055 \times 0.000457 \approx \$1.63$$

Fair value basis มักใกล้เคียงศูนย์มากสำหรับ BTC perp เพราะ T ต่ำมาก (ไม่กี่ชั่วโมง) — basis เบี่ยงเบนจากการ sentiment หรือ imbalance ของ demand

13.3 SVG Diagram: Basis Timeline และ Funding Rate



Panel บน: basis สูงขึ้นจนเกิน threshold → Entry signal | Panel ล่าง: funding rate สูงสุดที่ T3 ตรงกับ basis สูงสุด

13.4 Entry/Exit Rules

กฎการเข้าและออก basis trade แบ่งตาม direction ของ basis deviation:

เงื่อนไข	Direction	Action	เหตุผล
$\text{basis} > \text{fair} + k \times \sigma_{\text{basis}}$	Basis บวกสูงเกินไป	Short perp + Long spot	Perp แพงเกินไป รอ basis กลับลง
$\text{basis} < \text{fair} - k \times \sigma_{\text{basis}}$	Basis ลบต่ำเกินไป	Long perp + Short spot	Perp ถูกเกินไป รอ basis กลับขึ้น
$\text{basis} \rightarrow \text{fair value}$	กลับปกติ	ปิดทั้งสองขา	Mean reversion สำเร็จ take profit
$\text{funding payment} \geq \text{target yield}$	ได้ yield แล้ว	ปิดทั้งสองขา	เก็บ funding เพียงพอแล้ว
$ \text{basis} > 3 \times \sigma_{\text{basis}}$ (blow-up)	เกิน stop-loss	ปิด emergency	Basis ไม่กลับ — regime เปลี่ยน

สองแหล่ง P&L ของ Basis Trade

- Basis convergence:** กำไรเมื่อ basis กลับมา fair value (capital gain จาก spread)
- Funding income:** ถ้า short perp ในช่วง basis > 0 → รับ funding payment ทุก 8h ขณะ hold position

Trade ที่ดีจะมีทั้งสองแหล่ง — basis กลับ + funding เข้ากระเป๋าระหว่างรอ

ความเสี่ยงของ Basis Trade

- Basis blow-up:** ในช่วง mania (เช่น bull run รุนแรง) basis อาจขยายไปเรื่อยๆ ก่อนกลับ
- Spot execution risk:** การ short spot อาจมีปัญหา borrow cost หรือ availability
- Funding flip:** funding rate อาจกลับทิศ ทำให้ long perp ต้องจ่ายแทน
- Index divergence:** spot_index อาจ diverge จาก exchange ที่ใช้ hedge จริง

13.5 Cost Breakdown Table

ก่อน trade ต้องรู้ต้นทุนทุกประเภท เพื่อคำนวณ net edge:

ประเภทต้นทุน	ค่าปกติ (Bybit BTC)	ผลต่อ P&L	หมายเหตุ
Taker fee (entry)	0.055% per leg	-0.11% รวมสองขา	ใช้ limit order ลดเหลือ -0.01% (maker)
Taker fee (exit)	0.055% per leg	-0.11% รวมสองขา	รวม round-trip: -0.22%
Bid-ask spread	~\$1-5 (BTC perp)	≈ -0.002-0.008%	ขึ้นกับ liquidity ณ เวลานั้น
Slippage	0.01-0.05%	-0.01 ถึง -0.05%	ขึ้นกับขนาด order vs market depth
Funding cost (long)	±0.01-0.10% ทุก 8h	+ หรือ - ขึ้นกับทิศ	ถ้าไรถ้า short perp ในช่วง basis+
Spot borrow (short spot)	0.01-0.05%/วัน	-ต่อวันที่ถือ	ค่าเช่า BTC สำหรับ short บน spot margin — Bybit เรียก ~0.01%/วัน; ถ้า hedge ผ่าน inverse perp ไม่มีต้นทุนนี้
รวม round-trip	≈ -0.25 ถึง -0.40% ต่อ trade (ไม่รวม funding income)		

Breakeven Analysis: Basis ต้องกว้างแค่ไหนถึงคุ้ม?

ถ้า total cost ≈ 0.35% per round-trip → basis ต้อง deviate จาก fair value อย่างน้อย 0.35% จึงจะ breakeven

ถ้า $\sigma_{\text{basis}} = 0.15\%$ → entry threshold $k = 0.35/0.15 \approx 2.3\sigma$

ในทางปฏิบัติใช้ $k \geq 2.5\sigma$ เพื่อ buffer สำหรับ slippage และ execution uncertainty

13.6 Pseudo-code: perp_basis_signal()

```
def perp_basis_signal(perp_price, spot_index, rate_per_8h, time_to_funding_h, #  
    ✓ FIX: ชัดเจนว่าหน่วยเป็น ชั่วโมง hist_sigma, k=2.0): # คำนวณ basis จริง basis =  
    perp_price - spot_index # คำนวณ fair value basis จาก cost of carry # ✓ FIX:  
    rate_per_8h * (เวลาที่เหลือ/8) - unit สอดคล้องกันทั้งหมด # ✗ เดิม: rate *  
    (time_to_funding / (8 * 3600)) - ปนหน่วย seconds กับ per-8h fair_basis =  
    spot_index * rate_per_8h * (time_to_funding_h / 8.0) # คำนวณ z-score ของ basis  
    deviation zscore = (basis - fair_basis) / hist_sigma # Signal logic if zscore >  
    k: return "SHORT_PERP_LONG_SPOT", zscore # basis สูงเกิน elif zscore < -k: return  
    "LONG_PERP_SHORT_SPOT", zscore # basis ต่ำเกิน return "NO_SIGNAL", zscore
```

การคำนวณ hist_sigma ของ Basis

hist_sigma ควรคำนวณจาก rolling std ของ (basis - fair_basis) ในช่วง 7-30 วัน

ไม่ควรใช้ σ คงที่ เพราะ basis volatility เปลี่ยนตาม market regime

ในช่วง sideways: σ ต่ำ → threshold ต่ำ → เข้าง่ายขึ้น

ในช่วง trending: σ สูง → threshold สูง → เข้ายากขึ้น (ป้องกัน false signal)

การเพิ่ม Funding Yield Forecast ใน Signal

```
def basis_trade_expected_pnl(basis_entry, fair_basis, hist_sigma, funding_rate,  
    hold_hours, position_size, spot_price): # P&L จาก basis convergence (mean  
    reversion) basis_move = basis_entry - fair_basis # คาดว่า basis จะกลับมา fair  
    basis_pnl = basis_move * position_size / spot_price # P&L จาก funding (short  
    perp receives funding) n_payments = hold_hours / 8 # จำนวนรอบ funding (ทุก 8h)  
    funding_pnl = funding_rate * position_size * n_payments # ต้นทุน round-trip (fee  
    + slippage) total_cost = position_size * 0.0035 # ≈ 0.35% return basis_pnl +  
    funding_pnl - total_cost
```

13.7 Running Example: Bybit BTC Perp Basis Trade

Running Example: Bybit BTC Perp Basis Trade — ตัวเลขที่ละขั้น

สถานการณ์: วันที่ BTC ราคา pump แรง longs เปิดมาก basis บวกสูงผิดปกติ

ข้อมูล Input	ค่า	ที่มา
P_perp (Bybit mark price)	\$67,450	Bybit API
P_spot_index	\$67,100	Bybit spot index (avg หลาย exchanges)
Basis_actual	\$350 (+0.521%)	67,450 – 67,100
rate_per_8h (Bybit funding)	0.01% = 0.0001	Bybit default funding rate per 8h interval
time_to_funding_h	3.5 ชั่วโมง	เวลาถึง funding ถัดไป (หน่วย: ชั่วโมง)
Basis_fair	$67,100 \times 0.0001 \times (3.5/8.0) \approx \2.94	spot $\times$ rate_per_8h $\times$ (hours_left/8)
hist_sigma (30d rolling)	\$85 (0.127%)	std ของ (basis – fair_basis)
z-score	$(350 - 2.94) / 85 \approx +4.1\sigma$	ENTRY SIGNAL!
funding_rate (current)	+0.065%	Bybit funding history

Execution Plan และ P&L

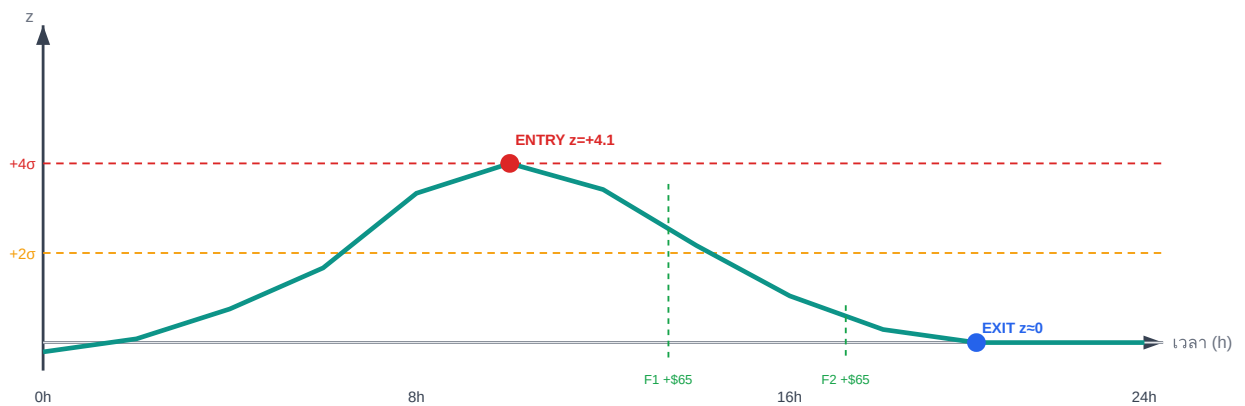
Entry ($z = +4.1\sigma$, basis = +\$350): Short Bybit BTC perp : \$100,000 notional at \$67,450 Long spot BTC (หรือ spot index hedge): \$100,000 at \$67,100 Hold 16 ชั่วโมง (2 รอบ funding): Funding received (short perp, longs pay us): $= 0.065\% \times \$100,000 \times 2$ รอบ = \$130 Exit (basis กลับ fair: basis $\approx$ \$20): ปิด Short perp ที่ \$67,120 $\rightarrow$ กำไร perp = $\$67,450 - \$67,120 = \$330$ per BTC ปิด Long spot ที่ \$67,100 (สมมติ spot ไม่เปลี่ยน) $\rightarrow$ กำไร spot = \$0 Basis P&L $\approx (\$350 - \$20) \times (100,000 / 67,450) \approx \489 ต้นทุน: Taker fee (entry + exit, 2 legs each) = $0.22\% \times \$100,000 = \220 Slippage estimate = $0.03\% \times \$100,000 = \30 Net P&L: = \$489 (basis convergence) + \$130 (funding income) – \$220 (taker fees) – \$30 (slippage) =

+\$369 → +0.37% บน \$100,000 ใน 16 ชั่วโมง ≈ Annualized ~20% (ถ้า trade frequency ≈ 1 trade/วัน)

สรุป: โครงสร้าง P&L ของ Basis Trade นี้

- **Basis convergence:** \$489 — แหล่งกำไรหลัก (basis เบี่ยง 4σ กลับมา fair)
- **Funding income:** \$130 — bonus ระหว่างรอ mean reversion
- **ต้นทุนรวม:** \$250 — fee + slippage
- **Net: +\$369** ใน 16 ชั่วโมง — measurable edge ที่ repeatable

Z-score Timeline ของ Trade นี้



Basis z-score ขึ้นถึง +4.1σ → เข้า trade → กลับมา 0 ใน 16 ชั่วโมง รับ 2 รอบ funding ระหว่างทาง

13.8 แบบฝึกหัด

โจทย์ 13.1 — คำนวณ Fair Basis และ Z-score

ข้อมูล: $P_{\text{perp}} = \$42,300$, $P_{\text{spot_index}} = \$42,100$

$\text{rate_per_8h} = 0.01\%$ (Bybit default), $\text{time_to_funding_h} = 5.5$ ชั่วโมง

hist_sigma ของ $(\text{basis} - \text{fair_basis}) = \60 , $k = 2.0$

ถาม: คำนวณ fair_basis , z-score และระบุว่า มี signal หรือไม่ถ้า $k = 2.0$

► ดูคำตอบ

โจทย์ 13.2 — คำนวณ Net P&L ของ Basis Trade

Trade: Short perp $\$80,000$ + Long spot $\$80,000$

Entry basis = $+\$280$, Exit basis = $+\$15$

Hold 24 ชั่วโมง (3 รอบ funding), $\text{funding_rate} = +0.05\%$ ต่อรอบ

Taker fee = 0.055% per leg, 4 legs total, slippage = 0.025%

สมมติ $\text{spot_price entry} \approx \$42,100$

ถาม: คำนวณ net P&L ที่ละส่วน

► ดูคำตอบ

โจทย์ 13.3 — Basis Blow-up: เมื่อ Trade ขาดทุน

คุณ Short perp ที่ basis = $+\$300$ ($z = +3.0\sigma$, $\text{hist_sigma} = \$100$)

แต่แทนที่ basis จะกลับ basis กลับขยายไปเรื่อยๆ

Stop-loss กำหนดไว้ที่ $z = +5.0\sigma$, position size = $\$50,000$, $\text{spot_price} \approx \$67,000$

ถาม: (1) Stop-loss trigger ที่ basis เท่าไร? (2) ขาดทุนเท่าไรเมื่อ stop-loss ถูก trigger?

► ดูคำตอบ

Cross-venue Strategy: ซื้อขายระหว่างสองตลาด พร้อมกัน

ควบคุม leg risk, synchronization, และ execution timing ข้ามตลาด

ตัวอย่างหลัก: Bybit BTC Perp ↔ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

กำไรมาจาก **spread** ไม่ใช่ direction — ต้องเปิดสองขาพร้อมกันเพื่อ lock spread
ถ้าเปิดได้ขาเดียว คุณไม่ได้ trade spread แต่กำลัง directional trade โดยไม่ตั้งใจ

$$\text{กำไร} = (\varepsilon_{\text{entry}} - \varepsilon_{\text{exit}}) - \text{fees_A} - \text{fees_B}$$

14.1 Latency Arbitrage vs Statistical Arbitrage

ทั้งสองกลยุทธ์ดูคล้ายกัน แต่มีความต่างพื้นฐาน ที่กำหนดทั้ง infrastructure และ edge ที่ต้องการ

มิติ	Latency Arbitrage (LA)	Statistical Arbitrage (SA)
Edge มาจาก	เร็วกว่าคนอื่น — race to fill	OU process ที่ mean-revert
Infrastructure	Colocation, FPGA, microsecond	Standard server, millisecond ก็พอ
Signal	Price discrepancy ชั่วคราว <100ms	z-score ของ spread หลาย minutes/hours
Competition	สูงมาก — HFT firms	ปานกลาง — hedge funds, prop desks
กำไรต่อ trade	เล็ก แต่บ่อยมาก	ใหญ่กว่า แต่รอนาน
Leg risk	สูงมาก — ต้องเร็วทั้งสองขา	ปานกลาง — มีเวลาจัดการ

ใช้ SA ในเล่มนี้เพราะอะไร?

Bybit ↔ Lighter BTC perp มี spread ที่ mean-revert ด้วย half-life หลายชั่วโมง ไม่ต้องการความเร็วระดับ microsecond แต่ต้องการ execution ที่ **synchronous** พอ เพื่อไม่ให้ leg risk กินกำไร

14.2 Leg Risk — ความเสี่ยงที่อันตรายที่สุดใน Cross-venue

Leg risk เกิดเมื่อเปิด Leg A สำเร็จ แต่ Leg B ล้มเหลว ทำให้เกิด *single-leg exposure* — ขณะนี้คุณไม่ได้ trade spread แต่กำลัง directional long/short โดยไม่ตั้งใจ และไม่มี hedge

สถานการณ์อันตราย — Leg B Fail

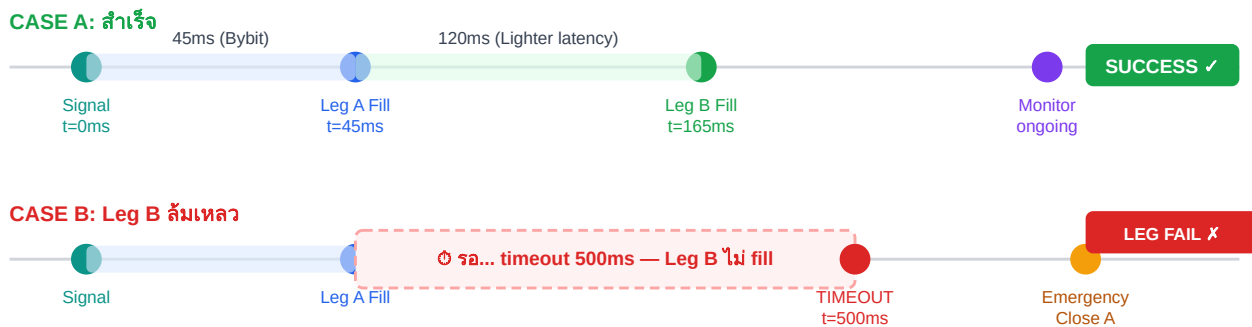
สัญญาณ: ϵ สูงเกิน $\rightarrow$ Long Bybit + Short Lighter

สำเร็จ: Long Bybit \$100,000 (Leg A) ✓

ล้มเหลว: Short Lighter ไม่ fill ใน 500ms (Leg B) ✗

ผล: คุณ Long BTC \$100,000 โดยไม่มี hedge — ถ้า BTC ร่วง 2% = ขาดทุน \$2,000 ทันที

Timeline ของ Leg Execution



ช่วง "Leg A filled แต่ Leg B ยังไม่ fill" คือ single-leg exposure — อันตรายที่สุด

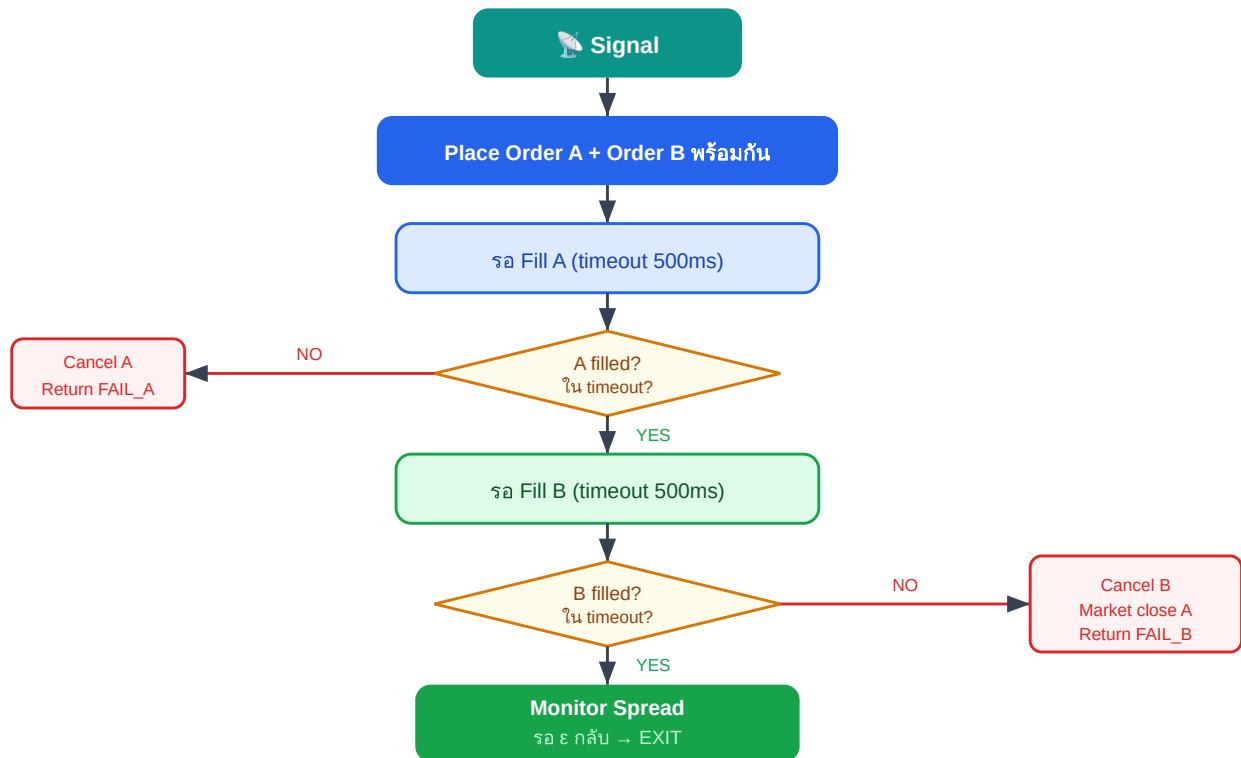
ช่วงสี่ชมพูระหว่าง Leg A fill ถึง timeout คือ window ที่ directional risk เต็มๆ

วิธีลด Leg Risk

- ใช้ **simultaneous order placement** — ส่งคำสั่งทั้งสองตลาดเกือบพร้อมกัน
- ตั้ง **timeout สั้น** (300–500ms) — ถ้า B ไม่ fill ให้ปิด A ทันที
- ใช้ **limit order ที่ aggressive** — ใกล้เคียง mid price เพื่อให้ fill เร็ว
- มี **emergency close routine** พร้อมใช้ตลอดเวลา

14.3 Execution Flow Diagram

กระบวนการ execution ที่สมบูรณ์ต้องจัดการ happy path และ failure path ทุก branch



Flowchart execution: ทุก branch ต้องมี handler — ไม่มี "undefined state"

14.4 Synchronization Strategy — สามแนวทาง

วิธีที่จะส่ง order ทั้งสองขามีผลต่อ leg risk, latency overhead และ complexity ของ code

แนวทาง	วิธีการ	Leg Risk	Latency	Complexity
Simultaneous	ส่งทั้ง A และ B ในเวลาเดียวกัน (async)	ต่ำ — overlap เล็กน้อย	ต่ำ	กลาง
Sequential	ส่ง A ก่อน รอ fill แล้วจึงส่ง B	สูง — รอ fill A = exposed	สูง (A_latency + B_latency)	ต่ำ
Conditional	ส่ง A ก่อน ถ้า fill ส่ง B ทันที ถ้าไม่ cancel	กลาง — มี logic ความคุม	กลาง	สูง

Bybit ↔ Lighter แนะนำ: Simultaneous + Timeout

ส่ง order ทั้งสองตลาดพร้อมกันใน async threads จากนั้น wait ด้วย timeout
ถ้าทั้งคู่ fill ภายใน 500ms → SUCCESS
ถ้า A fill แต่ B ไม่ fill → cancel B แล้ว market close A ทันที

14.5 Order Types ที่ใช้ใน Stat Arb

การเลือก order type มีผลต่อทั้ง fill probability และ slippage cost

Limit Order — แนะนำสำหรับ Stat Arb

- **ข้อดี:** ควบคุมราคาได้ — ไม่จ่ายเกิน entry price ที่คำนวณ
- **ข้อดี:** Maker fee ถูกกว่า taker fee (บางตลาดได้ rebate)
- **ข้อเสีย:** อาจไม่ fill ถ้าราคาวิ่งออกไป → partial fill risk
- **แก้ไข:** ใช้ limit ที่ aggressive (1 tick ใน spread) เพื่อ fill เร็ว

Market Order — ใช้เฉพาะกรณีฉุกเฉิน

- **ข้อดี:** Guarantee fill — ปิด position ได้แน่นอน
- **ข้อเสีย:** Slippage สูง โดยเฉพาะใน thin market เช่น Lighter
- **เมื่อใช้:** Emergency close เมื่อ Leg B ล้มเหลว เท่านั้น

14.6 Pseudo-code: cross_venue_execute()

```
def cross_venue_execute(signal, size, max_leg_wait_ms=500): if signal ==  
"LONG_A_SHORT_B": order_a = place_limit_order(venue="Bybit", side="BUY",  
size=size) order_b = place_limit_order(venue="Lighter", side="SELL", size=size)  
filled_a = wait_fill(order_a, timeout_ms=max_leg_wait_ms) if not filled_a:  
cancel(order_a) return "LEG_FAIL_A" filled_b = wait_fill(order_b,  
timeout_ms=max_leg_wait_ms) if not filled_b: cancel(order_b) # emergency: close  
leg A immediately with market order market_close(venue="Bybit", side="SELL",  
size=filled_a.qty) return "LEG_FAIL_B_CLOSED_A" return "SUCCESS", filled_a,  
filled_b
```

อธิบาย Logic แต่ละส่วน

- place_limit_order ทั้งสองบรรทัดควร call พร้อมกัน (async/thread) ไม่ใช่ sequential
- wait_fill(order_a, timeout_ms=500) — block ด้วย timeout 500ms
- ถ้า A ไม่ fill: cancel แล้วออก — ไม่มี exposure เลย
- ถ้า A fill แต่ B ไม่ fill: **ต้อง market close A ทันที** อย่ารอให้ราคาขยับ
- Return tuple (SUCCESS, filled_a, filled_b) สำหรับ logging และ monitor

สิ่งที่ต้อง Log ทุก Execution

timestamp, signal_value, venue_a_fill_price, venue_b_fill_price, venue_a_latency_ms,
venue_b_latency_ms, result, slippage_a, slippage_b

Log นี้ใช้ analyze เพื่อ tune timeout และตรวจจับ venue degradation

14.7 Running Example — Bybit ↔ Lighter Execution

Timing

Running Example: Bybit BTC Perp ↔ Lighter BTC Perp

สถานการณ์: z-score ของ spread = 2.3 → สัญญาณ Long Bybit, Short Lighter

- Bybit order placement → confirmation latency: **45ms**
- Lighter order placement → confirmation latency: **120ms**
- รวม sequence ที่ดีที่สุด (simultaneous): **$\max(45, 120) = 120\text{ms}$**
- รวม sequence ถ้า sequential: **$45 + 120 = 165\text{ms}$**

Timeline (simultaneous): t=0ms : ส่ง order Bybit + order Lighter พร้อมกัน
t=45ms : Bybit fill ✓ @ \$65,023.5 t=120ms : Lighter fill ✓ @ \$64,998.2
t=120ms : BOTH FILLED — spread locked = \$25.3 ($\epsilon = \log(65023.5) - 1.003 * \log(64998.2) = 0.0389\%$) ถ้า target exit $\epsilon = 0.00\%$ → expected profit = $0.0389\% \times \$100,000 = \38.9 ก่อน fee

Slippage จริง vs คาด: ถ้า signal คำนวณที่ mid price แต่ fill ที่ ask/bid → slippage ~0.5 tick ต่อข้าง
ต้องคิดในสมการ expected profit

14.8 แบบฝึกหัด

โจทย์ 14.1 — คำนวณ Leg Risk Window

Bybit latency = 60ms, Lighter latency = 150ms ใช้ sequential execution (ส่ง Bybit ก่อน แล้วส่ง Lighter หลัง Bybit fill)

ถาม: (ก) Leg risk window กว้างแค่ไหน? (ข) ถ้าใช้ simultaneous execution แทน risk window เหลือเท่าไร?

► ดูคำตอบ

โจทย์ 14.2 — Leg B ล้มเหลว สูญเสียเท่าไร?

Long Bybit \$200,000 fill ที่ \$65,000 แต่ Short Lighter ไม่ fill

Emergency close: market sell Bybit ที่ \$64,700 (BTC ร่วง 0.46% ระหว่างรอ)

Bybit taker fee = 0.05%

ถาม: ขาดทุนรวม (รวม fee) จากเหตุการณ์นี้เท่าไร?

► ดูคำตอบ

โจทย์ 14.3 — เลือก Timeout ที่เหมาะสม

จากข้อมูล historical: Lighter fill time distribution = median 80ms, P95 = 320ms, P99 = 680ms

ถ้าตั้ง timeout = 300ms จะ reject กี่% ของ trades? ถ้าตั้ง timeout = 600ms เพิ่มขึ้นอีก?

ถาม: ควรตั้ง timeout เท่าไรและทำไม?

► ดูคำตอบ

Dynamic Hedging ด้วย Kalman Filter: β ที่ เปลี่ยนตามเวลา

β_t ไม่คงที่ — Kalman Filter track β แบบ real-time โดยไม่ต้อง refit

ตัวอย่างหลัก: Bybit BTC Perp $\leftrightarrow$ Lighter BTC Perp

แก่นของบทนี้ — อ่านก่อน

β ระหว่าง Bybit และ Lighter ไม่คงที่ตลอด — เปลี่ยนตามสภาพตลาด Kalman Filter ให้ estimate β_t แบบ online โดยไม่ต้อง refit ทั้งหมด ทำให้ hedge ratio ตามทัน regime เปลี่ยน

State: $\beta_t = \beta_{t-1} + w_t$ | **Observation:** $r_{A,t} = \beta_t \cdot r_{B,t} + v_t$

15.1 ปัญหาของ Static β

บทก่อนๆ ใช้ β ที่ estimate จาก OLS บน window คงที่ เช่น 30 วัน แนวทางนี้มีปัญหาหลายประการ

ปัญหาของ Rolling OLS

- **Lag problem:** β วันนี้คือค่าเฉลี่ยของ 30 วันที่ผ่านมา ไม่ใช่ค่า "ตอนนี้"
- **Window selection:** window 7 วัน responsive แต่ noisy; window 60 วัน stable แต่ช้า ไม่มี window ที่ถูกเสมอ
- **Regime change:** เมื่อ volatility หรือ liquidity เปลี่ยนกะทันหัน β OLS ปรับไม่ทัน
- **Computational cost:** Refit OLS ทุก tick = matrix inversion ช้าๆ

Kalman Filter แก้ปัญหาอย่างไร?

- **Online update:** ปรับ β_t ทุก timestep ด้วยสูตรเดียว — ไม่ต้อง refit ทั้งหมด
- **ไม่มี window:** ใช้ Q และ R แทน window — ทั้งคู่มีความหมายทางสถิติชัดเจน
- **Uncertainty tracking:** P_t บอก confidence ใน estimate ปัจจุบัน
- **Fast computation:** $O(1)$ per update — เหมาะกับ real-time มาก

15.2 State Space Model — สองสมการหลัก

Kalman Filter ทำงานบน State Space Model ซึ่งแบ่งเป็นสองส่วน

สมการ STATE SPACE

Process equation: $\beta_t = \beta_{t-1} + w_t, w_t \sim N(0, Q)$

Observation equation: $r_{\{A,t\}} = \beta_t \cdot r_{\{B,t\}} + v_t, v_t \sim N(0, R)$

β_t = state ที่ซ่อนอยู่ (true hedge ratio ณ เวลา t) — ไม่สังเกตโดยตรง

$r_{\{A,t\}}, r_{\{B,t\}}$ = return ของ Bybit และ Lighter — สังเกตได้

w_t = process noise — β เปลี่ยนแค่ไหนต่อ step (Q ใหญ่ = เปลี่ยนเร็ว)

v_t = observation noise — return มี noise เท่าไร (R ใหญ่ = noisy มาก)

พารามิเตอร์	ความหมาย	ค่าที่ใช้ทั่วไป	ตัวอย่างตัวเลข	ผลเมื่อเพิ่ม
Q (process noise)	β เปลี่ยนเร็วแค่ไหนต่อ step	1e-5 ถึง 1e-3	Q=1e-3 $\rightarrow \beta$ drift $\approx \sqrt{(1e-3)} \approx 0.032$ ต่อ bar Q=1e-5 $\rightarrow \beta$ drift ≈ 0.003 ต่อ bar (10× ช้ากว่า)	β ตอบสนองเร็วขึ้น แต่ noisy
R (observation noise)	Return มี noise เท่าไร	1e-3 ถึง 1e-1	R=0.01 $\rightarrow$ return std $\approx 1\%$; R=0.001 $\rightarrow \approx 0.3\%$ สำหรับ BTC perp hourly: R $\approx 0.0001-0.001$	Kalman gain ลด $\rightarrow$ ปรับช้าลง
P_0 (initial uncertainty)	ความไม่แน่ใจใน β ตอนเริ่ม	1.0	P_0=1.0 $\rightarrow$ warm-up ~ 50 bars; P_0=0.01 $\rightarrow$ เริ่มเชื่อดีค่า β เดิมมาก	warm-up period สั้นลง

15.3 Kalman Filter Steps — Predict และ Update

ทุก timestep มีสองขั้นตอน: **Predict** (คาดการณ์ล่วงหน้า) และ **Update** (ปรับด้วยข้อมูลใหม่)

ขั้นตอน Predict

$\beta_{\{t|t-1\}} = \beta_{\{t-1|t-1\}}$ (คาด β จาก prior) $P_{\{t|t-1\}} = P_{\{t-1|t-1\}} + Q$
(uncertainty เพิ่มขึ้น เพราะเวลาผ่าน)

เพราะ process equation บอกว่า β drift ด้วย $w_t \sim N(0, Q) \rightarrow$ uncertainty เพิ่มทุก step

ขั้นตอน Update (เมื่อเห็น $r_{A,t}$ และ $r_{B,t}$)

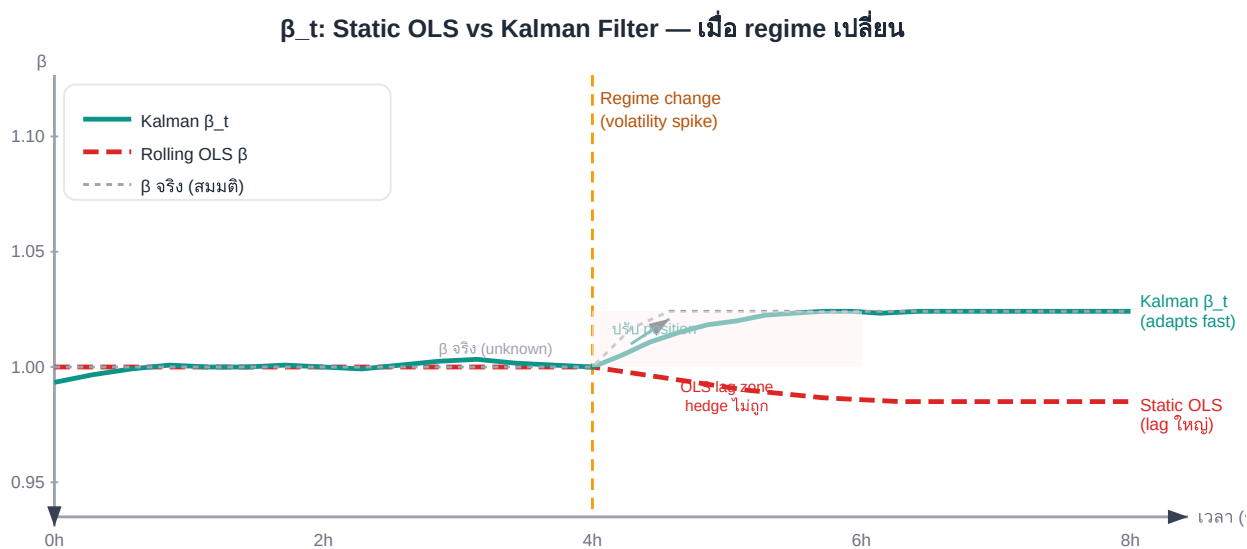
innovation = $r_{\{A,t\}} - \beta_{\{t|t-1\}} \cdot r_{\{B,t\}}$ (ความต่างจากที่คาด) $S = r_{\{B,t\}}^2 \cdot P_{\{t|t-1\}} + R$ (variance ของ innovation) $K_t = P_{\{t|t-1\}} \cdot r_{\{B,t\}} / S$
(Kalman Gain) $\beta_{\{t|t\}} = \beta_{\{t|t-1\}} + K_t \cdot \text{innovation}$ (update β) $P_{\{t|t\}} = (1 - K_t \cdot r_{\{B,t\}}) \cdot P_{\{t|t-1\}}$ (update uncertainty)

ความหมายของ Kalman Gain K_t

K_t อยู่ระหว่าง 0 ถึง 1 และบอกว่า "เชื่อข้อมูลใหม่มากแค่ไหน"

- K_t ใกล้ 1: เชื่อข้อมูลใหม่มาก (Q ใหญ่หรือ R เล็ก) — β ปรับเร็ว
- K_t ใกล้ 0: เชื่อ prior มาก (Q เล็กหรือ R ใหญ่) — β ปรับช้า
- ระยะแรก (P สูง): K สูง — warm up เร็ว; เมื่อ P ลด: K ลด — stable estimate

15.4 SVG Diagram — β_t Kalman vs Static OLS



เมื่อ regime เปลี่ยน Kalman ปรับ β_t เร็วกว่า OLS มาก ลด hedge error ใน zone สีชมพู

15.5 Pseudo-code: kalman_beta_update()

```
def kalman_beta_update(beta, P, r_A, r_B, Q=1e-5, R=1e-3): # --- Predict step -
-- beta_pred = beta #  $\beta$  ไม่เปลี่ยน (random walk assumption) P_pred = P + Q #
uncertainty เพิ่มขึ้น # --- Update step --- innovation = r_A - beta_pred * r_B #
ความต่างจากที่คาด S = r_B**2 * P_pred + R # variance ของ innovation K = P_pred *
r_B / S # Kalman Gain beta_new = beta_pred + K * innovation # update  $\beta$  P_new =
(1 - K * r_B) * P_pred # update uncertainty return beta_new, P_new
```

วิธีใช้ใน loop จริง

```
# Initialization beta = 1.0 # เริ่มจาก 1:1 (prior) P = 1.0 # high uncertainty ช่วง
แรก # Real-time update for each new price tick: r_A = log(price_A_now /
price_A_prev) r_B = log(price_B_now / price_B_prev) beta, P =
kalman_beta_update(beta, P, r_A, r_B, Q=1e-5, R=1e-3) # Recompute spread with
new beta epsilon = log(price_A) - beta * log(price_B) z_score = (epsilon -
epsilon_mean) / epsilon_std
```

⚠ Implementation Trap: epsilon_mean และ epsilon_std ต้องอัปเดตตาม β

ทำไม θ และ σ_{stat} เดิมใช้ไม่ได้? เพราะ $\epsilon = \log(P_A) - \beta \cdot \log(P_B)$ — ถ้า β เปลี่ยน ค่า ϵ ทุกจุดในอดีตก็เปลี่ยนด้วย รวากับว่าเรา recompute ϵ series ทั้งหมดใหม่ด้วย β ใหม่ ดังนั้น mean และ std ที่ calibrate ไว้บน ϵ เดิมจะ mismatch กับ ϵ ชุดใหม่ทันที

ตัวอย่าง: β เปลี่ยนจาก 1.000 $\rightarrow$ 1.005 เมื่อ $\log(P_B) \approx 10 \rightarrow \epsilon$ เปลี่ยนทุกจุดไป $-0.05 \rightarrow \theta$ เดิมที่ 0.001 กลายเป็น -0.049 ทันที ถ้า z-score ยังใช้ θ เก่า จะ off ไปถาวร

วิธีแก้ที่แนะนำ: ใช้ **EMA ที่อัปเดต real-time ทุก bar** แทนค่าคงที่จาก calibration:

```
# ✅ Concrete solution: Welford-style EMA mean+variance # Initialization จาก
historical calibration alpha_ema = 0.02 # EMA decay (effective window  $\approx 1/\alpha = 50$  bars)
epsilon_mean = epsilon_0 # ค่าเริ่มต้นจาก historical mean ของ  $\epsilon$ 
epsilon_var = epsilon_var0 # variance เริ่มต้นจาก historical var # Real-time
update loop for each new price tick: r_A = log(price_A_now / price_A_prev) r_B
= log(price_B_now / price_B_prev) # 1. Update  $\beta$  via Kalman beta, P =
kalman_beta_update(beta, P, r_A, r_B, Q=1e-5, R=1e-3) # 2. Recompute epsilon
ด้วย beta ใหม่ epsilon = log(price_A) - beta * log(price_B) # 3. ✅ FIX: คำนวณ
diff ก่อน update mean (Welford-style) # ป้องกัน variance bias จากการใช้ updated
mean ใน variance update diff = epsilon - epsilon_mean # ใช้ OLD mean # 4. Update
mean epsilon_mean = epsilon_mean + alpha_ema * diff # 5. Update variance ด้วย
```

```
OLD diff (EMA variance:  $w_{\text{new}} \cdot \text{diff}^2 + w_{\text{old}} \cdot \text{var}_{\text{old}}$ )  $\text{epsilon\_var} = (1 - \alpha_{\text{ema}}) * \text{epsilon\_var} + \alpha_{\text{ema}} * \text{diff}^2$   $\text{epsilon\_std} = \sqrt{\text{epsilon\_var} + 1e-10}$  # guard zero division # 6. z-score  $\hat{\mu}$  unbiased  $z\_score = (\text{epsilon} - \text{epsilon\_mean}) / \text{epsilon\_std}$ 
```

15.6 Choosing Q and R — Tuning พารามิเตอร์

การเลือก Q และ R เป็น trade-off ระหว่าง responsiveness กับ stability

สถานการณ์	Q ที่แนะนำ	R ที่แนะนำ	ผลที่ได้
β เปลี่ยนเร็ว (volatile market)	ใหญ่ (1e-4)	ปกติ (1e-3)	β ตอบสนองเร็ว แต่ noisy
β เปลี่ยนช้า (stable pair)	เล็ก (1e-6)	ปกติ (1e-3)	β smooth แต่ lag เมื่อ regime เปลี่ยน
Data noisy มาก (thin market)	ปกติ (1e-5)	ใหญ่ (1e-2)	ไม่ over-react ต่อ noise
Data สะอาด (deep market)	ปกติ (1e-5)	เล็ก (1e-4)	update เร็วขึ้นจาก each observation

วิธีหา Q และ R จากข้อมูล

Empirical Calibration

- R:** ประมาณจาก variance ของ residual ε บน historical data
$$R \approx \text{Var}(r_A - \beta_{\text{static}} \cdot r_B)$$
- Q:** ประมาณจาก variance ของการเปลี่ยนแปลง β ระหว่าง rolling windows
$$Q \approx \text{Var}(\beta_{\{t\}} - \beta_{\{t-1\}})$$
 จาก rolling OLS ที่ window เล็ก
- Cross-validate:** test บน out-of-sample เพื่อหา (Q,R) ที่ให้ spread mean-revert ที่สุด

15.7 Position Adjustment เมื่อ β เปลี่ยน

เมื่อ Kalman ให้ $\beta_{\text{new}} \neq \beta_{\text{old}}$ คุณต้องปรับ position เพื่อรักษา hedge ratio ที่ถูกต้อง

สูตรปรับ POSITION

$$\text{delta_position_B} = (\beta_{\text{new}} - \beta_{\text{old}}) \times \text{size_A}$$

$\text{delta_position_B} > 0$: β เพิ่ม → ต้อง short B เพิ่มขึ้น (หรือ long B น้อยลง)

$\text{delta_position_B} < 0$: β ลด → ต้อง short B น้อยลง (หรือ long B เพิ่ม)

ทำ adjustment เฉพาะถ้า $|\text{delta}| > \text{threshold}$ เพื่อลด transaction cost

Threshold สำหรับ Rebalancing

อย่าปรับทุก tick เพราะ transaction cost กินกำไร ตั้ง threshold เช่น:

```
if abs(beta_new - beta_old) > 0.01: #  $\beta$  เปลี่ยน > 1%
    delta = (beta_new - beta_old) * size_A
    adjust_position(venue="Lighter", delta=delta)
    beta_old = beta_new
```

0.01 เป็น rule of thumb — calibrate จาก fee structure จริง

15.8 Running Example — Bybit ↔ Lighter β Drift

Running Example: β เปลี่ยนจาก 0.98 → 1.05 ใน 2 ชั่วโมง

สถานการณ์: BTC มี funding rate surge บน Lighter ทำให้ราคา Lighter ขึ้นเร็วกว่า Bybit ชั่วคราว

เริ่มต้น: $\beta = 0.98$, $P = 0.001$ $t=0h$: $r_A=+0.12\%$, $r_B=+0.10\%$ → innovation $\neq 0$ → β เริ่มปรับขึ้น $t=0.5h$: $r_A=+0.15\%$, $r_B=+0.14\%$ → $\beta \approx 0.995$ $t=1.0h$: $r_A=+0.18\%$, $r_B=+0.17\%$ → $\beta \approx 1.015$ $t=1.5h$: $r_A=+0.20\%$, $r_B=+0.19\%$ → $\beta \approx 1.035$ $t=2.0h$: $r_A=+0.22\%$, $r_B=+0.21\%$ → $\beta \approx 1.05$ ✓

การปรับ position:

- $size_A = 1.0$ BTC (Long Bybit)
- เริ่มต้น short Lighter = 0.98 BTC ($\beta=0.98$)
- หลัง 2 ชั่วโมง $\beta=1.05$ → $\delta = (1.05-0.98) \times 1.0 = \mathbf{0.07 \text{ BTC}}$
- ต้อง short Lighter เพิ่มอีก 0.07 BTC เพื่อ hedge ถูกต้อง

ค่า ε ถ้าไม่ปรับ: $\varepsilon = \log(P_A) - 0.98 \times \log(P_B)$ แต่ β จริงเป็น 1.05 → ε drift ขึ้นจาก basis error ไม่ใช่ α จริง → อาจ trigger entry ปลอม

15.9 แบบฝึกหัด

โจทย์ 15.1 — คำนวณ Kalman Update ด้วยมือ

ให้: $\beta = 1.00$, $P = 0.001$, $Q = 1e-5$, $R = 1e-3$

Observation ใหม่: $r_A = +0.15\%$, $r_B = +0.12\%$

ถาม: คำนวณ β_{new} และ P_{new} ที่ละขั้น

► ดูคำตอบ

โจทย์ 15.2 — ผลของ Q ต่าง

ทดสอบ Kalman กับ $Q = 1e-6$ และ $Q = 1e-3$ บน series เดียวกันที่ β จริงเปลี่ยนจาก $1.0 \rightarrow 1.1$ ใน 100 steps

ถาม: describe ว่า Q แต่ละค่าจะ track β_{true} ได้อย่างไร มีผลดีผลเสียอะไร?

► ดูคำตอบ

โจทย์ 15.3 — Rebalancing Threshold

β เปลี่ยนจาก $1.00 \rightarrow 1.03$ ($\delta = 0.03$), $\text{size}_A = 5$ BTC, Lighter taker fee = 0.07%

ราคา BTC = \$65,000

ถาม: ค่า transaction cost ของการ rebalance คือเท่าไร? ควร rebalance หรือไม่ ถ้า expected P&L จากการแก้ hedge error = \$35?

► ดูคำตอบ

อ้างอิงสำคัญ (References)

- **Kalman, R. E. (1960).** A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45. — บทความต้นฉบับ Kalman filter; state-space recursion สำหรับ dynamic β_t ใน Ch.15

- **Welch, G., & Bishop, G. (2006).** *An Introduction to the Kalman Filter*. UNC Chapel Hill TR 95-041. — tutorial ที่อ่านง่ายที่สุดสำหรับ practical Kalman implementation; ครอบคลุม process noise Q และ measurement noise R
- **Elliott, R. J., van der Hoek, J., & Malcolm, W. P. (2005).** Pairs trading. *Quantitative Finance*, 5(3), 271–276. — ประยุกต์ Kalman filter กับ pairs trading spread ที่คล้าย Ch.15 โดยตรง
- **Vidyamurthy, G. (2004).** *Pairs Trading: Quantitative Methods and Analysis*. Wiley. — Ch.8 ของหนังสือนี้กล่าวถึง dynamic hedge ratio ด้วย Kalman

Ch.15 Dynamic Hedging ด้วย Kalman Filter | ต่อไป → **Ch.16: Multi-leg Portfolios**

Multi-leg Portfolios: บริหาร Portfolio ของ Spreads

หลาย spread พร้อมกัน — จัดการ correlation และ allocate capital อย่างมีประสิทธิภาพ

ตัวอย่าง: BTC Bybit ↔ Lighter + ETH Bybit ↔ Lighter + BTC Basis

แก่นของบทนี้ — อ่านก่อน

การรัน spread เดียวมีความเสี่ยงจาก idiosyncratic event ของ pair นั้น — portfolio ของหลาย spread ลด risk แต่ต้องระวัง **hidden correlation** ที่เพิ่มขึ้นในช่วงวิกฤต

$$w^* = (1/2\lambda) \Sigma^{-1} \mu \leftarrow \text{mean-variance optimal weights}$$

16.1 ทำไมต้องหลาย Spread

การ diversify ข้าม spread ให้ประโยชน์หลัก:

ประโยชน์

- ลด variance ของ P&L รวม — spread แต่ละคู่ไม่ correlated 100%
- Deploy capital ต่อเนื่อง — ถ้า spread หนึ่ง "ไม่มีสัญญาณ" spread อื่นอาจมี
- ลด dependency บน pair เดียว — ถ้า exchange หนึ่ง maintenance อีก strategy ยังรัน

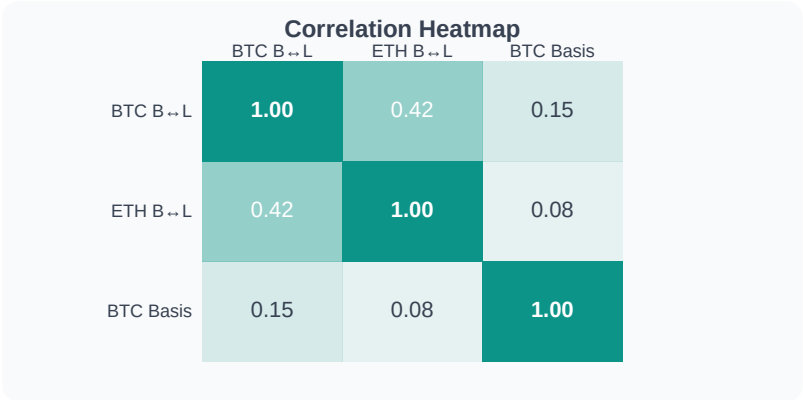
ความเสี่ยง

- Hidden correlation — BTC dump → ทุก crypto spread volatile พร้อมกัน
- Capital lock-up — ต้องการ margin สำหรับแต่ละ spread แยกกัน
- Operational complexity — monitor หลาย pair พร้อมกัน

16.2 Covariance Matrix ของ Spreads

นิยาม Σ = covariance matrix ขนาด $n \times n$ (n = จำนวน spread):

```
 $\Sigma_{ij} = \text{Cov}(\epsilon_i, \epsilon_j) = E[(\epsilon_i - \mu_i)(\epsilon_j - \mu_j)]$  ตัวอย่าง 3 spreads: BTC_B↔L  
ETH_B↔L BTC_Basis BTC_B↔L [ 1.00 0.42 0.15 ] ← variance normalized ETH_B↔L [  
0.42 1.00 0.08 ] BTC_Basis[ 0.15 0.08 1.00 ]
```



Correlation heatmap — สี่เหลี่ยมเข้มหมายถึง correlation สูง (ไม่ดีสำหรับ diversification)

16.3 Mean-Variance Allocation

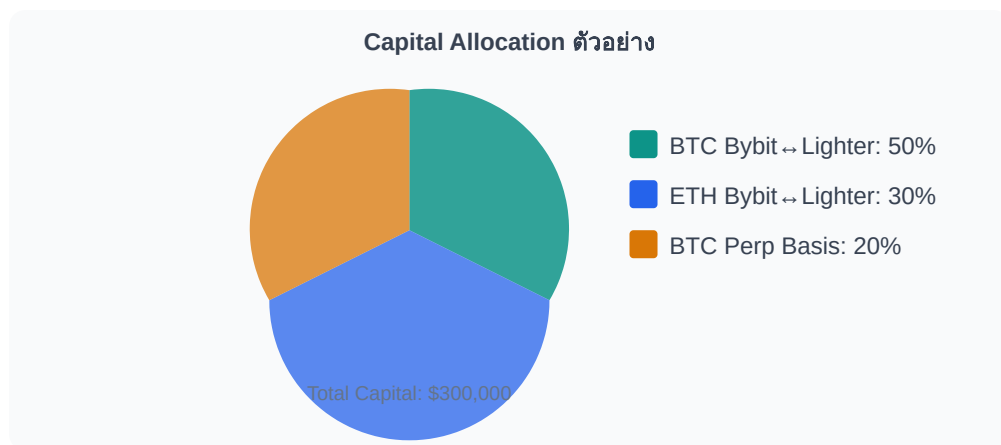
เป้าหมาย: หา weight vector $\mathbf{w}$ ที่ maximize risk-adjusted return:

$\max \mathbf{w}'\boldsymbol{\mu} - \lambda \times \mathbf{w}'\boldsymbol{\Sigma}\mathbf{w}$ subject to: $\sum w_i = 1, w_i \geq 0, w_i \leq w_{\max}$ Solution (unconstrained): $\mathbf{w}^* = (1/2\lambda) \times \boldsymbol{\Sigma}^{-1} \times \boldsymbol{\mu}$ โดย: $\boldsymbol{\mu}$ = vector ของ expected return แต่ละ spread $\boldsymbol{\Sigma}$ = covariance matrix λ = risk aversion parameter (ยิ่งสูง = conservative มากขึ้น) $\mathbf{w}^*$ = optimal weights

ข้อควรระวัง

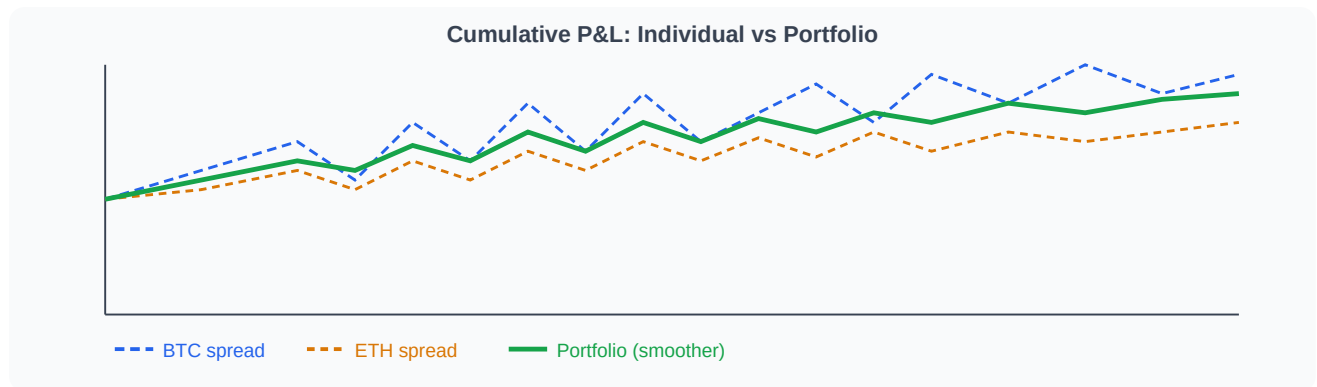
Mean-variance optimization **sensitive** มากต่อ estimate ของ $\boldsymbol{\mu}$ — error เล็กน้อยใน $\boldsymbol{\mu}$ ทำให้ $\mathbf{w}^*$ เปลี่ยนแปลงมาก วิธีแก้: ใช้ shrinkage (เช่น Ledoit-Wolf) หรือ equal-weight เป็น baseline แล้วปรับ deviation ตาม conviction เท่านั้น

16.4 Portfolio Weight — Pie Chart



ตัวอย่าง capital allocation — BTC spread ได้ weight มากสุดเพราะ Sharpe สูงสุดและ risk ต่ำสุด

16.5 Drawdown Comparison



Portfolio ผสม (เขียว) มี drawdown น้อยกว่าและ P&L smoother กว่า individual spread

16.6 Pseudo-code portfolio_allocate()

```
def portfolio_allocate(expected_returns, cov_matrix, risk_aversion=1.0,
max_weight=0.6, min_weight=0.05): """ expected_returns: vector of  $\mu_i$ 
(annualized expected return) cov_matrix:  $\Sigma$  matrix (n×n) risk_aversion:  $\lambda$ 
parameter (higher = more conservative) max_weight: cap ต่อ strategy เดียว
min_weight: minimum allocation ถ้า include strategy นั้น """ n =
len(expected_returns) # Inverse covariance (Ledoit-Wolf shrinkage recommended)
inv_cov = ledoit_wolf_inverse(cov_matrix) # Raw MV optimal weights raw_weights
= inv_cov @ expected_returns / (2 * risk_aversion) # Clip to [0, max_weight]
weights = clip(raw_weights, 0, max_weight) # Zero out strategies below
min_weight threshold weights = [w if w >= min_weight else 0.0 for w in weights]
# Normalize to sum = 1 total = sum(weights) if total > 0: weights = [w / total
for w in weights] return weights # list of n weights summing to 1.0
```

16.7 Correlation Risk ในช่วงวิกฤต

Hidden Correlation Problem

ในช่วงตลาดปกติ BTC spread กับ ETH spread correlate 0.42 — แต่ในช่วง BTC dump correlation อาจพุ่งเป็น 0.85+ เพราะทั้งสองตลาดถูก panic ขายพร้อมกัน

นี่คือ "correlation breakdown" — assumption ของ portfolio optimization ล้มเหลวตอนที่ต้องการ protection มากที่สุด

```
def detect_correlation_spike(spread_returns, window=30, threshold=0.70):  
    """ตรวจว่า rolling correlation ระหว่าง spreads สูงขึ้นผิดปกติ"""  
    rolling_corr =  
    compute_rolling_correlation(spread_returns, window)  
    max_off_diagonal =  
    max_corr_excluding_diagonal(rolling_corr)  
    if max_off_diagonal > threshold:  
        return True, f"Correlation spike: {max_off_diagonal:.2f}"  
    return False, None
```

16.8 Running Example — 3 Spread Portfolio

Running Example: Portfolio ของ 3 Spreads

Strategy	Weight	Capital	Expected Sharpe	σ ต่อปี
BTC Bybit $\leftrightarrow$ Lighter	50%	\$150,000	2.1	12%
ETH Bybit $\leftrightarrow$ Lighter	30%	\$90,000	1.7	15%
BTC Perp Basis	20%	\$60,000	1.4	8%
Portfolio	100%	\$300,000	2.3	10%

สังเกต: Portfolio Sharpe (2.3) สูงกว่า Sharpe ของแต่ละ strategy เดี่ยว — เป็นผลจาก diversification benefit เมื่อ correlation ไม่เท่ากับ 1

16.9 แบบฝึกหัด

แบบฝึกหัดบทที่ 16

- ▶ ข้อ 1: ถ้า BTC dump ทำให้ correlation ระหว่าง BTC และ ETH spread เพิ่มขึ้นเป็น 0.90 — ควรทำอะไร?
- ▶ ข้อ 2: ทำไม equal-weight portfolio มักดีกว่า MV optimal ในทางปฏิบัติ?
- ▶ ข้อ 3: เพิ่ม spread ที่ 4 เข้า portfolio — ควรตรวจสอบสิ่งใดก่อน?

Commodity Basis: Calendar Spread ในตลาด Futures

Front – Back futures basis มาจาก storage cost + convenience yield

ทำความเข้าใจโครงสร้างราคา Futures ก่อนที่จะ trade calendar spread

แก่นของบทนี้ — อ่านก่อน

Calendar spread คือ basis ระหว่าง futures สองตัวต่างวัน expiry ของ asset เดียวกัน ค่านี้ไม่ใช่ random — มันถูกกำหนดโดยต้นทุนการเก็บรักษาและ convenience yield ของ commodity นั้น เมื่อ basis เบี่ยงออกจาก fair value ก็เกิดโอกาส arbitrage

$$\text{Basis} \approx (\text{storage_cost} - \text{convenience_yield}) \times (T_2 - T_1) / 365$$

17.1 Calendar Spread คืออะไร

Calendar spread คือการ long futures ที่ expiry เร็ว (front month) และ short futures ที่ expiry ช้า (back month) ของ underlying เดียวกัน หรือกลับกัน โดยที่เราไม่ได้ bet ทิศทางของราคา underlying แต่ bet ว่าความต่างระหว่างสอง expiry จะกลับสู่ fair value

ตัวอย่างพื้นฐาน: Crude Oil Calendar Spread

CL June futures = \$80.00 | CL September futures = \$82.50
Calendar spread = Front – Back = \$80.00 – \$82.50 = **-\$2.50**
Market คาดว่า oil ใน Sep จะแพงกว่า Jun \$2.50 → ตลาดอยู่ใน **Contango**

เปรียบเทียบ: Calendar Spread vs. Outright

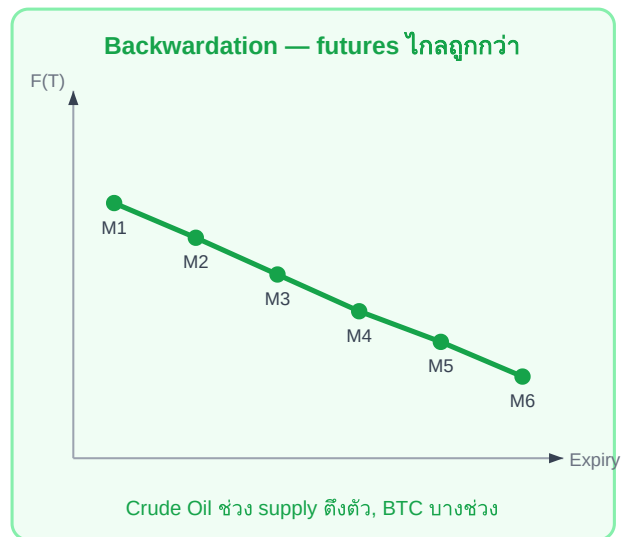
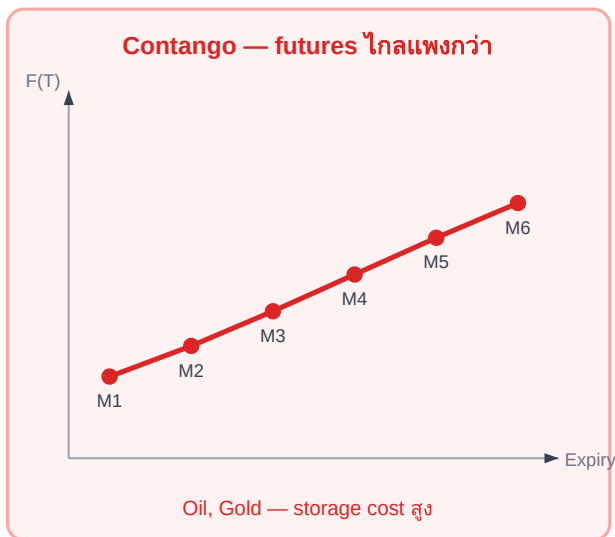
Outright long CL: P&L ขึ้นตรงกับราคา oil — เสี่ยง directional risk เต็มๆ
Calendar Spread: P&L ขึ้นกับ ความต่าง ระหว่าง expiry — delta ต่อ oil เกือบ zero, เสี่ยงน้อยกว่ามาก

ประเภท Position	Long Front / Short Back	Short Front / Long Back
ชื่อเรียก	Long Calendar Spread	Short Calendar Spread
กำไรเมื่อ	Contango ลดลง (spread กว้างขึ้น)	Contango เพิ่มขึ้น (spread แคบลง)
สถานการณ์ที่ดี	อุปสงค์ระยะสั้นพุ่งสูง, storage ดึงตัว	supply ระยะสั้นล้นตลาด
Risk หลัก	Contango ขยายมากกว่าที่คาด	Market พลิกเป็น Backwardation

17.2 Contango vs. Backwardation

โครงสร้าง futures term structure บอกว่า market คาดการณ์ราคาในอนาคตอย่างไร:

- **Contango:** $F(T_{back}) > F(T_{front})$ — futures ไกลแพงกว่า futures ใกล้ — ปกติสำหรับ commodity ที่มีต้นทุนเก็บรักษาสูง (oil, grain, metals)
- **Backwardation:** $F(T_{back}) < F(T_{front})$ — futures ไกลถูกกว่า — เกิดเมื่อ convenience yield สูง หรือ อุปสงค์ระยะสั้นตึงตัวมาก
- **Flat:** ราคาเท่ากันทุก expiry — หายาก มักเกิดช่วงผ่านระหว่าง Contango ↔ Backwardation



Contango (ซ้าย): term structure ลาดขึ้น | Backwardation (ขวา): term structure ลาดลง

17.3 Fair Basis Formula — Cost of Carry Model

ราคา futures ที่ "fair" ต้องสะท้อน cost of carry ครบถ้วน มิฉะนั้นจะมี cash-and-carry arbitrage:

สูตร COST OF CARRY

$$F(T) = S \times e^{\{(r + u - q) \times T\}}$$

โดยที่: **S** = ราคา spot, **r** = risk-free rate, **u** = storage cost (% ต่อปี), **q** = convenience yield (% ต่อปี), **T** = เวลาถึง expiry (ปี)

$$\text{Calendar Spread Fair} = F(T_1) - F(T_2) = S \times e^{\{(r+u-q)T_1\}} - S \times e^{\{(r+u-q)T_2\}}$$

เมื่อ basis เบี่ยงจาก fair value เกิดโอกาส arbitrage:

ถ้า $F(T_2) - F(T_1) > \text{fair_basis}$ → contango แพงเกินไป → Long $F(T_1)$, Short $F(T_2)$ → wait for convergence
ถ้า $F(T_2) - F(T_1) < \text{fair_basis}$ → backwardation/contango น้อยเกินไป → Short $F(T_1)$, Long $F(T_2)$ → wait for convergence

Parameter	Crude Oil (ตัวอย่าง)	Gold (ตัวอย่าง)	BTC Futures
r (risk-free)	5.25% / ปี	5.25% / ปี	5.25% / ปี
u (storage)	0.5–1.5% / ปี	0.1–0.3% / ปี	~0% (digital)
q (convenience)	2–10% / ปี (volatile)	0.5–1% / ปี	0% (ไม่มี utility)
ลักษณะ	Contango หรือ Back ได้ทั้งคู่	มักเป็น Contango เสมอ	ขึ้นกับ funding rate

17.4 ตัวอย่าง: Crude Oil Calendar Spread

Running Example: CL (Crude Oil) Calendar Spread

ข้อมูล ณ วันที่วิเคราะห์:

- CL Front (July) = **\$80.00**/barrel
- CL Back (September) = **\$82.00**/barrel
- Observed spread (Back – Front) = +\$2.00 (Contango \$2)

คำนวณ fair basis สำหรับ 60 วัน ($T = 60/365 \approx 0.164$ ปี):

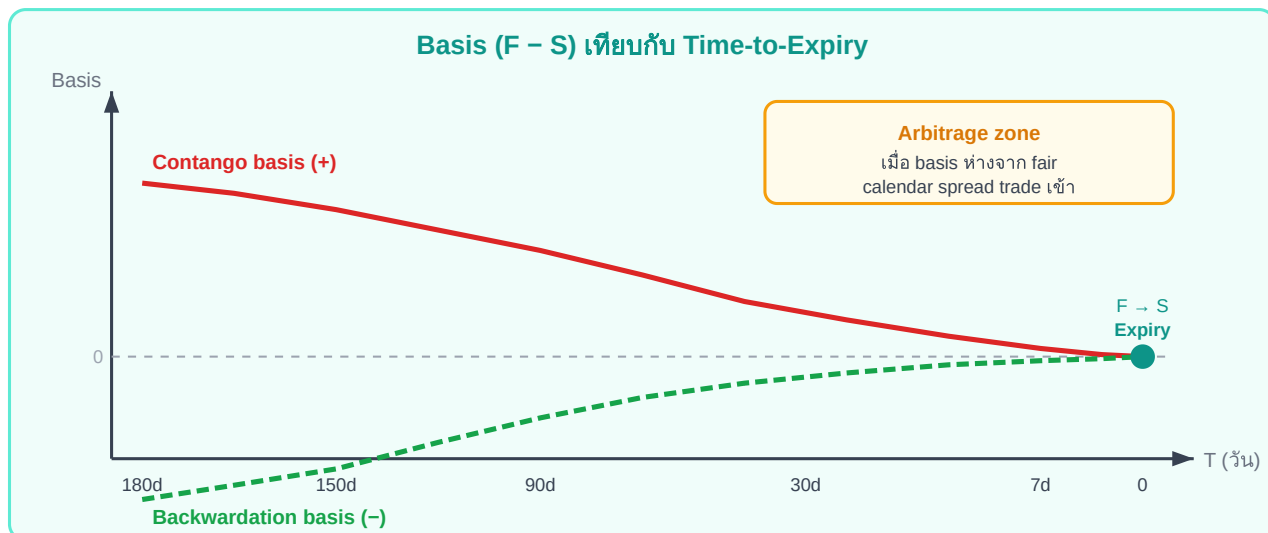
$r = 5.25\%$, $u = 1.0\%$, $q = 3.0\% \rightarrow (r + u - q) = 3.25\%$ Fair $F(\text{back}) / \text{Fair } F(\text{front}) = e^{\{(r+u-q) \times (T_{\text{back}} - T_{\text{front}})\}} = e^{\{0.0325 \times (60/365)\}} = e^{\{0.00534\}} \approx 1.00535$ Fair spread = $F(\text{front}) \times 0.00535 = \$80 \times 0.00535 = \$0.428$ Observed spread = \$2.00 vs Fair = \$0.43 $\rightarrow$ Contango แพงเกินไปมาก (\$1.57 excess) $\rightarrow$ Trade: Long Front (July) + Short Back (Sep)

ความเสี่ยงที่ต้องระวัง

- **Convenience yield** เปลี่ยนแปลงกะทันหัน: ข่าว OPEC, supply disruption $\rightarrow$ Backwardation พุ่งแรง
- **Delivery risk**: futures ใกล้ expiry ต้อง physical delivery หรือ roll ออก
- **Roll cost**: ถ้า contango กว้าง การ roll position มีต้นทุนสูง

17.5 Basis vs. Time-to-Expiry

เมื่อ expiry ใกล้เข้ามา basis ของ futures จะต้อง converge สู่ 0 (เนื่องจาก $F(T \rightarrow 0) = S$)



Basis ทุกประเภทต้อง converge สู่ 0 เมื่อถึง expiry — นี่คือ "pull to par" ของ futures

17.6 เทียบกับ Crypto Perpetual — Concept เดียวกัน แต่วางกันตรงไหน?

Crypto perpetual futures ใช้ concept เดียวกับ calendar spread แต่มีความแตกต่างสำคัญ:

ลักษณะ	Commodity Calendar Spread	Crypto Perpetual Basis
Expiry	มี expiry — convergence มีวันกำหนด	ไม่มี expiry — ไม่มี forced convergence
Mechanism ดึงกลับ	Cost of carry + expiry convergence	Funding rate (ทุก 8h หรือ 1h)
Basis = ?	$F(T_2) - F(T_1)$ vs cost of carry	Perp – Spot vs funding rate integral
Risk	Delivery, roll cost, supply shock	Funding ขาด liquidity, exchange risk
Calendar spread ใน crypto	ทำได้กับ quarterly futures	Perp vs. quarterly = "basis trade"

BTC Basis Trade: Perp vs. Quarterly Future

BTC Perp = \$65,000 | BTC Sep Quarterly = \$66,200

Observed basis = +\$1,200 (quarterly แพงกว่า 1.85%)

Annualized funding rate implied = $1.85\% \times (365/90) \approx 7.5\%$ / ปี

ถ้า funding rate จริงต่ำกว่า 7.5% → Quarterly แพงเกินไป → **Long Perp + Short Quarterly**

17.7 แบบฝึกหัด

โจทย์ 17.1 — คำนวณ Fair Calendar Spread

Natural Gas (NG) futures: Front (August) = \$2.80/MMBtu, Back (November) = \$3.20/MMBtu
 $r = 5\%$, storage cost $u = 8\%$ ต่อปี, convenience yield $q = 2\%$ ต่อปี, $T_{\text{back}} - T_{\text{front}} = 90$ วัน

ถาม: Fair spread คือเท่าไร? Basis ปัจจุบัน (observed) เป็น Contango มากเกินไปหรือน้อยเกินไป?

► ดูคำตอบ

โจทย์ 17.2 — Crypto Basis Comparison

BTC Perpetual = \$67,500 | BTC December Quarterly Futures = \$68,900

เวลาถึง expiry Dec = 120 วัน | Risk-free rate = 5% ต่อปี

ถาม: Implied annualized basis คือเท่าไร? ถ้า funding rate เฉลี่ยที่คาด = 10% ต่อปี ควร trade อย่างไร?

► ดูคำตอบ

Options Overlay: Put-Call Parity Arbitrage

$\varepsilon = \text{Call} - \text{Put} - (F - K)e^{-rT}$ — deviation คือโอกาส arbitrage

ค้นหาและ exploit ความไม่สมดุลใน options market อย่างเป็นระบบ

แก่นของบทนี้ — อ่านก่อน

Put-Call Parity เป็นกฎพื้นฐานที่ต้องเป็นจริงในตลาดที่ไม่มี arbitrage เมื่อ $\varepsilon \neq 0$ แสดงว่า call หรือ put ราคาผิดสัมพันธ์กัน และสามารถทำ riskless profit ได้โดยการ construct portfolio ที่สอดคล้อง

$$\varepsilon = C - P - (F - K) \times e^{-rT}$$

18.1 Put-Call Parity — สูตรและที่มา

Put-Call Parity เป็น no-arbitrage relationship ที่เชื่อม call, put, forward, และ strike เข้าด้วยกัน:

PUT-CALL PARITY สำหรับ EUROPEAN OPTIONS

สำหรับ European options บน futures contract F ที่ strike K expiry T :

$$C - P = (F - K) \times e^{-rT}$$

สำหรับ European options บน spot asset S :

$$C - P = S \times e^{-qT} - K \times e^{-rT}$$

โดยที่: C = call price, P = put price, r = risk-free rate, q = dividend/funding yield

ที่มาของสูตร: พิจารณา 2 portfolio ที่ให้ payoff เดียวกัน:

Portfolio	ส่วนประกอบ	Payoff เมื่อ $F_T > K$	Payoff เมื่อ $F_T \leq K$
A: Long Call	Buy C, Invest $(F-K)e^{-rT}$	$(F_T - K) + (F-K)$	$0 + (F-K)$
B: Long Fwd + Put	Long F at K, Buy P	$(F_T - K) + 0$	$0 + (K - F_T)$

เนื่องจาก payoff เท่ากัน ราคาต้องเท่ากัน $\rightarrow C - P = (F - K)e^{-rT}$ ต้องเป็นจริงเสมอ

18.2 Parity Deviation ε — เมื่อไรจึง Arbitrage ได้

นิยาม PARITY DEVIATION

$$\varepsilon = C - P - (F - K) \times e^{-rT}$$

ถ้า $\varepsilon > 0$: Call แพงเกินไปเทียบกับ put → Sell call, Buy put, Buy forward

ถ้า $\varepsilon < 0$: Put แพงเกินไปเทียบกับ call → Buy call, Sell put, Sell forward

ถ้า $\varepsilon = 0$: parity สมบูรณ์ — ไม่มีโอกาส arbitrage

เงื่อนไขที่ $\varepsilon \neq 0$ ในทางปฏิบัติ

- **Bid-Ask spread:** ε ต้องเกิน bid-ask cost รวมทุก leg ก่อนจะ profitable
- **Margin/collateral cost:** ต้องใช้ margin วาง → มีต้นทุน opportunity
- **Execution risk:** ต้อง fill ทุก leg พร้อมกัน ไม่เช่นนั้นเป็น legged risk
- **American vs European:** American options มี early exercise premium → parity แตกต่าง

18.3 BTC Options ตัวอย่าง — Deribit Put-Call Parity

Running Example: BTC Options บน Deribit

ข้อมูลจาก Deribit สำหรับ BTC-28JUN-65000 (Strike K = \$65,000, expiry 30 วัน):

- BTC Futures (28JUN) = **\$66,200**
- Call (K=65000, T=30d) = **\$2,850**
- Put (K=65000, T=30d) = **\$1,520**
- Risk-free rate $r = 5\%$ ต่อปี $\rightarrow e^{-rT} = e^{-0.05 \times 30/365} = 0.9959$

Theoretical $C - P = (F - K) \times e^{-rT} = (\$66,200 - \$65,000) \times 0.9959 = \$1,200 \times 0.9959 = \$1,195.08$
Observed $C - P = \$2,850 - \$1,520 = \$1,330$
 $\epsilon = \$1,330 - \$1,195 = +\$135 \rightarrow$ Call แพงเกินไปเทียบกับ Put Trade: Sell Call + Buy Put + Long Forward (at K=\$65,000) Expected profit $\approx \$135$ (ก่อนหักค่าธรรมเนียม)

ตรวจสอบต้นทุนก่อน Trade

Deribit taker fee $\approx 0.03\%$ ต่อ option position (per leg)

4 legs: Sell C + Buy P + Long F + Short F (delta hedge) $\rightarrow$ total fee $\approx 0.12\% \times$ notional

Notional $\approx \$65,000 \rightarrow$ fee $\approx \$78$

Net profit $\approx \$135 - \$78 = \mathbf{\$57}$ ต่อ BTC $\rightarrow$ ยังคุ้ม แต่ต้องระวัง execution

18.4 IV Surface Arbitrage — Calendar Spread ใน IV

นอกจาก put-call parity แล้ว ยังมีโอกาส arbitrage จากความไม่สอดคล้องของ Implied Volatility (IV) ข้าม expiry:

ประเภทของ IV Surface Arbitrage

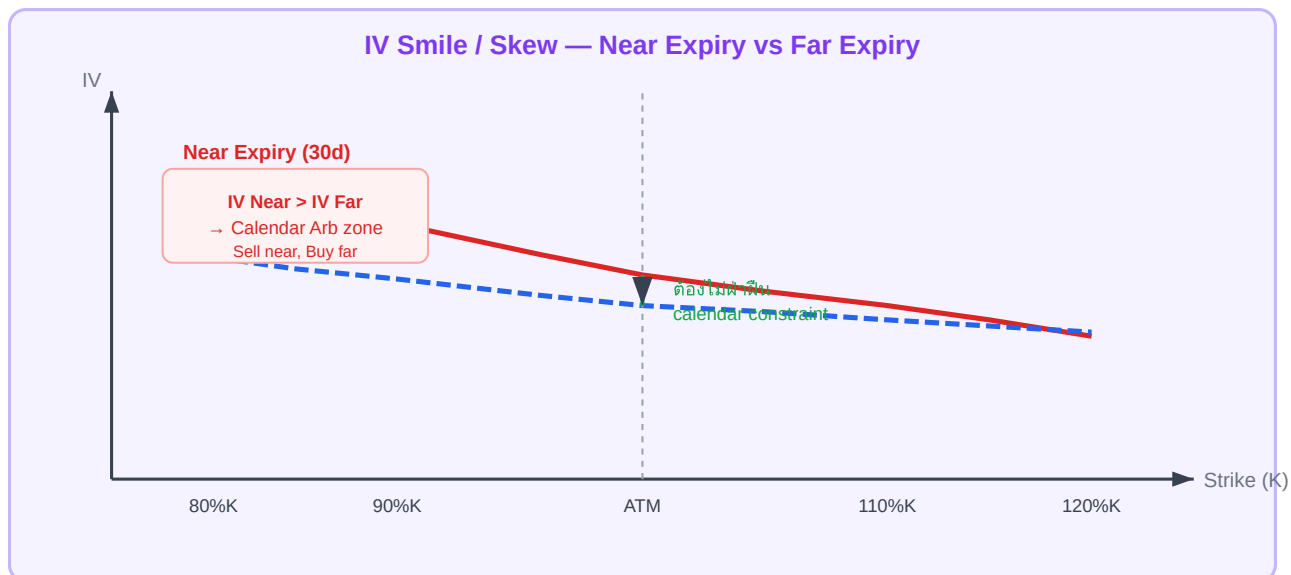
- **Calendar Spread IV Arb:** IV ของ near-expiry สูงกว่า far-expiry ผิดปกติ → sell near, buy far vega
- **Strike Spread (Skew Arb):** IV skew ระหว่าง OTM put และ OTM call ผิดปกติ
- **Butterfly Arb:** IV ของ ATM ผิดจาก weighted average ของ OTM wings

No-Arbitrage Constraints ของ IV Surface

Constraint	สูตร / เงื่อนไข	ถ้าฝ่าฝืน
Calendar spread	Total variance $V(T) = IV^2 \times T$ ต้องเพิ่มตาม T	Sell near, buy far → calendar arb
Butterfly	$\partial^2 C / \partial K^2 \geq 0$ (call spread เป็น convex)	Butterfly spread ให้ riskless profit
Call spread	$\partial C / \partial K \leq 0$ (call price ลดเมื่อ K ขึ้น)	Vertical spread arb
Horizontal bound	$C(K, T_2) \geq C(K, T_1)$ สำหรับ $T_2 > T_1$	Calendar spread arb

18.5 IV Surface Diagram (2D Slice)

แผนภาพ 2D slice ของ IV surface ที่ expiry คงที่ (IV vs. Strike):



Near expiry IV (แดง) ต้องไม่สูงกว่า far expiry IV อย่างผิดปกติที่ strike เดียวกัน มิฉะนั้นเป็น calendar arb

18.6 Delta Hedge ของ Options Spread

เมื่อ trade put-call parity arbitrage หรือ IV surface arb ต้องระวังว่า position มี **delta exposure** ที่ต้อง hedge ออก:

ขั้นตอน Delta Hedge

1. คำนวณ net delta ของ options portfolio: $\Delta_{\text{net}} = \Delta_{\text{call}} - \Delta_{\text{put}} + \Delta_{\text{forward}}$
2. Sell/Buy futures ในปริมาณ Δ_{net} เพื่อให้ net delta ≈ 0
3. Re-hedge เมื่อ delta drift (เนื่องจาก gamma effect) หรือทุก interval ที่กำหนด
4. ระวัง gamma risk: position ที่ delta-neutral อาจมี gamma ที่ทำให้ P&L เป็น non-linear

ตัวอย่าง: ϵ trade บน BTC K=65000, T=30d Long Put: $\Delta_{\text{put}} = -0.42 \rightarrow \text{contribution} = -1 \times (-0.42) = +0.42$ Short Call: $\Delta_{\text{call}} = +0.58 \rightarrow \text{contribution} = -1 \times (0.58) = -0.58$ Long Fwd: $\Delta_{\text{fwd}} = +1.00 \rightarrow \text{contribution} = +1.00$ Net $\Delta = +0.42 - 0.58 + 1.00 = +0.84 \rightarrow$ Sell 0.84 BTC futures เพื่อ delta-neutral

Re-hedge Frequency

สำหรับ crypto options ที่ volatile มาก: re-hedge ทุก 30–60 นาที หรือทุกครั้งที่ delta drift เกิน ± 0.05
Transaction cost ของ re-hedge ต้องรวมใน edge calculation (บทที่ 19)

18.7 แบบฝึกหัด

โจทย์ 18.1 — คำนวณ ϵ และ Direction

ETH Options บน Deribit: Strike $K = \$3,500$, Expiry = 45 วัน

ETH Futures (45d) = $\$3,620$ | Call = $\$285$ | Put = $\$148$ | $r = 5\%$ ต่อปี

ถาม: คำนวณ ϵ และระบุ direction ของ trade arbitrage

► ดูคำตอบ

โจทย์ 18.2 — IV Calendar Constraint

BTC options ที่ $K = \text{ATM}$: $\text{IV}_{30d} = 72\%$, $\text{IV}_{90d} = 58\%$

Total variance: $V_{30} = 0.72^2 \times (30/365) = 0.0426$, $V_{90} = 0.58^2 \times (90/365) = 0.0832$

ถาม: IV surface นี้ violate calendar constraint หรือไม่? ถ้า violate ควร trade อย่างไร?

► ดูคำตอบ

Cost Analysis & Edge: คำนวณ Net Edge ก่อน เทรด

Gross spread – fee – bid/ask – slippage – funding – legging buffer = Net edge

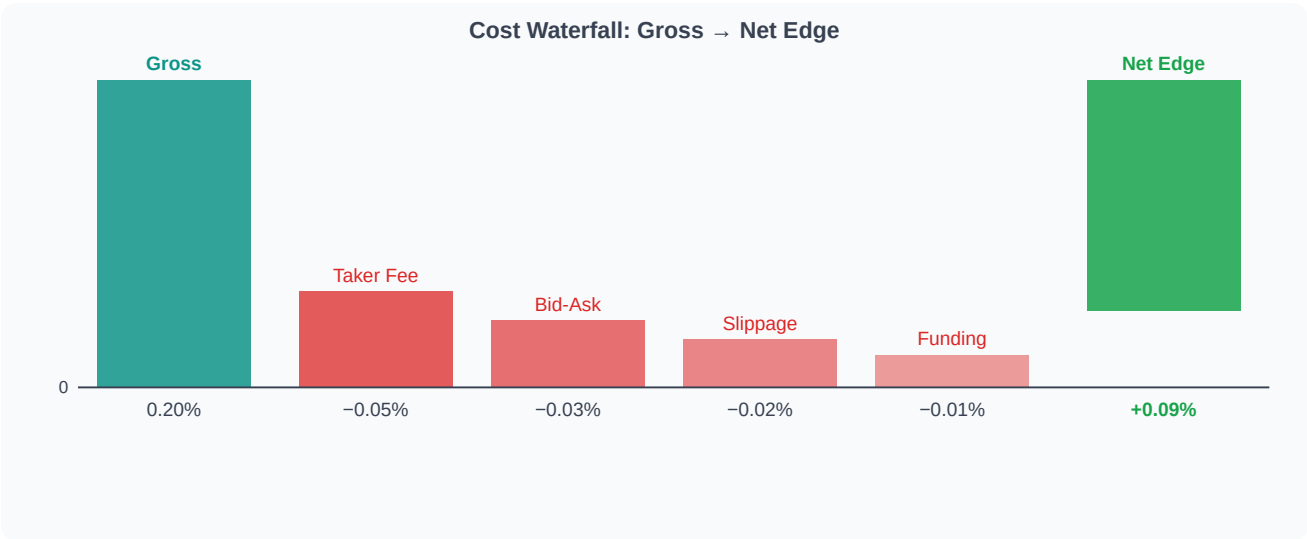
ถ้า net edge ≤ 0 → ไม่เทรด ไม่ว่า signal จะดูดีแค่ไหน

แก่นของบทนี้ — อ่านก่อน

Stat arb มีต้นทุนซ่อนอยู่หลายชั้น — signal ที่ดูมี edge 0.20% อาจมี net edge ลบหลัง cost หักออกทั้งหมด **คำนวณก่อนเสมอ** — ห้าม assume

$$\text{Net Edge} = \text{Gross} - \Sigma(\text{fees} + \text{spread} + \text{slippage} + \text{funding} + \text{legging})$$

19.1 Edge Waterfall Diagram



Waterfall chart: แต่ละ cost component ลดลงจาก gross spread ของ 0.20% — เหลือ net edge เพียง 0.09%

19.2 Fee Structure

Exchange	Maker Fee	Taker Fee	หมายเหตุ
Bybit Perp	-0.025% (rebate)	+0.055%	VIP levels ลด taker ได้
Lighter Perp	-0.010%	+0.030%	clob-based, faster fills
Round trip cost (2 takers)			$(0.055+0.030)\times 2 = 0.17\%$
Round trip cost (2 makers)			rebate: -0.07% (receive)

Maker vs Taker Strategy

ถ้า latency ไม่ critical → ใช้ limit order (maker) ทั้งคู่ — ลด cost จาก -0.17% เป็น -0.03% ต่อ round trip แต่ risk partial fill และ leg mismatch เพิ่มขึ้น

19.3 Bid-Ask Spread Cost

Cost per leg = $(\text{Ask} - \text{Bid}) / 2 = \text{half-spread}$ สำหรับ 2 legs round trip: Total bid-ask cost = half-spread_A + half-spread_B ตัวอย่าง Bybit BTC: Bid = 65,000, Ask = 65,002 Half-spread = \$1 = $1/65,000 = 0.0015\%$ Lighter BTC: Bid = 64,998, Ask = 65,001 Half-spread = \$1.50 = 0.0023% Total: $0.0015\% + 0.0023\% = 0.0038\%$ ต่อ direction Round trip (entry + exit): $0.0076\% \approx 0.008\%$

19.4 Slippage Model

Market impact (slippage) เพิ่มขึ้นตาม size ที่เทียบกับ average daily volume:

$\text{Slippage} \approx \sigma_{\text{daily}} \times \sqrt{\text{size} / \text{ADV}}$ โดย: σ_{daily} = daily volatility ของ asset
size = ขนาด order (notional) ADV = average daily volume (notional) ตัวอย่าง:
 $\sigma_{\text{daily}} = 0.02$ (2%) size = \$100,000 ADV = \$500,000,000 (Bybit BTC perp
~\$500M/day) $\text{Slippage} = 0.02 \times \sqrt{100,000 / 500,000,000} = 0.02 \times 0.014 =$
 $0.00028 = 0.028\%$

19.5 Funding Cost

Funding cost ต่อ 8h = $|\text{funding_rate}| \times \text{notional}$ Total funding cost = $\text{funding_rate} \times \text{hold_periods} \times \text{notional}$ ตัวอย่าง: Funding rate = 0.01% ต่อ 8h (positive → longs จ่าย) Hold time = 2 funding periods = 16h Notional = \$100,000
Cost (ถ้า short perp) = $+0.01\% \times 2 \times \$100,000 = +\$20$ (รับ) Cost (ถ้า long perp) = $-0.01\% \times 2 \times \$100,000 = -\$20$ (จ่าย)

Funding Direction ขึ้นกับ Position

ถ้า basis > 0 และ short perp → รับ funding (ลด cost)

ถ้า basis > 0 และ long perp → จ่าย funding (เพิ่ม cost)

ต้องคำนวณรวมในทุก scenario ก่อน decide trade direction

19.6 Legging Buffer

Legging risk = ความเสี่ยงจากช่วงเวลาระหว่าง fill leg A และ fill leg B ในช่วงนั้น
position เป็น single-leg → exposed ต่อ market movement Legging buffer estimate:
$$\text{buffer} = \sigma_{\text{spread}} \times \sqrt{\text{avg_fill_latency_s} / \text{seconds_per_period}}$$
 ตัวอย่าง:
 $\sigma_{\text{spread}} = 0.05\%$ per period $\text{avg_fill_latency} = 0.5$ วินาที $\text{seconds_per_period} = 60$
วินาที (1 นาที) $\text{buffer} = 0.05\% \times \sqrt{0.5/60} = 0.05\% \times 0.091 = 0.0045\%$

19.7 Break-even Analysis

Minimum σ_{stat} เพื่อให้ trade คำนวณค่า: Break-even $\sigma = \text{total_cost} / (z_{\text{entry}} - z_{\text{exit}})$
ตัวอย่าง: $\text{total_cost} = 0.11\%$ $z_{\text{entry}} = 2.0$, $z_{\text{exit}} = 0.5$ $\sigma_{\text{stat}} = z \times \sigma_{\text{spread}} \rightarrow$
spread ที่ entry $= 2.0 \times \sigma_{\text{spread}}$ Break-even: $0.11\% / (2.0 - 0.5) = 0.073\% \rightarrow$
ต้องการ $\sigma_{\text{spread}} \geq 0.073\%$ เพื่อให้ entry at $z=2.0$ cover cost $\rightarrow$ ถ้า $\sigma_{\text{spread}} = 0.05\% \rightarrow$
entry spread $= 0.10\%$, net edge $= -0.01\%$ (ไม่ trade!)

19.8 Pseudo-code compute_edge()

```
def compute_edge(spread_at_entry_pct, sigma_spread_pct, entry_z, exit_z,
fee_a=0.00055, fee_b=0.00030, half_spread_a=0.00002, half_spread_b=0.00003,
size=100_000, adv=500_000_000, sigma_daily=0.02, funding_per_8h=0.0001,
hold_periods=1, avg_fill_latency_s=0.5, seconds_per_period=60): gross =
spread_at_entry_pct / 100.0 # convert to decimal # Fee cost (round trip, taker
both sides) fee_cost = (fee_a + fee_b) * 2 # Bid-ask cost (round trip) ba_cost
= (half_spread_a + half_spread_b) * 2 # Slippage cost slip_cost = sigma_daily *
sqrt(size / adv) * 2 # entry + exit # Funding cost (funding_per_8h in decimal
fraction, e.g. 0.0001 = 0.01%/8h) fund_cost = funding_per_8h * hold_periods #
Legging buffer leg_buffer = (sigma_spread_pct/100) *
sqrt(avg_fill_latency_s/seconds_per_period) total_cost = fee_cost + ba_cost +
slip_cost + fund_cost + leg_buffer net_edge = gross - total_cost
breakeven_sigma = total_cost / (entry_z - exit_z) * 100 # as percent return {
"gross_pct": gross * 100, "total_cost_pct": total_cost * 100, "net_edge_pct":
net_edge * 100, "breakeven_sigma_pct": breakeven_sigma, "viable": net_edge > 0
}
```

19.9 Running Example — Bybit ↔ Lighter Cost Breakdown

Running Example: BTC Spread Trade Cost Analysis

Component	Calculation	Cost (bps)
Gross spread at entry ($z=2.0$, $\sigma=7.5\text{bps}$)	$2.0 \times 7.5\text{bps}$	+15.0 bps
Taker fee Bybit ($0.055\% \times 2$)	round trip	-11.0 bps
Taker fee Lighter ($0.030\% \times 2$)	round trip	-6.0 bps
Bid-ask spread (both venues)	$2 \times \text{half-spread}$	-0.8 bps
Slippage (\$100k on \$500M ADV)	$\sigma \times \sqrt{\text{size}/\text{ADV}}$	-0.3 bps
Funding (1 period, $0.01\%/8\text{h}$)	hold 8h	-1.0 bps
Legging buffer (0.5s latency)	$\sigma \times \sqrt{0.5/60}$	-0.5 bps
Net Edge		-4.6 bps (-0.046%, LOSS)

สรุป: Gross 15bps แต่ taker cost (17bps) + อื่น ๆ (2.6bps) = 19.6bps → net = -4.6bps → ห้ามเทรด
ในสภาพ retail taker — ต้องใช้ maker tier (rebate) หรือรอจน $z \geq 3.3$

Break-even σ : total cost 19.6bps / (2.0 - 0.5) = 13.1bps → ต้องการ $\sigma_{\text{spread}} \geq 13.1\text{bps}$ (σ จริง = 7.5bps ❌ ไม่ผ่าน) → ลด cost ด้วย maker pricing หรือเลือก venue ที่ fee ต่ำกว่า

19.10 แบบฝึกหัด

แบบฝึกหัดบทที่ 19

- ▶ ข้อ 1: ถ้า $\sigma_{\text{spread}} = 3\text{bps}$ และ $\text{total cost} = 4\text{bps}$ — ควรเทรดที่ $z=2.0$ ไหม?
- ▶ ข้อ 2: วิธีลด fee cost จาก 17bps (taker) เป็น rebate (maker) — ทำได้อย่างไร?
- ▶ ข้อ 3: ถ้าต้องการ minimum net edge 5bps — คำนวณ minimum z_{entry} ที่ต้องใช้

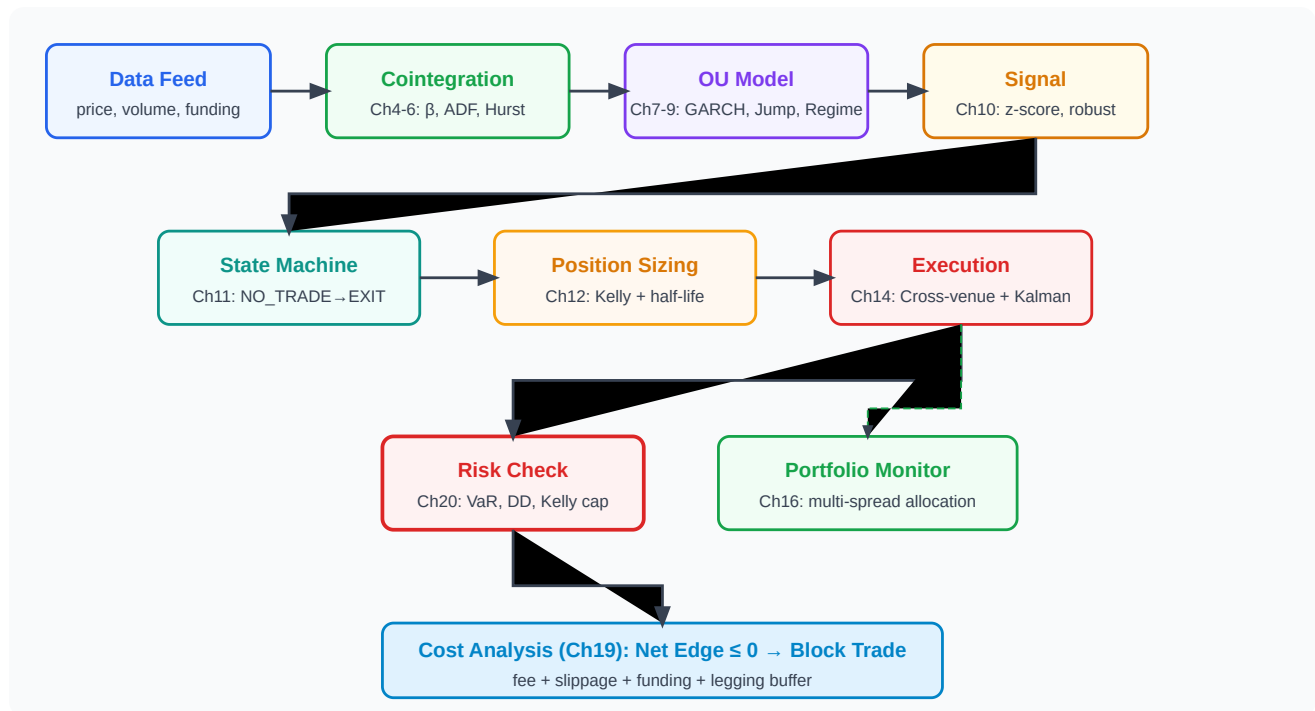
Risk Management & Portfolio: สรุป Framework ทั้งหมด

จาก cointegration ถึง risk limits — ภาพรวมของ stat arb system ที่สมบูรณ์
ทุกชิ้นส่วนทำงานร่วมกัน — เพิกเฉยชั้นเดียวทำให้ทั้งระบบพัง

แก่นของบทนี้ — บทสรุป

Stat arb ไม่ใช่แค่สูตรคณิตศาสตร์ — มันคือระบบที่ต้องการ **data quality + statistical rigor + execution discipline + risk management** ทำงานพร้อมกัน ขาดด้านใดด้านหนึ่งทำให้ edge หายไป

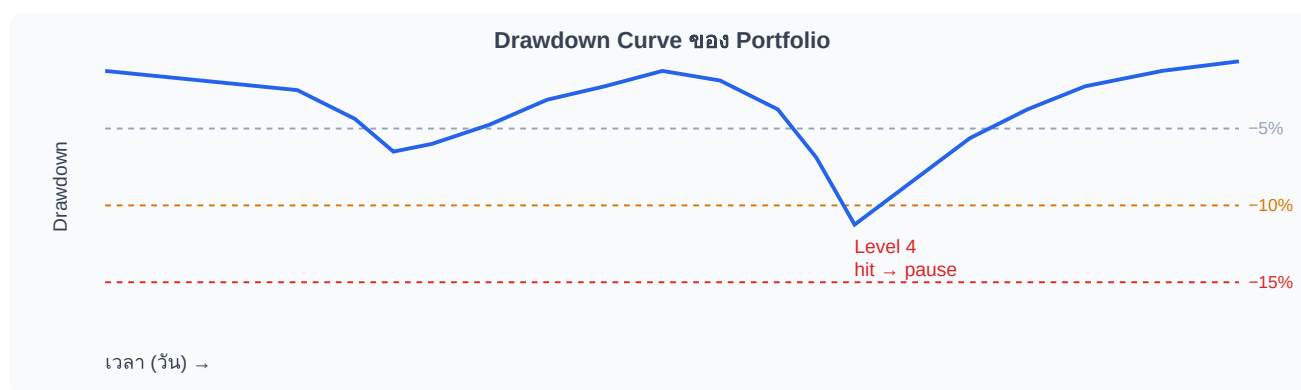
20.1 Framework Overview Diagram



Architecture ของ Stat Arb System — ทุก module เชื่อมกันเป็น pipeline จาก data ถึง execution

20.2 Drawdown Limits

Hierarchy ของ drawdown limits: Level 1 – Per Trade: $\max \text{ loss} = \text{entry_z} \times \sigma_{\text{spread}} \times \text{size} \times 1.5$ # $\times 1.5 = \text{jump-risk buffer}$ Level 2 – Per Day: $\max \text{ daily PnL loss} = 2\%$ ของ total capital Level 3 – Per Strategy: $\max \text{ drawdown} = 10\%$ ของ strategy capital Level 4 – Portfolio: $\max \text{ portfolio drawdown} = 15\%$ → system pause เมื่อ hit limit: Level 1 → close trade Level 2 → close all trades, no new entries ถึงวันถัดไป Level 3 → reduce size 50%, review parameters Level 4 → full stop, manual review required



Drawdown curve — เมื่อแตะ level 4 (-15%) ระบบหยุด รอ manual review และ recalibration

20.3 VaR/CVaR ของ Spread Position

$VaR_{95} = \sigma_{spread} \times 1.645 \times size \times \sqrt{hold_periods}$ $CVaR_{95} = E[loss \mid loss > VaR_{95}] \approx \sigma_{spread} \times 2.063 \times size \times \sqrt{hold_periods}$ (สำหรับ Normal distribution) ตัวอย่าง: $\sigma_{spread} = 0.075\%$ (7.5bps ต่อ period) $size = \$100,000$ $hold = 1$ period $VaR_{95} = 0.00075 \times 1.645 \times 100,000 = \123 $CVaR_{95} = 0.00075 \times 2.063 \times 100,000 = \155 (expected loss ใน worst 5%)

20.4 Black Swan Scenarios

Scenario	Detection	Response	Recovery
Flash crash (>10% in 5m)	price_change > 5× σ_{daily}	Cancel all orders, close positions at market	Wait 30m, recheck cointegration
Exchange outage	API timeout > 30s	Hedge single leg on alternative venue	Close hedge when API restores
Cointegration breakdown	ADF p-value > 0.10 rolling	Enter INVALIDATED state immediately	Refit model with 30-day rolling data
Funding rate spike (>0.5%/8h)	Rate change > 3× historical σ	Reduce size 50%, widen stop-loss	Normalize after 1 funding period

20.5 Pseudo-code risk_check()

```
def risk_check(portfolio, new_trade, limits): """ portfolio: current open
positions + daily PnL new_trade: proposed trade parameters limits: RiskLimits
object (max_size, daily_stop, max_corr, max_var) """ # 1. Check trade size if
new_trade.notional > limits.max_single_size: return "REJECT", "Size exceeds
limit" # 2. Check daily drawdown if portfolio.daily_pnl < -limits.daily_stop:
return "REJECT", "Daily stop hit" # 3. Check correlation with existing
positions for pos in portfolio.positions: corr =
rolling_correlation(new_trade.spread, pos.spread, window=30) # 30 trading days
if corr > limits.max_correlation: return "REJECT", f"Correlation {corr:.2f}
with {pos.id}" # 4. Check portfolio VaR hypothetical = portfolio.positions +
[new_trade] new_var = compute_portfolio_var(hypothetical, confidence=0.95) if
new_var > limits.max_var * portfolio.total_capital: return "REJECT", f"VaR
{new_var:.1%} exceeds limit" # 5. Check edge (from Ch19) edge_result =
compute_edge(new_trade) if not edge_result["viable"]: return "REJECT", f"Net
edge {edge_result['net_edge_pct']:.2f}% <= 0" return "APPROVE", "All checks
passed"
```

20.6 สรุปบทเรียนทั้งหมด

Ch.1–3

พื้นฐาน Synthetic Process

$\varepsilon = \log(P_A) - \beta \cdot \log(P_B)$ คือ spread ที่ mean-revert; OU process อธิบาย dynamics; half-life = $\ln(2)/\kappa$

Ch.4–6

Statistical Foundation

OLS/Kalman β ; Engle-Granger cointegration; Hurst exponent $H < 0.5$ = mean-reverting

Ch.7–9

Advanced OU Models

GARCH conditional volatility; Jump-Diffusion fat tail; Regime-Switching Markov chain

Ch.10

Robust Z-score

Median/MAD robust ต่อ outlier; rolling window z-score; entry/exit band design

Ch.11

State Machine

7-state machine ป้องกัน double-entry, race condition; INVALIDATED เมื่อ coin หาย

Ch.12

Position Sizing

Kelly $f^* = \text{edge}/\sigma^2$; half-life scaling; jump discount; max drawdown constraint

Ch.13

Perpetual Basis

Basis = perp – spot; fair basis จาก carry cost; entry เมื่อ $\text{excess} > k \cdot \sigma$

Ch.14–15

Execution

Cross-venue leg risk; synchronization; Kalman dynamic β real-time update

Ch.16

Portfolio

MV optimal weights; correlation heatmap; equal-weight robust baseline

Ch.17–18

Applications

Commodity calendar spread (storage/convenience yield); Options parity arbitrage

Ch.19

Cost Analysis

Gross – fee – spread – slippage – funding – legging = Net edge; compute before every trade

Ch.20

Risk Management

4-level drawdown limits; VaR/CVaR; black swan procedures; portfolio heat monitoring

20.7 Next Steps

เส้นทางจากทฤษฎีสู่ Live Trading

1. **Backtest ครบทุก module** — test บน historical data ≥ 1 ปี รวม regime ต่างๆ ตรวจสอบว่า Sharpe, max drawdown อยู่ใน target
2. **Paper Trade 1–3 เดือน** — รัน system โดยไม่ใช้เงินจริง บันทึก slippage จริง vs model, latency จริง, execution quality
3. **Live Trading ด้วย Small Size** — เริ่มต้นด้วย 10% ของ target size เพิ่มทีละน้อยเมื่อ P&L confirmed
4. **Monitor และ Recalibrate** — refit OU parameters ทุกสัปดาห์; ตรวจสอบ cointegration ทุกวัน; review regime detection หลัง major events
5. **Expand Strategies** — เพิ่ม spread ใหม่เข้า portfolio เมื่อมีทุนและ operational capacity พอ

Golden Rules ของ Stat Arb

- ไม่มี edge ที่ "ถาวร" — ต้อง monitor และ adapt ตลอดเวลา
- Cost analysis ก่อนทุก trade — ไม่ assume
- Risk limits คือ non-negotiable — ไม่ override เพราะ "ครั้งนี้ต่างออกไป"
- Regime awareness เสมอ — ตลาดมีสถานะต่างกัน parameter ชุดเดียวไม่พอ
- Execution quality กำหนด profitability เท่ากับ signal quality

แบบฝึกหัดสุดท้าย — Capstone

- คำถาม: Bybit และ Lighter ประกาศ merge platform — spread หายไปทันที ระบบควร respond อย่างไร?